VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JNANA SANGAMA" Belagavi, Karnataka - 590 018, INDIA

MAJOR PROJECT (BCS705) Report on

"FortifyHub: A Secure Multi-Tenant Campus Placement, Attendance and Career Development Platform"

Submitted in partial fulfillment of the requirements for the conferment of degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

Submitted by

JAYANTH GOPALA V	**1BY23CS092**
LEELA VENKATA SAI SUMANTH E	**1BY23CS111**
NAGARJUN GOWDA K N	**1BY23CS130**
NAKUL B	**1BY23CS132**

Under the Guidance of

Prof. Beerappa Belasakarge

Assistant Professor, Department of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(An autonomous institution under VTU)

(Accredited By National Assessment & Accreditation Council (NAAC))

(Approved by AICTE, New Delhi & Affiliated to Visvesvaraya Technological University,Belagavi)

Doddaballapura Main Road, Avalahalli, Yelahanka, Bengaluru-560119.

2026-2027

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(An Autonomous Institute under VTU)

(Accredited By National Assessment & Accreditation Council (NAAC))

(Approved by AICTE, New Delhi & Affiliated to Visvesvaraya Technological University,Belagavi)

Doddaballapura Main Road, Avalahalli, Yelahanka, Bengaluru, Karnataka, INDIA - 560 119

CERTIFICATE

This is to certify that the project entitled **"FORTIFYHUB: A SECURE MULTI-TENANT CAMPUS PLACEMENT, ATTENDANCE AND CAREER DEVELOPMENT PLATFORM"** is a bonafide work carried out by **Jayanth Gopala V (1BY23CS092), Leela Venkata Sai Sumanth E (1BY23CS111), Nagarjun Gowda K N (1BY23CS130) & Nakul B (1BY23CS132)** in partial fulfillment for the award of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE AND ENGINEERING** of the **Visvesvaraya Technological University, Belagavi**, during the year 2026–27. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect to work for the B.Tech degree.

Signature of the Guide **Prof. Beerappa Belasakarge** Assistant Professor, Dept. of CSE BMSITM, Bengaluru	**Associate Head** **Dr. Mahesh G** Professor Dept. of CSE, BMSITM
HOD **Dr. SATISH KUMAR T** Professor Dept. of CSE, BMSITM	**Principal** **Dr. SANJAY H A** Professor BMSITM, Bengaluru

External Viva-Voce

Name of the Examiner	**Signature with Date**
1.	.....
2.	.....

INSTITUTE VISION

To emerge as one of the finest technical institutions of higher learning, to develop engineering professionals who are technically competent, ethical and environment friendly for betterment of the society.

INSTITUTE MISSION

Accomplish stimulating learning environment through high quality academic instruction, innovation and industry–institute interface.

DEPARTMENT VISION

To be a centre of excellence in Computer Science and Engineering education and research, nurturing technically competent, ethically responsible, and socially conscious professionals to meet global challenges and drive sustainable innovation.

DEPARTMENT MISSION

M1. To impart quality education in Computer Science and Engineering by integrating fundamental knowledge with emerging technologies and industry practices and research aptitude through a curriculum enriched with project-based learning, professional activities, and collaborative initiatives.

M2. To foster innovation, problem-solving, entrepreneurship and research culture through industry interaction and interdisciplinary learning.

M3. To develop graduates with strong ethical values, leadership qualities, and a commitment to lifelong learning through co-curricular and extra-curricular activities.

PROGRAM EDUCATIONAL OBJECTIVES

PEO 1 – Professional Excellence: Pursue successful careers in industry, academia, and entrepreneurship by applying the foundational knowledge of Computer Science and Engineering with professional competence.

PEO 2 – Higher Education and Lifelong Learning: Engage in higher studies, research, or professional development programs, demonstrating a commitment to lifelong learning in a rapidly evolving technological landscape.

PEO 3 – Ethics and Social Responsibility: Exhibit ethical behaviour, effective communication, teamwork, and leadership qualities, with a strong sense of responsibility toward society and the environment.

PROGRAM SPECIFIC OUTCOMES

PSO 1 : Apply theoretical foundations, Algorithmic principles and software engineering practices to develop efficient and scalable IT solutions.

PSO 2: Design effective systems by leveraging principles of computing and communication technologies.

PROGRAM OUTCOMES

PO 1 : Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO 2 : Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO 3 : Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO 4 : Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO 5 : Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO 6 : The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7)

PO 7 : Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO 8 : Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO 9 : Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO 10 : Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO 11 : Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

ACKNOWLEDGEMENT

We are happy to present this major project report after completing the work successfully. This major project would not have been possible without the guidance, assistance and suggestions of many individuals. We express our deep sense of gratitude to each and every one who has helped us to make this major project a success.

We heartily thank **Dr. Sanjay H A, Principal, BMS Institute of Technology & Management** for his constant encouragement and inspiration in taking up this major project.

We heartily thank **Dr. Satish Kumar T, Head of the Department of Computer Science and Engineering, BMS Institute of Technology & Management**, for his constant encouragement and inspiration in taking up this major project.

We heartily thank **Dr. Mahesh G, Associate Head of CSE Cluster-1, Computer Science and Engineering, BMS Institute of Technology & Management**, for his constant encouragement and inspiration in taking up this major project.

We gratefully thank our project guide, **Prof. Beerappa Belasakarge, Assistant Professor, Department of Computer Science and Engineering**, for his encouragement and advice throughout the course of the major project work.

We heartily thank the project coordinators **Dr. Dhanalakshmi B K and Mrs. Anusha K L, Associate Professor / Assistant Professor, Dept. of Computer Science and Engineering**, for their constant follow-up and advice throughout the major project work.

Special thanks to all the staff and SPARK members of the Computer Science and Engineering Department, and to the Training and Placement Office, for their help, feedback and kind cooperation during requirement gathering and testing.

Lastly, we thank our parents and friends for their encouragement and support given to us in order to finish this major project work.

Jayanth Gopala V (1BY23CS092)

Leela Venkata Sai Sumanth E (1BY23CS111)

Nagarjun Gowda K N (1BY23CS130)

Nakul B (1BY23CS132)

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(An Autonomous Institution affiliated to VTU, Belagavi)

(Accredited By National Assessment & Accreditation Council (NAAC))

(Approved by AICTE, New Delhi & Affiliated to Visvesvaraya Technological University,Belagavi)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Doddaballapura Main Road, Avalahalli, Yelahanka, Bengaluru, Karnataka, INDIA - 560 119

DECLARATION

We hereby declare that the major project titled "**FortifyHub: A Secure Multi-Tenant Campus Placement, Attendance and Career Development Platform**" is a record of original project work undertaken for partial fulfillment of the Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belagavi during the year 2026–2027. We have completed this major project work under the guidance of **Prof. Beerappa Belasakarge, Assistant Professor, Dept. of CSE**.

We also declare that this major project report has not been submitted for the award of any degree, diploma, associateship, fellowship or other titles anywhere else.



Jayanth Gopala V
USN: 1BY23CS092

Signature



Leela Venkata Sai Sumanth E
USN: 1BY23CS111

Signature



Nagarjun Gowda K N
USN: 1BY23CS130

Signature



Nakul B
USN: 1BY23CS132

Signature

ABSTRACT

Campus placement and academic administration in engineering colleges still depend on spreadsheets, e-mail threads, messaging groups and isolated tools. The mini-project phase of this work produced a single-college prototype that proved the value of automated eligibility checks and a sandboxed coding judge. However, the prototype could not be deployed for real institutions: it had no tenant isolation, weak account security, no attendance or reporting modules and no production deployment pipeline.

This report presents **FortifyHub**, the major-project evolution of that prototype into a **secure, multi-tenant Software-as-a-Service (SaaS) platform** that a single installation can offer to many colleges. Adding a new college is pure data entry; the platform automatically provisions the tenant, its default roles and a subscription-aware permission matrix. The system is built as three cooperating services – an **authentication service** (the security boundary), a **core service** implemented as a modular monolith (students, classes, attendance, placement, reports, notifications, e-mail) and a **background worker** (BullMQ) – with a React 18 + Vite frontend, PostgreSQL 16, PgBouncer and Redis 7.

Security is a first-class design goal. Passwords use **argon2id**; access tokens are **RS256 JWTs verified locally** by the core service with the auth service's public key; refresh tokens rotate with **family-based theft detection**; sensitive actions require **step-up e-mail OTP**; users may enrol **TOTP** and **WebAuthn passkeys**. Tenant isolation is enforced twice – by application-level scoping and by **PostgreSQL Row-Level Security (RLS)** with `FORCE ROW LEVEL SECURITY` – and the most sensitive personal fields are **encrypted at rest with AES-256-GCM**. Every privileged action is written to an append-only audit log.

Functionally, the platform provides dynamic role-based access control (RBAC) with per-page permissions, a rule-based eligibility engine, an application state machine, tiered two-offer placement policies, drive rounds and shortlists, barcode-based attendance with database-level de-duplication, placement analytics (department-wise, CTC distribution, year-on-year trends, accreditation reports), bulk e-mail with cost preview, an off-campus opportunity feed protected by an SSRF guard, and a role-aware resume generator whose descriptions are optionally rewritten by a **self-hosted small language model (Qwen3 1.7B via Ollama)** without any personal data leaving the core service. A central **single sign-on (SSO) hub** links the placement platform to the **FortifyHub online examination platform**, where coding answers are graded server-side by a Judge0/isolate sandbox and protected by proctoring and plagiarism checks.

The platform is containerised with Docker, fronted by Cloudflare and Traefik, and delivered through a GitHub Actions CI/CD pipeline. It is validated by an automated smoke and security regression suite (tenant isolation, encryption at rest, token revocation, race conditions, rate limiting) and by a role-by-role acceptance walkthrough on the live production deployment.

Keywords: Multi-Tenant SaaS, Placement Management, Role-Based Access Control, Row-Level Security, JWT (RS256), Argon2id, WebAuthn, Single Sign-On, AES-256-GCM, BullMQ, Online Judge, Small Language Models, Docker, CI/CD

Table of Contents

Acknowledgements i

Abstract iii

List of Figures viii

List of Tables xi

1 Introduction 1

1.1 Background . . . 1

1.1.1 Content and Background . . . 1

1.1.2 Evolution from the Mini Project . . . 1

1.2 Overview . . . 2

1.2.1 Scope of the Project . . . 2

1.2.2 Applications of the Project . . . 3

1.3 Motivation . . . 3

1.3.1 Motivation for the Study . . . 3

1.4 Problem Statement . . . 4

1.4.1 Problem Identified . . . 5

1.5 Overall Objective of the Project . . . 6

1.5.1 Objectives of the Project . . . 6

1.6 Summary . . . 7

1.7 Organization of the Report . . . 7

2 Literature Survey 8

2.1 Review of Existing Work . . . 8

2.1.1 Related Systems . . . 8

2.1.2 Review of Security Standards and Enabling Technologies . . . 9

2.2 Comparative Analysis of Existing Systems . . . 10

2.2.1 Comparative Review of Existing Work . . . 10

2.3 Research Gap Analysis . . . 14

2.3.1 Gap Analysis . . . 14

2.4 Specific Objectives of the Project after Gap Analysis . . . 15

2.5 Proposed Solution . . . 15

2.6 Summary . . . 15

3 System Requirement Specification 17

3.1 Overall Description . . . 17

3.1.1 Product Perspective . . . 17

3.1.2 User Classes: Major Stakeholders and Roles . . . 17

3.1.3 Operating Environment and Deployment Scenarios . . . 18
3.1.4 Operational Assumptions and Constraints . . . 18
3.2 Functional Requirements . . . 18
3.3 Non-Functional Requirements . . . 19
3.4 Hardware Requirements . . . 20
3.5 Software Requirements . . . 20
3.6 Feasibility Analysis . . . 21
3.7 Summary . . . 21

4 System Design and Architecture 22
4.1 System Architecture . . . 22
4.1.1 High-level Architecture . . . 22
4.1.2 Integrated Ecosystem . . . 24
4.1.3 Component Interaction . . . 25
4.1.4 Scalability Considerations . . . 25
4.1.5 Design Patterns Used . . . 26
4.2 Hardware Architecture . . . 26
4.3 Software Module Design . . . 27
4.3.1 High-level Functional Modules . . . 27
4.3.2 Detailed Module Descriptions . . . 28
4.4 Security and Access Control . . . 35
4.4.1 Authentication . . . 35
4.4.2 Authorization . . . 36
4.4.3 Tenant Isolation and Data Protection . . . 36
4.5 Database Design . . . 37
4.5.1 Core Entities and Relationships . . . 37
4.5.2 Data Storage and Consistency . . . 38
4.5.3 Simplified ER Diagram . . . 39
4.5.4 Detailed ER Diagram – Identity Database (`auth_db`) . . . 40
4.5.5 Detailed ER Diagram – Domain Database (`core_db`) . . . 41
4.6 Data Flow Diagram (Level 0 and Level 1) . . . 42
4.6.1 Level 0 – Context Diagram . . . 42
4.6.2 Level 1 . . . 43
4.7 Use Case Diagram . . . 44
4.8 Activity Diagram . . . 45
4.9 Sequence Diagram / Transition Diagram . . . 46
4.9.1 Request Processing Sequence . . . 46
4.9.2 Single Sign-On Sequence . . . 47
4.9.3 Application State Transition Diagram . . . 48

5 System Methodology 49
5.1 Proposed Methodology . . . 49
5.1.1 Requirement Analysis . . . 49

5.1.2 System Design . . . 49
5.1.3 Incremental Implementation . . . 50
5.1.4 Testing and Deployment . . . 50
5.1.5 System Overview . . . 50
5.1.6 Contributions . . . 51
5.2 Algorithms . . . 51
5.3 Explanation of the Algorithms . . . 55
5.3.1 Eligibility Checking . . . 55
5.3.2 Application Workflow State Machine . . . 55
5.3.3 Tiered Two-Offer Placement Policy . . . 55
5.3.4 Refresh Token Rotation with Theft Detection . . . 55
5.3.5 Single Sign-On with One-Time Codes . . . 55
5.3.6 Attendance Marking with an Exactly-Once Guarantee . . . 56
5.3.7 Readiness and Engagement Indicators . . . 56
5.3.8 Resume Project Relevance Heuristic . . . 56
5.3.9 Server-side Code Judging . . . 57
5.3.10 SSRF Validation for External Job Feeds . . . 57
5.4 Pseudocode . . . 57
5.4.1 Eligibility Checking . . . 57
5.4.2 Application Workflow State Machine . . . 58
5.4.3 Tiered Two-Offer Placement Policy . . . 58
5.4.4 Refresh Token Rotation with Theft Detection . . . 58
5.4.5 Single Sign-On with One-Time Codes . . . 59
5.4.6 Attendance Marking with an Exactly-Once Guarantee . . . 59
5.4.7 Server-side Code Judging . . . 59
5.4.8 SSRF Validation for External Job Feeds . . . 60

6 Implementation and Experimental Setup 61
6.1 Test Cases . . . 61
6.2 Data Set . . . 62
6.3 Experimental Setup . . . 62
6.3.1 System Implementation Overview . . . 62
6.3.2 Repository Structure . . . 63
6.3.3 Deployment and CI/CD Implementation . . . 63
6.4 Implementation of the Modules . . . 64
6.4.1 Authentication Service Implementation . . . 64
6.4.2 Core Service Implementation . . . 67
6.4.3 Placement Module Implementation . . . 70
6.4.4 Attendance Module Implementation . . . 73
6.4.5 Background Worker Implementation . . . 74
6.4.6 Resume Generation and AI Service . . . 74
6.4.7 Frontend Implementation . . . 75

6.4.8 Examination Platform Integration 76
6.5 Description of Usage 77

7 System Testing **78**
Testing Strategy 78
7.1 Unit Testing 78
7.2 Functional Testing 79
7.2.1 Placement System Testing 80
7.3 Integration Testing 83
7.3.1 Automated Test Suite 83
7.3.2 Coding Assessment System Testing (Examination Platform Integration) 84
7.3.3 Performance and Load Testing 85
7.4 Acceptance Testing 85
7.4.1 User Acceptance Testing 85
7.5 Defect Summary 86

8 Results and Discussions **88**
8.1 Functional Achievements 88
8.2 Performance Improvements 89
8.3 Comparison with the Mini-project Prototype 89
8.4 Graphical Analysis of Test Results 89
8.5 Quantitative Comparison with the Prototype 90
8.6 Code Base Composition 90
8.7 Justification of Requirements 91
8.8 Comparison with Existing Systems 92
8.9 Visual Results of UI Implementation 94
8.9.1 Authentication and Single Sign-On 94
8.9.2 Platform and Tenant Administration 95
8.9.3 Platform Console and Feature Management 96
8.9.4 Subscriptions and Test Reports 98
8.9.5 Student Exam Experience 99
8.9.6 Attendance 100
8.10 Visual Results for Coding Assessment Module 100
8.10.1 Exam Creation and Question Bank 101
8.10.2 Deployment 102
8.10.3 Server Monitoring and Deployment History 103
8.11 Discussion 104

Conclusion and Future Enhancements **105**

References **106**

APPENDIX A - Abbreviations **109**

APPENDIX B - Complex Engineering Problem **111**

APPENDIX C - Sustainable Development Goal Addressed **114**

APPENDIX D - Research Publications/Patent **115**

APPENDIX E - Plagiarism Report (Similarity Report and AI Detection) **122**

APPENDIX F - API Examples **123**

APPENDIX G - Local Development Setup **124**

APPENDIX H - Security Controls Checklist **125**

List of Figures

Sl. No.	Caption	Pg. No.
4.1	Deployment architecture of the placement platform (dashed: internal-only calls and backups)	23
4.2	Integrated FortifyHub ecosystem: SSO hub, placement platform, examination platform and AI service	24
4.3	Simplified ER diagram of the core placement and attendance entities	39
4.4	Detailed entity-relationship diagram of the identity database (`auth_db`)	40
4.5	Detailed entity-relationship diagram of the domain database (`core_db`)	41
4.6	Data flow diagram – Level 0 (context diagram)	42
4.7	Data flow diagram – Level 1	43
4.8	Use case diagram of the FortifyHub placement platform	44
4.9	Activity diagram – student applying to a placement drive	45
4.10	Request processing and authorization pipeline in the core service	46
4.11	Central single sign-on flow with a single-use, platform-bound code	47
4.12	Application life-cycle state machine	48
6.1	Continuous integration and deployment pipeline	64
8.1	Test case outcomes by group	90
8.2	Prototype versus major project on measurable metrics	90
8.3	Lines of code per component	91
8.4	Login page with email/password and ID-card sign-in options	94
8.5	Central SSO hub with platform picker	94
8.6	Super admin – colleges (tenants) and subscriptions	95
8.7	Role permission matrix showing per-role View, Add, Update and Remove permissions	95
8.8	Append-only audit log	96
8.9	Exam platform console: colleges, students and live attempts	96
8.10	Job-feed providers powering the shared off-campus opportunities board	97
8.11	Page catalog of plan-gated feature pages	97
8.12	Subscription plans and tenant subscriptions	98
8.13	Post-exam test report for the first live BMSIT exam	98
8.14	Student home showing open practice tests and profile details	99
8.15	Practice test introduction with question count, autosave and unlimited attempts	99
8.16	ID-card based sign-in used for exam-day attendance	100
8.17	Examination platform coding interface	100
8.18	Exam management with status, schedule, duration and question count	101
8.19	Question bank with type, difficulty, topic and marks filters	101
8.20	Server-side execution result for a sample test case	102

8.21 Production services running under Dokploy 102
8.22 Dokploy server monitoring of CPU, memory, disk and Docker usage 103
8.23 Dokploy deployment history of the production services 103

List of Tables

Sl. No.	Caption	Pg. No.
1.1	Mini-project prototype versus major-project platform	1
2.1	Standards and technical literature applied in the design	9
2.2	Comparative review of existing work on student placement systems and prediction models (Papers 1–5)	13
2.3	Comparative review of existing work on student placement systems and recruitment platforms (Papers 6–10)	14
3.1	Stakeholders, roles and responsibilities	17
4.1	Technology stack used for implementation	27
5.1	Implementation phases and goals	50
6.1	Test case groups and outcomes	61
6.2	Approximate size of the placement platform code base (lines, excluding dependencies)	63
7.1	Units under test and the tests that verify them	79
7.2	Authentication and access control test cases	80
7.3	Multi-tenancy and data protection test cases	81
7.4	Student portal functional test cases	81
7.5	Administration and placement office test cases	82
7.6	Attendance and background processing test cases	83
7.7	Automated security regression groups	84
7.8	Coding verdict and integrity test cases	84
7.9	Role-based acceptance walkthrough on the production deployment	86
7.10	User feedback summary (ratings out of 5)	86
7.11	Summary of key failed test cases and their resolution	86
8.1	Summary of achieved functional objectives	88
8.2	Measured scope of the prototype versus the major project	89
8.3	Justification of functional requirements	91
8.4	Justification of non-functional requirements	92
8.5	Feature comparison with existing systems	92
8.6	Summary of WP competencies matched	112
8.7	Mapping of the project with UN Sustainable Development Goals	114
8.8	Mapping of OWASP API Security Top 10 (2023) risks to implemented controls	125

Chapter 1

Introduction

1.1 Background

1.1.1 Content and Background

Placement, attendance and student-record processes in educational institutions require continuous coordination between students, class teachers, proctors, heads of departments, placement officers, principals and recruiters. Historically, most institutions have used a combination of spreadsheets, e-mail, messaging groups and local databases for this purpose. The result is fragmented data, duplicate effort and slow communication.

In the mini-project phase, the team built the *College Placement and Career Development Platform*, a single-college prototype with job postings, eligibility checks and a Docker-based coding sandbox. The prototype demonstrated that automation can dramatically reduce shortlisting effort, but it was designed for one institution, used simplified authentication and was deployed only with Docker Compose on a development machine.

1.1.2 Evolution from the Mini Project

Table 1.1 summarises how the major project extends the mini-project prototype.

Table 1.1: Mini-project prototype versus major-project platform

Aspect	**Mini Project (Prototype)**	**Major Project (FortifyHub)**
Tenancy	Single college	Many colleges in one deployment; tenant provisioning by data entry
Architecture	GraphQL gateway + loosely defined services	Auth service + core modular monolith + worker; clean module boundaries
Authentication	Basic JWT login	argon2id, RS256 JWT, refresh rotation with theft detection, TOTP, passkeys, step-up OTP, SSO
Authorization	Fixed roles	Dynamic roles, per-page/per-action permissions, subscription-gated pages
Data isolation	Not applicable	App-level scoping + PostgreSQL Row-Level Security (FORCE)
Data protection	Plain columns	AES-256-GCM encryption of sensitive fields, audit log, redaction

Table 1.1 (continued)

Aspect	Mini Project (Prototype)	Major Project (FortifyHub)
Modules	Jobs, applications, coding sandbox	+ attendance, classes, departments, rounds, offers, policies, analytics, reports, e-mail, resume, off-campus feed
Assessments	Local Docker judge (Go)	Integrated examination platform with Judge0/isolate server-side grading, proctoring, plagiarism checks
Background work	None	BullMQ worker: CSV import, reports, files, e-mail, maintenance
Deployment	Docker Compose on a laptop	Cloudflare + Traefik + Dokploy, PgBouncer, backups to R2, GitHub Actions CI/CD
Testing	Manual test cases	Automated smoke + security regression suite and production walkthrough

1.2 Overview

The major project, **FortifyHub**, re-engineers the system from the ground up as a **production-grade, multi-tenant SaaS platform**. One deployment serves many colleges (tenants), each with its own users, roles, subscription plan and strictly isolated data. The first two target institutions are engineering colleges in Bengaluru with roughly 4,000–8,000 students each, which sets the scale and the security bar for the design.

1.2.1 Scope of the Project

The scope of the major project is to design, implement, deploy and validate a web-based, multi-tenant platform. The system:

- supports a platform **super admin** who on-boards colleges, manages subscription plans, add-ons and the global page catalogue without code changes;
- supports tenant roles (tenant admin, principal, HOD, teacher, proctor, placement officer, placement head, student) and custom roles with per-page, per-action permissions;
- manages departments, classes, student profiles (academics, projects, internships, skills, certifications, documents) and bulk CSV import;
- manages attendance sessions with barcode/ID-card scanning and manual marking;
- manages companies, jobs (drives), eligibility rules, applications, rounds, shortlists, offers, drive categories and placement policies;

- provides analytics, accreditation-style reports, announcements, notifications, web-push and bulk e-mail;
- generates role-targeted resumes, optionally enhanced by a self-hosted language model;
- integrates with the FortifyHub examination platform (coding and MCQ assessments) through single sign-on and account provisioning.

Native mobile applications, payment-gateway billing, Kubernetes-based autoscaling and full AI-driven job matching are outside the current scope and are listed under future enhancements.

1.2.2 Applications of the Project

FortifyHub can be applied in several contexts:

- **College Training and Placement Office (TPO):**
  Day-to-day management of placement drives, company visits, eligibility, rounds, offers, selection notices and placement policy.
- **Academic Departments and Class Teachers:**
  Attendance sessions, class rosters, student profiles, engagement tracking and department-wise placement monitoring.
- **Institution Management and Accreditation:**
  Principal and HOD dashboards, year-on-year trends and printable accreditation reports (for example, for NAAC/NBA documentation).
- **Multi-institution SaaS Offering:**
  A single deployment serves many colleges under Basic, Gold and Premium subscription tiers with trials and add-ons.
- **Skill Development and Assessment Programmes:**
  Coding contests, placement-preparation tests and practice assessments through the integrated examination platform, plus preparation resources and resume generation.
- **Alumni and Industry Engagement:**
  Off-campus opportunity feeds, company documents and recruiter communication channels.

1.3 Motivation

1.3.1 Motivation for the Study

Three observations motivated the major project:

- **Institutions need one system, not many.** Placement, attendance and student profiles use the same student master data. Keeping them in separate tools causes inconsistent records and repeated data entry.
- **Student data is sensitive.** Placement platforms hold names, contact details, academic records, identity numbers and resumes of thousands of minors and young adults. A breach or a cross-college data leak is unacceptable, so security has to be engineered in, not added later.
- **Software must be sellable and operable.** A college platform must be cheap to run, easy to on-board (no code change per college), safe to upgrade and able to survive peak loads such as result announcements or a placement drive.

A secure, modular and multi-tenant platform can reduce manual effort, improve transparency, protect student data and support better career outcomes, which motivates the design and implementation of FortifyHub.

1.4 Problem Statement

The campus placement and academic-administration process in many institutions is fragmented, largely manual and insecure. Student data, attendance, job openings, eligibility rules, assessment results and communication are spread across spreadsheets, e-mail and unrelated portals, and existing software is usually built for one institution with coarse permissions and little protection of personal data.

The major problems observed are:

- **High manual workload:** Placement and department staff repeatedly verify records, check eligibility, take attendance and prepare reports by hand.
- **Scattered information:** Academic details, resumes, skills, attendance and assessment scores exist in different systems, causing inconsistency during shortlisting and accreditation.
- **Low student transparency:** Students have no single place to track eligibility, application status, rounds, offers, policies and announcements.
- **No integrated testing system:** Coding and aptitude tests run on external tools that do not share identity or data with the placement workflow.
- **Minimal analytics:** Institutions lack consolidated insights on placement percentage, CTC distribution, department performance, engagement and year-on-year trends.
- **Weak security and privacy:** Shared admin accounts, no audit trail, weak password storage and no protection against one college seeing another college's data.

- **Limited scalability and operability:** Spreadsheet-based or single-tenant systems cannot be offered to many colleges or survive peak loads.

Therefore, this project aims to develop a secure, multi-tenant and scalable campus placement, attendance and career development platform that centralises all these activities by offering:

- tenant provisioning with subscription-aware features and no per-college code changes,
- dynamic role-based access control with per-page and per-action permissions,
- unified student profiles, classes, departments and attendance,
- automated eligibility validation and policy-aware application workflows,
- secure coding assessments through an integrated examination platform,
- real-time notifications, announcements and bulk e-mail,
- analytics dashboards and accreditation reports, and
- defence-in-depth security: strong authentication, database-enforced isolation, encryption at rest and auditing.

This platform reduces manual effort, improves transparency, protects student data and supports efficient, data-driven placement and academic management.

1.4.1 Problem Identified

Despite the importance of placements and attendance, most colleges still rely on:

- manual tracking of student data, attendance and applications in spreadsheets,
- ad-hoc communication through e-mail, phone calls, notices and messaging groups,
- isolated assessment tools that are not linked to student profiles or placement drives,
- generic portals with a single “admin” account, no fine-grained permissions and no audit trail,
- single-institution software that must be copied and re-configured for every new college.

This ecosystem causes inconsistent records, difficulty in enforcing eligibility and offer policies, slow shortlisting, poor visibility for students and, most importantly, weak protection of personal data. There is no single platform that combines placement workflows, attendance, secure assessments, analytics and strong multi-tenant security.

1.5 Overall Objective of the Project

1.5.1 Objectives of the Project

The key objectives of the major project are:

- **Design a multi-tenant SaaS architecture** in which a new college is added by data entry and each college's data is isolated both in application code and in the database.
- **Build a hardened authentication service** with argon2id hashing, RS256 JWTs, rotating refresh tokens with theft detection, multi-factor authentication (TOTP, passkeys), step-up verification, account lockout, rate limiting and single sign-on.
- **Implement dynamic RBAC** where tenant administrators create roles and grant view, create, edit, delete and export actions on individual pages, limited by the college's subscription plan.
- **Centralise academic and placement activities** – students, classes, attendance, companies, jobs, eligibility, applications, rounds, offers and policies – in one consistent data model.
- **Automate eligibility validation and placement policies** (CGPA, 10th/12th percentage, backlogs, gap years, batch, semester, department, skills, resume presence, policy acknowledgement and tiered offer limits).
- **Provide analytics and reports** for placement officers, principals and accreditation, generated asynchronously by a background worker.
- **Enable secure technical assessments** by integrating the placement platform with an examination platform that grades code server-side in an isolated sandbox.
- **Protect personal data** through encryption at rest, append-only audit logging, SSRF protection and least-privilege database users.
- **Deploy and operate the system in production** with containers, a reverse proxy, connection pooling, backups and a CI/CD pipeline, and validate it with automated security regression tests.

These objectives collectively aim to reduce administrative overhead, enhance data reliability and security, improve student preparedness and support a smooth end-to-end placement experience for all stakeholders.

1.6 Summary

This chapter introduced the background of campus placement and academic administration, the evolution of the mini-project prototype into the multi-tenant FortifyHub platform, its scope and applications, the motivation behind the work, the problem statement and the objectives of the project. In short, the project replaces fragmented, manual and insecure processes with one secure platform that a single deployment can offer to many colleges.

1.7 Organization of the Report

Chapter 2 reviews existing work, compares existing systems, identifies the research gap and states the specific objectives and proposed solution. Chapter 3 presents the system requirement specification together with the feasibility analysis. Chapter 4 describes the system design and architecture: software modules, security and access control, database design and the data-flow, use-case, activity and sequence diagrams. Chapter 5 presents the proposed methodology and the algorithms used, each written as steps, explained and given as pseudocode. Chapter 6 describes the test cases, data set, experimental setup and implementation. Chapter 7 presents unit, functional, integration and acceptance testing, and Chapter 8 the results with graphs, requirement justification and comparison with existing systems. The report then concludes with future enhancements. Appendix B maps the project to complex engineering problem attributes and Appendix C to the UN Sustainable Development Goals.

Chapter 2

Literature Survey

2.1 Review of Existing Work

Existing platforms provide portions of the required functionality, but none deliver the complete set of features needed for a secure, multi-tenant, analytics-driven campus platform. Common categories include coding practice portals, placement (TPO) portals, learning management systems (LMS) and general recruitment platforms.

- Coding practice and challenge websites (HackerRank, LeetCode) provide strong coding evaluation capabilities but lack institutional workflows and student–job eligibility mapping.
- Placement TPO portals manage job postings and basic workflows but rarely support integrated assessments, analytics, fine-grained permissions or multi-college tenancy.
- LMS solutions excel at delivering academic content but do not address placement pipelines, shortlisting rules or recruiter-side operations.

The proposed platform bridges these gaps by integrating rule-based shortlisting, role-specific dashboards, attendance, cohort analytics and a secure assessment platform into a single ecosystem, built on a hardened multi-tenant security architecture.

2.1.1 Related Systems

A survey of existing systems highlights four categories that informed the design:

- **Coding Judges:**
  Online judges isolate untrusted code, run multiple test cases and return verdicts. Judge0 [28] is an open-source, scalable execution system built on the `isolate` sandbox. Such systems emphasise isolation, time and memory limits and language support, but lack institutional workflows such as eligibility filtering and drive mapping.
- **LMS and TPO Portals:**
  Learning management systems and placement portals manage student information and job postings, but rarely include dynamic eligibility rules, offer policies, attendance, audit logs or integrated coding assessments.
- **Recruitment Platforms:**
  Generic applicant-tracking systems support job postings and recruiter dashboards, but

are not tailored for campus settings where batch-wise rules, multi-round drives, faculty involvement and institutional policies are essential.

- **Multi-Tenant SaaS Platforms:**
  Literature on multi-tenancy [23, 24] describes shared-database/shared-schema designs where a tenant identifier on every row provides isolation at low cost. The same pattern, reinforced with database-level Row-Level Security [25], is adopted in this project.

2.1.2 Review of Security Standards and Enabling Technologies

Because the major project is a production multi-tenant system, the design was also guided by published standards and established technical literature. Table 2.1 summarises these works and how each one is applied in FortifyHub.

Table 2.1: Standards and technical literature applied in the design

Sl.	Standard / Work	Key Idea	Application in FortifyHub
1	RFC 7519 – JSON Web Token [13]	Compact, signed claims for stateless authentication.	RS256-signed access tokens (15 min) verified locally by the core service.
2	RFC 6749 and RFC 9700 – OAuth 2.0 and its Security BCP [14, 15]	Short-lived codes, refresh-token rotation and reuse detection.	SSO one-time code exchange; rotating refresh tokens with family revocation on reuse.
3	RFC 9106 – Argon2 [16]	Memory-hard password hashing resistant to GPU attacks.	argon2id password hashing with tuned memory/time cost and lazy re-hashing.
4	RFC 6238 – TOTP [17]	Time-based one-time passwords as a second factor.	Optional TOTP enrolment with QR code and recovery codes.
5	W3C WebAuthn Level 2 [18]	Phishing-resistant public-key credentials (passkeys).	Passkey registration and login for staff and students.
6	NIST SP 800-63B [19]	Guidance on authenticators, throttling and lockout.	Account lockout after failed attempts, per-endpoint rate limits, step-up verification.
7	NIST SP 800-38D – AES-GCM [20]	Authenticated encryption with 96-bit nonces.	Encryption at rest of sensitive student fields with tamper detection.
8	Sandhu et al.; Ferraiolo et al. – RBAC [21, 22]	Permissions attached to roles; users assigned to roles.	Dynamic roles with per-page and per-action permissions per tenant.

Sl.	Standard / Work	Key Idea	Application in FortifyHub
9	Guo et al.; Chong et al. – Multi-tenancy [23, 24]	Shared schema with a tenant identifier; isolation and customisation trade-offs.	Shared-schema design with `tenant_id` on every row and subscription-based customisation.
10	PostgreSQL Row Security Policies [25]	Database-enforced row filtering per session.	`FORCE ROW LEVEL SECURITY` policies keyed on a transaction-local tenant setting.
11	OWASP API Security Top 10 and SSRF Prevention Cheat Sheet [26, 27]	Common API risks: broken object-level authorization, SSRF, mass assignment.	Tenant-scoped lookups, Joi `stripUnknown`, SSRF guard for external job feeds.
12	Dosilovic and Mekterovic – Judge0 [28]	Robust, scalable online code execution on the isolate sandbox.	Server-authoritative grading in the examination platform.
13	Schleimer et al. – Winnowing [29]	Document fingerprinting for plagiarism detection.	Basis for similarity checks (shingling / Jaccard) in the examination platform.
14	Qwen3 Technical Report [30]	Small open-weight language models with a non-thinking mode.	Self-hosted Qwen3 1.7B (via Ollama) for resume description rewriting.
15	Newman – Building Microservices [31]	Service boundaries; modular monolith as a starting point.	Separate auth service; core as a modular monolith with extractable modules.

2.2 Comparative Analysis of Existing Systems

2.2.1 Comparative Review of Existing Work

A review of research papers, prediction models and recruitment systems was undertaken to understand existing solutions for campus placements. The surveyed works fall into four categories: prediction-focused models, deep learning and recommendation systems, recruitment portals and emerging experimental approaches. Tables 2.2 and 2.3 compare ten representative papers [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]. The key observations are summarised below.

1. Early Machine Learning Models (Papers 2–3)

These works predict whether a student will be placed from academic and behavioural data.

- Use classical ML models such as Naive Bayes, Decision Trees, Logistic Regression and PCA-based classifiers.
- Achieve moderate to high accuracy (71%–95%).

- Strengths:
  - Improve understanding of student performance factors.
  - Useful for counsellors and academic insights.
- Limitations:
  - Not connected to real placement workflows.
  - No job posting, application tracking or recruiter coordination.
  - Offline analytical tools only; no real-time system.

2. Deep Learning and Recommendation-Based Systems (Papers 4–5)

Recent research attempts to enhance prediction accuracy and provide skill recommendations.

- Techniques used: LSTM, RNN and feed-forward networks; skill recommendation engines using ML classifiers.
- Outcomes: high accuracy (up to 98%) and personalised recommendations for skill improvement.
- Limitations:
  - Treat placement purely as a prediction problem.
  - Do not include eligibility rules, job workflows or multi-round assessments.
  - No support for coding evaluations, access control or recruiter dashboards.

3. Recruitment Portals and Placement Management Systems (Papers 7–9)

These systems aim to digitise campus recruitment operations.

- Offer registration, job postings, company coordination and interview scheduling.
- Benefits: reduce paperwork, improve communication and provide basic dashboards.
- Major drawbacks:
  - Lack predictive analytics and readiness indicators.
  - No coding sandbox or secure code execution.
  - Mostly monolithic, single-institution and not security-hardened.
  - Limited automation and no integration of eligibility or offer policies.

4. Emerging Research Directions (Papers 6 and 10)

- **Paper 6: Emotional Quotient (EQ)-based prediction** – considers non-cognitive traits; interesting conceptually but with limited empirical validation.
- **Paper 10: Graph Neural Network (GNN)-based recommendations** – links students, skills and companies in a graph; promising accuracy but not deployed in real college systems.
- Limitations (both papers): prototype-level works with no integration into placement workflows, no multi-role dashboards and no secure execution layer.

5. Common Limitations Identified Across All Systems

- Small, static datasets limit generalisation of ML models.
- Lack of an end-to-end workflow (job posting $\rightarrow$ eligibility $\rightarrow$ assessments $\rightarrow$ rounds $\rightarrow$ offers $\rightarrow$ analytics).
- No containerised or server-authoritative coding evaluation.
- No support for multi-tenant usage, resume generation, real-time notifications, attendance or audit trails.
- Security is rarely discussed: password storage, token handling, MFA and data isolation are absent from all surveyed systems.
- Poor scalability – most systems use monolithic, single-institution architectures.

6. Need for a Unified, Modern Placement Platform

A next-generation platform must integrate:

- Secure, server-side graded coding assessments.
- Automatic eligibility evaluation and policy enforcement.
- Unified dashboards for students, faculty, placement officers, management and recruiters.
- Cohort-level analytics, trends and accreditation reports.
- Real-time communication through notifications, announcements and e-mail.
- A scalable, multi-tenant architecture with strong authentication, fine-grained authorization and encryption at rest.

Conclusion

Existing systems contribute valuable ideas, but none provides a comprehensive, scalable and secure solution for multi-institution placement operations. FortifyHub fills this gap by integrating assessments, workflows, analytics, attendance and automation into one ecosystem protected by a defence-in-depth security model.

Table 2.2: Comparative review of existing work on student placement systems and prediction models (Papers 1–5)

Sl.	Paper Title, Authors, Venue / Year	Methodology / Algorithm / Technology Used	Measurable Parameters	Outcomes (Efficiency / Accuracy / Results)	Limitations	Gaps Identified
1	*Campus Ease: Elevating Student Placements and Career Success*, M. Mahendru et al., IEEE ICEECT, 2024.	Mobile application using Flutter and Dart with Firebase backend; Java and Spring Boot services; guided workflows, notifications and centralised placement resources.	No explicit quantitative metrics; qualitative evaluation based on features and user feedback.	Better coordination and perceived efficiency in career preparation; numeric improvements not provided.	Scalability, security, analytics and coding assessments are not discussed.	No analytics dashboards, coding sandbox or multi-tenant architecture.
2	*Student Placement Analyzer: A Recommendation System Using Machine Learning*, S. K. Thangavel et al., IEEE ICACCS, 2017.	Educational data mining with Fuzzy C-Means clustering and a Naive Bayes classifier to predict placement categories.	Prediction accuracy and comparison with WEKA classifiers.	About 71.66% accuracy on a small dataset; decision trees in WEKA reach about 84%.	Offline prediction; small dataset; no real-time portal.	No integration with job posting, applications or assessments.
3	*Predicting Student Placement using PCA and Machine Learning Technique*, V. Hegde et al., IEEE ICCCNT, 2023.	PCA for feature reduction followed by logistic regression, decision tree and random forest.	Accuracy, confusion matrix, training time before and after PCA.	Best model close to 95% accuracy with fewer features.	Single institution; deployment and usability not covered.	Purely predictive analytics; no workflow or dashboards.
4	*Student Placement Prediction using Deep Learning Techniques*, T. Maragatham et al., IEEE ICECA, 2024.	Feed-forward, RNN and LSTM models compared with classical ML.	Accuracy, precision, recall, F1 and loss curves.	Some models reach around 98% accuracy.	Needs more data and compute; hard to interpret; no deployment.	Model accuracy only; no end-to-end placement management.
5	*Student Placement Prediction and Skill Recommendation System using Machine Learning Algorithms*, R. Kadu et al., IEEE ICICT, 2024.	ML classifiers for placement probability plus a skill/course recommendation engine.	Prediction accuracy and qualitative relevance of recommendations.	80–90% accuracy depending on classifier.	Advisory tool only; no multi-role portal or sandboxed assessments.	Not integrated with institutional portals; no cohort analytics.

Table 2.3: Comparative review of existing work on student placement systems and recruitment platforms (Papers 6–10)

Sl.	Paper Title, Authors, Venue / Year	Methodology / Algorithm / Technology Used	Measurable Parameters	Outcomes (Efficiency / Accuracy / Results)	Limitations	Gaps Identified
6	*Student Placement Probabilistic Assessment Using Emotional Quotient with Machine Learning: A Conceptual Case Study*, N. Kathirisetty et al., IEEE Access, 2023.	Conceptual framework combining probabilistic ML with Emotional Quotient attributes.	Change in prediction quality when EQ features are added.	EQ attributes improve discrimination between high and low placement probability.	Mostly conceptual; EQ measurement subjective.	EQ assessment not integrated into operational platforms.
7	*Placement Management System using Machine Learning*, T. Ajay et al., IJRAR, 2023.	Web portal with ML models recommending suitable jobs.	Model accuracy and basic usability feedback.	Close to 90% accuracy; faster shortlisting in case studies.	Monolithic, not containerised; no coding assessments.	No secure execution, multi-tenancy or readiness dashboards.
8	*Placement Management System for Campus Recruitment*, A. Kumar et al., IJISRT, 2020.	MVC web portal with a relational database for registrations, postings and selections.	Number of users and qualitative time savings.	Reduces paperwork and centralises information.	No ML, recommendation or coding evaluation; basic reports.	Digitises existing workflows without analytics or modern architecture.
9	*Integrated Campus Recruitment Platform (PlaceMe)*, M. Sinthuja, IJRASET, 2024.	Full-stack recruitment system with registration, postings, interview scheduling and notifications.	User satisfaction survey and automated task counts.	Improves communication and reduces missed updates.	No predictive analytics or sandboxed assessments; security not analysed.	No AI features or external judge integration.
10	*Smart Campus Placement System using Graph Neural Networks*, A. Pathak et al., TIJER, 2024.	Graph of students, skills, courses and companies with GNN-based recommendations.	Top-k accuracy and hit rate versus collaborative filtering.	Higher accuracy and hit rate on sparse data.	Prototype; complex graph maintenance.	No complete, secure and scalable platform.

2.3 Research Gap Analysis

2.3.1 Gap Analysis

From analysing the above systems and studying placement workflows, the following gaps were identified:

- Lack of integration between coding/aptitude assessments and eligibility-based application workflows.
- Limited support for cohort-level analytics, year-on-year trends, engagement tracking and accreditation reporting.
- Absence of tools to administer multi-round drives, tiered offer policies, policy acknowledgement and automated notifications.
- No secure, server-authoritative code execution in existing placement systems (client-reported results can be forged).
- No multi-tenant support, fine-grained RBAC, strong authentication (MFA, passkeys), encryption at rest or audit trails.

- No privacy-preserving use of AI for resumes and no event-driven background processing.

The proposed platform addresses these gaps with a multi-tenant architecture that integrates assessments, eligibility rules, analytics, attendance and communication workflows under a defence-in-depth security model.

2.4 Specific Objectives of the Project after Gap Analysis

Based on the gaps identified above, the specific objectives of the project are:

1. **Unify placement workflows with secure assessments:** connect eligibility-based drives, multi-round shortlisting and offers with server-authoritative coding assessments under one identity (single sign-on and account provisioning).
2. **Make the platform multi-tenant and secure by design:** isolate every college's data in both the application and the database, and protect accounts and personal data with fine-grained RBAC, multi-factor authentication, passkeys, encryption at rest and an append-only audit trail.
3. **Automate placement rules and reporting:** enforce eligibility, policy acknowledgement and tiered offer limits automatically, notify eligible students, and generate cohort analytics, year-on-year trends and accreditation reports in the background.
4. **Add privacy-preserving career support:** generate role-targeted resumes with a self-hosted language model, without any personal data leaving the platform.

2.5 Proposed Solution

The proposed solution, **FortifyHub**, is a secure, multi-tenant SaaS platform in which one deployment serves many colleges. An authentication service (argon2id, RS256 JWTs, rotating refresh tokens with theft detection, TOTP, passkeys, step-up OTP and SSO) forms the security boundary; a core service built as a modular monolith manages students, classes, attendance, companies, drives, eligibility, applications, rounds, offers, policies, analytics and communication; and a BullMQ worker handles imports, reports, e-mail and maintenance. Tenant isolation is enforced twice, by application-level scoping and by PostgreSQL Row-Level Security, sensitive fields are encrypted with AES-256-GCM, and the platform is linked through a central SSO hub to an examination platform that grades code server-side in a Judge0/isolate sandbox.

2.6 Summary

This chapter reviewed coding judges, placement and learning portals, recruitment platforms, multi-tenant SaaS literature, ten research papers and the security standards that guide the de-

sign. None of the surveyed systems combines placement workflows, integrated secure assessment, analytics, attendance and multi-tenant security. The specific objectives derived from this gap lead to the proposed FortifyHub platform, whose requirements are specified in the next chapter.

Chapter 3
System Requirement Specification

This chapter specifies what the FortifyHub platform must do and the conditions under which it operates. It describes the product, its users and operating environment, lists the functional and non-functional requirements, the hardware and software requirements, and analyses the feasibility of the project.

3.1 Overall Description

3.1.1 Product Perspective

FortifyHub is a new, self-contained, web-based multi-tenant SaaS platform that replaces the single-college mini-project prototype. It is accessed through a browser and interfaces with external systems: the FortifyHub examination platform (single sign-on and account provisioning), e-mail providers (Resend / Amazon SES), Cloudflare R2 object storage, administrator-configured job-feed providers and a self-hosted language model for resume rewriting. Its functional modules are described in Section 4.3.1.

3.1.2 User Classes: Major Stakeholders and Roles

A system role (`tenant_admin`) plus editable role templates are seeded for every new college. Tenant administrators can edit these templates or create custom roles (for example "Hostel Warden").

Table 3.1: Stakeholders, roles and responsibilities

Role	Responsibilities in the platform
Platform Super Admin	On-boards colleges, manages plans, add-ons, trials, the global page catalogue, job-feed providers, platform settings and the global audit log. Has no access to tenant data pages.
Tenant Admin	Full control of one college: users, roles, permission matrix, settings, registration options, drive categories, policy, e-mail and audit.
Principal / HOD	Institution- or department-level oversight: students, classes, attendance reports, placement analytics and accreditation reports (mostly view/export).
Teacher / Proctor	"My Classes": rosters, attendance sessions (scan and manual), student history and class summaries.
Placement Officer / Head	Companies, drives, eligibility, applications, rounds, shortlists, selection notices, offers, visits, preparation resources, off-campus board and communications.

Table 3.1 (continued)

Role	Responsibilities in the platform
Student	Own profile, projects, internships, skills and documents; opportunities, policy acknowledgement, applications, selections, journey, resume builder, attendance and notifications.
Examination roles	Student, faculty and admin roles on the examination platform, provisioned from the placement identity (`exam_role`).

3.1.3 Operating Environment and Deployment Scenarios

- **Local development:** PostgreSQL and Redis in Docker; each service runs with hot reload; e-mail printed to the console.
- **Single-server production:** all services on one Ubuntu server with Docker Compose (Dokploy) behind Traefik and Cloudflare – the current deployment model.
- **Split deployment:** the worker (and AI or judge workloads) on a second machine via queue scoping, with the primary server running auth, core and the databases.
- **Managed cloud:** managed PostgreSQL and Redis with the same images orchestrated by Kubernetes, requiring only configuration changes.

3.1.4 Operational Assumptions and Constraints

- Peak load corresponds to placement seasons, result announcements and attendance at the start of classes.
- Clients have stable internet access; idempotent APIs and scan idempotency keys tolerate retries.
- Secrets (JWT keys, database passwords, encryption key, internal shared secret) are provisioned through the deployment platform and never committed.
- Institutional policies on data retention, trial periods (read-only after 30 days, lockout after 44, archival after 90) and access control are configured per tenant.
- Third-party services (Cloudflare, e-mail providers, Judge0) are subject to their own availability and quotas.

3.2 Functional Requirements

- FR1: The super admin shall create, suspend and edit tenants and assign plans, add-ons and trials.

- FR2: A tenant admin shall create users and custom roles and grant page-level actions within the subscription.
- FR3: Users shall log in with password and optional TOTP/passkey; sensitive actions shall require step-up verification.
- FR4: Staff shall create departments, classes and students individually or by CSV import.
- FR5: Teachers shall create attendance sessions and mark students by barcode scan or manually, exactly once per session.
- FR6: Placement officers shall manage companies, drives, eligibility, rounds, shortlists, offers, categories and policies.
- FR7: Students shall view eligible opportunities, acknowledge the policy, upload a resume, apply and track status.
- FR8: The system shall notify eligible students of new drives and deliver announcements, web-push and bulk e-mail.
- FR9: The system shall generate analytics, CSV/Excel exports and accreditation reports asynchronously.
- FR10: The system shall generate role-targeted resumes and support SSO into the examination platform.

The justification for each functional requirement is given in Section 8.7 (Results and Discussions).

3.3 Non-Functional Requirements

- **Security:** strong authentication, least-privilege authorization, tenant isolation in two layers, encryption at rest, audit logging and secrets only in environment variables.
- **Reliability:** core requests keep working when the auth service is down; background jobs retry; hourly backups to R2.
- **Performance:** permission checks served from cache; eligibility evaluated as a single database query; heavy work off the request path.
- **Scalability:** stateless services, connection pooling, partitioning and queue-based work distribution.
- **Maintainability:** module boundaries, versioned migrations, idempotent seeders, structured JSON logs and CI/CD.

- **Usability:** responsive layout with a mobile bottom navigation, global search, clear error codes and role-specific dashboards.
- **Portability:** no hard-coded URLs or credentials; the same images run locally, on a droplet or on a cloud provider.

The justification for each non-functional requirement is given in Section 8.7 (Results and Discussions).

3.4 Hardware Requirements

- **Development:** 8 GB RAM, 4 CPU cores, 20 GB free disk; Docker Desktop for PostgreSQL and Redis.
- **Minimum production (one college pilot):** 2 vCPU, 2–4 GB RAM, 50 GB SSD (for example a cloud droplet), plus swap.
- **Recommended production (several colleges with AI and exams on one box):** 4+ vCPU, 16 GB RAM (the Qwen3 1.7B model needs about 1.1 GB and is unloaded when idle), 100 GB SSD.
- **Client:** any modern browser; optional USB barcode scanner or a phone camera for ID-card scanning.

3.5 Software Requirements

- Node.js 20 LTS, npm
- React 18, Vite 5, React Router 6, Zustand, Axios, ZXing (barcode decoding)
- Express.js, Sequelize ORM, Joi, Helmet, express-rate-limit, Winston
- argon2, jsonwebtoken, speakeasy + qrcode (TOTP), WebAuthn library
- PostgreSQL 16, PgBouncer, Redis 7, BullMQ
- Docker, Docker Compose, Dokploy, Traefik v3, Ubuntu 24.04 LTS
- Cloudflare (DNS, WAF, Tunnel, R2, Pages, Workers, Hyperdrive)
- Ollama with the Qwen3 1.7B model (optional AI features)
- Judge0 CE with isolate (examination platform)
- GitHub Actions and GitHub Container Registry (CI/CD)

3.6 Feasibility Analysis

- **Technical feasibility:** every component of the stack (Node.js, Express, React, PostgreSQL, Redis, BullMQ, Docker, Traefik) is mature open-source software, and the team had already built a working prototype with a similar stack in the mini project. Feasibility is confirmed by the production deployment of the platform.
- **Economic feasibility:** the stack carries no licence fees; a one-college pilot runs on a single small server (Section 3.4); the language model is self-hosted, so there is no per-request AI cost; and on-boarding a new college is data entry rather than development work, which keeps the cost of adding customers low.
- **Operational feasibility:** role-specific dashboards mirror the existing work of teachers, placement officers and administrators, CSV import loads existing student records, and the role-by-role acceptance walkthrough on the production deployment confirmed that each role can complete its workflow.
- **Schedule feasibility:** the work was divided into incremental phases with clear exit criteria (Table 5.1), so each phase produced a demonstrable result.
- **Legal and ethical feasibility:** personal data is protected through tenant isolation, encryption at rest, least-privilege access and audit logging, and personal data is never sent to the language model; further compliance features aligned with the Digital Personal Data Protection Act, 2023 are planned as future work.

3.7 Summary

This chapter described the product, its users and environment, and specified ten functional requirements, seven non-functional requirements and the hardware and software needed. The feasibility analysis shows that the project is technically, economically, operationally and legally feasible. The next chapter presents the design that satisfies these requirements.

Chapter 4
System Design and Architecture

This chapter explains the high-level architecture, core components, data flow and design decisions of FortifyHub. It presents the deployment architecture, the integrated ecosystem with the examination platform, the entity-relationship model of both databases, the security architecture, scalability considerations, key interaction patterns and the system requirements. In addition, it describes the hardware deployment topology, the software module design, and the data-flow, use-case, activity and sequence diagrams of the system.

4.1 System Architecture

4.1.1 High-level Architecture

The system is composed of:

- **Frontend (`web`):** React 18 single-page application built with Vite, Zustand state store, permission-gated navigation; served by Nginx.
- **Authentication service (`auth-service`):** the security boundary – login, MFA, passkeys, step-up, refresh tokens, tenants, subscriptions, roles, permissions, page catalogue, SSO and the audit log. It is the only holder of the JWT private key.
- **Core service (`core-service`):** a *modular monolith* with self-contained modules for students, classes, departments, attendance, placement, notifications, e-mail, reports and accounts.
- **Background worker (`worker`):** BullMQ consumers for CSV imports, reports, file processing, bulk e-mail, notifications and scheduled maintenance.
- **Persistence:** PostgreSQL 16 with two databases (`auth_db`, `core_db`) owned by separate least-privilege users, fronted by PgBouncer; Redis 7 for caching, token revocation and queues; Cloudflare R2 (S3-compatible) for files.
- **Edge and deployment:** Cloudflare (DNS, TLS, WAF, DDoS, tunnel), Traefik reverse proxy managed by Dokploy, Docker images built and deployed by GitHub Actions.
- **Companion services:** a central SSO login hub, the FortifyHub examination platform (Cloudflare Workers + Judge0) and a self-hosted `resume-ai` service backed by Ollama.

The architecture deliberately avoids splitting the domain into many micro-services. Authentication is separated because it is the security boundary and has a different change rate;

everything else lives in one core service with strict internal module boundaries (a module owns its routes, controller, service and models, and other modules call its service interface, never its models). This keeps deployment simple on a small server while leaving a clean path to extract a module later.

A key design principle is **local JWT verification**. The core service verifies every access token with the auth service's *public* key and never calls the auth service on the hot path. Therefore, if the auth service is down, existing sessions keep working and only new logins fail – the "auth down means everything down" failure mode is avoided.

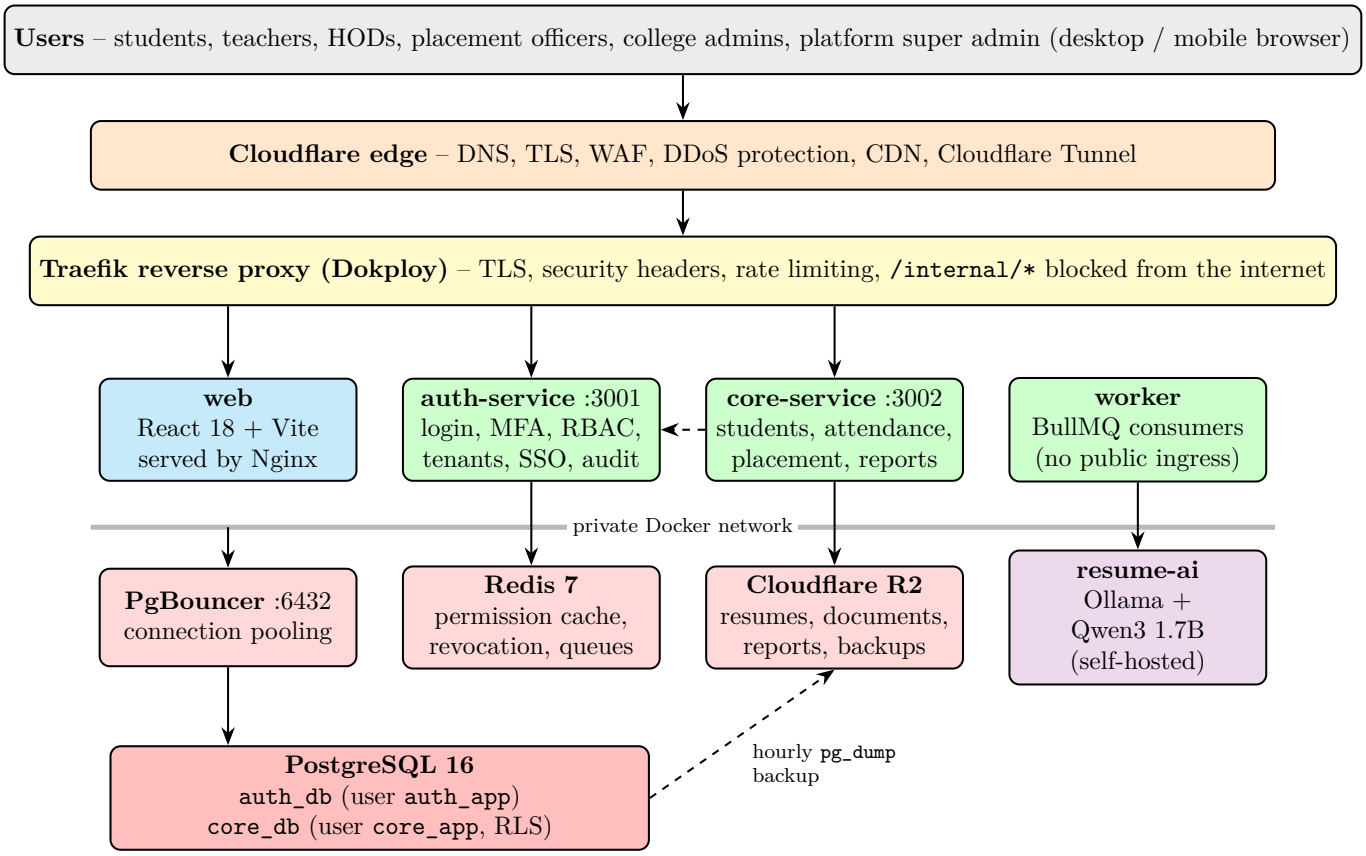


Figure 4.1: Deployment architecture of the placement platform (dashed: internal-only calls and backups)

4.1.2 Integrated Ecosystem

The placement platform is one of two products that share a single identity provider. The **central SSO hub** lets a user sign in once and open either the placement platform or the examination platform. The auth service is the *identity master*: when a student or staff account is created or deleted on the placement side, the account is provisioned to (or purged from) the examination platform. The examination platform runs on Cloudflare Pages and Workers, grades code through a Judge0/isolate sandbox and stores its data in a PostgreSQL database reached through Cloudflare Hyperdrive with per-college RLS.

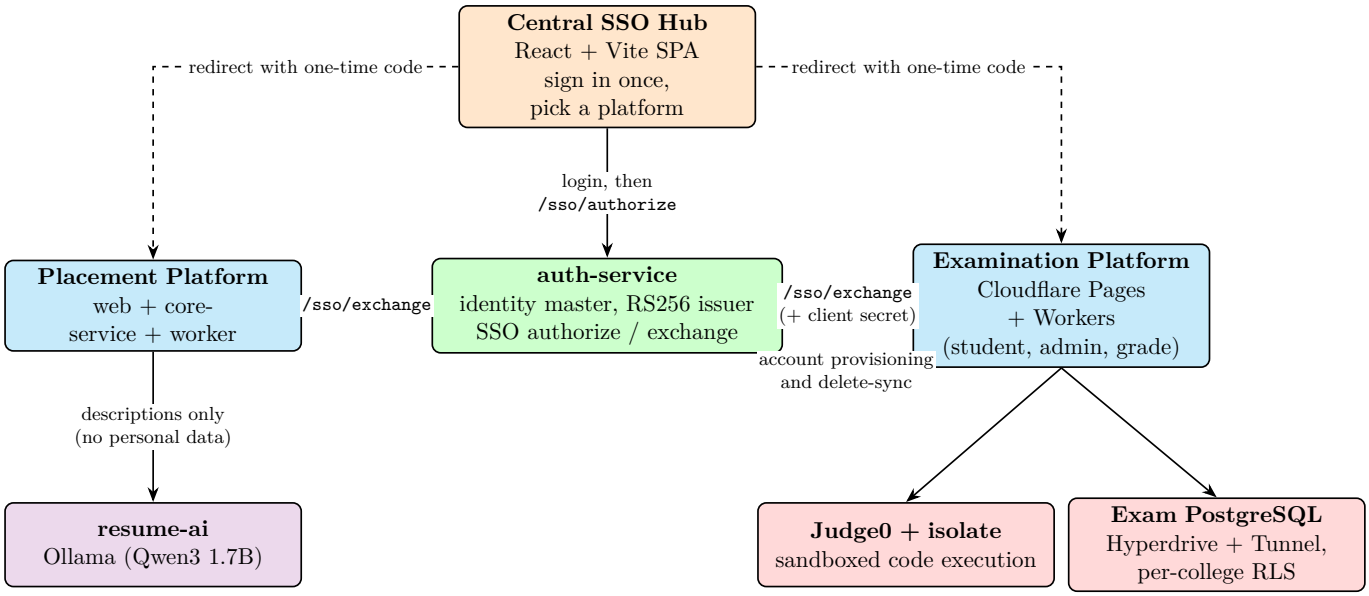


Figure 4.2: Integrated FortifyHub ecosystem: SSO hub, placement platform, examination platform and AI service

4.1.3 Component Interaction

Figure 4.10 shows how every authenticated request to the core service is processed. Key interactions are:

- The frontend calls the auth service for identity operations and the core service for domain operations, both through Traefik.
- The core service verifies the JWT locally and checks the revocation list in Redis.
- Permissions are read from Redis; on a cache miss or `permission_hash` drift, the core service calls the auth service's internal endpoint.
- The request runs inside a tenant-scoped transaction; the transaction commits *before* the response bytes are sent.
- Long-running work is enqueued to BullMQ and processed by the worker, which updates job status that the frontend polls.
- Privileged actions are recorded through the auth service's audit endpoint.

4.1.4 Scalability Considerations

The platform is sized for colleges with 4,000–8,000 students each and is designed to scale by configuration rather than by rewriting:

- **Stateless services:** auth and core keep no session state in memory, so more replicas can be added behind Traefik.
- **Connection pooling:** PgBouncer multiplexes many application connections onto a small number of PostgreSQL connections; the RLS context uses `SET LOCAL`, which is safe under transaction pooling.
- **Asynchronous work:** imports, reports, bulk e-mail and file processing run in the worker; per-tenant concurrency caps stop one college from monopolising the queue. Queue scoping (`WORKER_QUEUES`) allows an overflow worker on another machine.
- **Partitioning:** monthly partitions keep `attendance_records` and `audit_logs` queries fast as data grows.
- **Edge caching:** static frontend assets are cached by Cloudflare.
- **Adaptive AI load:** the resume-ai service scales its number of model workers with CPU load so that it never starves the examination judge running on the same box.
- **Migration path:** all configuration comes from environment variables, so the same images can move from a single droplet to AWS or Kubernetes without code changes.

4.1.5 Design Patterns Used

The system uses:

- **Modular monolith** for the core service, with module-owned routes, controllers, services and models.
- **Layered request pipeline** (middleware chain): authentication → authorization → tenant context → validation → controller → service → model.
- **Token-based stateless authentication** with asymmetric signatures and a revocation list.
- **Cache-aside** for permission maps with hash-based invalidation.
- **Producer–consumer (work queue)** with BullMQ for background jobs and repeatable schedules.
- **State machine** for application and offer life cycles.
- **Strategy pattern** for pluggable e-mail providers (console, Resend, Amazon SES) selected by configuration.
- **Repository / ORM pattern** with Sequelize models and versioned migrations.
- **Gateway at the edge** (Traefik) for routing, TLS and cross-cutting policies.

4.2 Hardware Architecture

FortifyHub is a software project and does not involve custom hardware. Its physical deployment topology is as follows:

- **Edge:** Cloudflare provides DNS, TLS, WAF, DDoS protection, CDN caching and the tunnel to the origin server.
- **Application server:** a single Ubuntu 24.04 LTS server runs all containers (web, auth-service, core-service, worker, PostgreSQL, PgBouncer, Redis and the optional resume-ai service) behind Traefik; its sizing is given in Section 3.4.
- **Storage:** Cloudflare R2 stores resumes, documents, generated reports and the hourly database backups.
- **Client devices:** any modern desktop or mobile browser; attendance uses a USB (serial) barcode scanner or a phone camera to read student ID cards.
- **Scale-out path:** the worker and the AI or judge workloads can move to a second machine through queue scoping, and managed PostgreSQL/Redis can replace the local containers through configuration only.

4.3 Software Module Design

The platform is implemented with the technology stack summarised in Table 4.1. The core service is a modular monolith in which each module owns its routes, controller, service and models.

Table 4.1: Technology stack used for implementation

Layer	Technologies Used
Frontend	React 18, Vite 5, React Router 6, Zustand, Axios, ZXing (barcode), SheetJS (Excel export)
Backend services	Node.js 20, Express.js, Sequelize ORM, Joi, Helmet, express-rate-limit, Winston
Security	argon2 (argon2id), jsonwebtoken (RS256), speakeasy (TOTP), WebAuthn, Node `crypto` (AES-256-GCM, SHA-256)
Databases & cache	PostgreSQL 16 (RLS, partitioning, JSONB), PgBouncer, Redis 7
Background jobs	BullMQ workers and repeatable schedules
Storage & e-mail	Cloudflare R2 (S3 API), Resend / Amazon SES
AI	Ollama, Qwen3 1.7B (self-hosted)
Assessments	Examination platform on Cloudflare Workers, Judge0 + isolate, Hyperdrive
Deployment	Docker, Dokploy, Traefik v3, Cloudflare, GitHub Actions, GHCR

4.3.1 High-level Functional Modules

- **Tenant and Subscription Management** – tenant provisioning, plans, add-ons, trials, capacity and page catalogue.
- **User, Role and Permission Management** – users, dynamic roles, per-page permissions, password resets and step-up.
- **Authentication and SSO** – login, MFA, passkeys, refresh rotation, logout, central SSO.
- **Academic Structure and Student Profiles** – departments, classes, class teachers, students, sub-records, files and CSV import.
- **Attendance** – sessions (multi-class), barcode and manual marking, histories, summaries.
- **Placement** – companies, drives, eligibility, applications, rounds, offers, categories, policy, visits, selections, preparation resources and off-campus opportunities.
- **Assessment Integration** – SSO and account provisioning to the examination platform.
- **Analytics and Reports** – placement analytics, engagement, year-on-year trends, accreditation and exports.

- **Communication** – announcements, in-app notifications, web-push and bulk e-mail.
- **Career Tools** – role-targeted resume builder with optional AI rewriting and a student journey timeline.

4.3.2 Detailed Module Descriptions

This section presents the major modules of the platform along with their responsibilities, schema components and characteristics.

Frontend Module

Frontend responsibilities include:

- Role-based dashboards (student, teacher, placement, admin, super admin) and permission-gated navigation.
- Opportunity search, drive details, applications, rounds, selections and a student journey timeline.
- Attendance scanning through a USB serial scanner or the camera (ZXing), with manual marking dialogs.
- Notification centre, announcements, global search and a mobile bottom navigation.

Key Components

- **Login, MFA and passkey pages:** password, e-mail OTP, TOTP and WebAuthn flows, forced password change.
- **Step-up modal:** intercepts `STEP_UP_REQUIRED` responses, obtains an OTP-backed token and retries the action.
- **RequirePerm and ProtectedRoute:** route guards driven by the permission map (UX only; the backend enforces).
- **Resume page:** role selection, preview and export of the generated resume.

Auth and RBAC Module

- Issues RS256 access tokens and rotating refresh tokens; publishes the public key for local verification.
- Enforces password policy, account lockout, rate limits, MFA and step-up.
- Computes each user's effective permissions (union over roles, filtered by the tenant's subscription) and caches them.

- Provisions tenants in one transaction: tenant, first admin, default roles and permission matrix.
- Records privileged actions in the partitioned, append-only audit log.

Coding Platform Module

Coding assessments are delivered by the integrated examination platform:

- Exams with sections, MCQ and coding questions, sub-tasks and weighted, hidden or sample test cases.
- Server-authoritative grading through Judge0/isolate for Python, JavaScript, TypeScript, Java, C, C++, Go and other languages.
- Attempts, answers, certificates and results analytics for faculty.

Sandbox Safety and Isolation

- Code runs inside the isolate sandbox (Linux namespaces and control groups) with CPU-time, wall-time, memory, stack and output limits.
- No network access during execution; each run uses a fresh, discarded box.
- Client-reported results are ignored; the score is recomputed on the server.
- Proctoring signals (tab switches, developer tools, copy/paste, snapshots) and plagiarism similarity checks support exam integrity.

Analytics Module

- Placement totals, placement and application percentages, CTC statistics and CTC bands.
- Top recruiters, department-wise breakdown and year-on-year trends by graduating batch.
- Engagement roll-ups per department and class (profile completion, resume, applied, placed).
- Accreditation report, placement-proof manifest and export formats for re-upload.

Readiness Scoring Readiness is expressed through the engagement indicators defined earlier rather than a single opaque score, so that faculty can see exactly which factor (profile, resume, participation or outcome) needs attention.

Tenants and Subscriptions Module

Schema Components

- **Tenants:** *id, name, slug, status, created_at, deleted_at*
- **Subscription_Plans / Subscription_Addons:** *name, annual_price, student_capacity, included_page_ids / page_ids*
- **Tenant_Subscriptions:** *tenant_id, plan_id, addon_ids, is_trial, trial_ends_at, grace_period_ends_at, status*

Characteristics

- Flat annual pricing by tier (Basic, Gold, Premium, Enterprise) that gates features and student capacity.
- 30-day trial; read-only after day 30, lockout after day 44 and archival after day 90, driven by a scheduled worker job.

Roles and Permissions Module

Schema Components

- **Roles:** *id (PK), tenant_id (NULL = system role), name, description, is_system*
- **Pages:** *id, code, display_name, category, min_plan_tier, requires_addon_id, is_active*
- **Permissions:** action types – view, create, edit, delete, export
- **Role_Permissions:** *role_id, page_id, actions (jsonb)*
- **User_Roles:** *user_id, role_id, granted_by, granted_at, expires_at*

Characteristics

- Defines access boundaries per page and action; about 65 pages in the catalogue.
- Supports custom roles and time-bound grants; the tenant admin role is undeletable.
- Changes to roles and permissions require step-up verification and are audited.

Users Module

Schema Components

- **Users:** *id, tenant_id, email, password_hash, mfa_enabled, mfa_secret (encrypted), status, failed_login_count, locked_until, must_change_password, exam_role*
- **Refresh_Tokens, MFA_Recovery_Codes, WebAuthn_Credentials, Password_Reset_Tokens, Email_Verification_Tokens** – all secrets stored hashed or encrypted.

Characteristics

- Central authentication hub for all roles and both platforms.
- Student accounts are provisioned automatically from student records (auto-login with first-login password change).
- Admin-mediated password reset follows a role hierarchy and is reported to the affected user.

Companies Module

Schema Components

- **Companies:** *id, tenant_id, name, sector, hr_contacts (jsonb, encrypted), logo_file_id, status*
- **Company_Documents / Company_Visits:** documents shared with students and visit schedules.

Characteristics

- Represents recruiters with contact details and documents; supports blacklisting.
- Links with drives, offers and top-recruiter analytics; CSV export filtered by required skill.

Jobs Module

Schema Components

- **Jobs:** *id, tenant_id, company_id, category_id, role, description, ctc, location, job_type, eligibility (jsonb), deadline, seats, status*
- **Drive_Categories:** *code, name, sort_order, min/max CTC, max_offers_in_category, blocks_lower_categories*
- **Drive_Rounds / Shortlists, Drive_Bookmarks, Selection_Notices**

Characteristics

- Draft → open → closed/cancelled life cycle; opening a drive automatically notifies every eligible student who has not applied.
- Category is derived from CTC when not specified; soft delete and restore.

Students Module

Schema Components

- **Students:** identity (USN, barcode, name, e-mail, phone), class, batch, semester, CGPA, 10th/12th percentage, backlogs, gap years, social links, encrypted Aadhaar and PAN.
- **Import_Jobs:** CSV/Excel import status and row-level errors.
- **Student_Options / Tenant_Registration_Settings:** configurable dropdowns and self-registration rules.

Characteristics

- Core entity for attendance, eligibility, analytics and resumes.
- Identity numbers are revealed only on an explicit "View" action by authorised users, which is audited.
- Students can claim their record and edit permitted fields of their own profile.

Student Achievements Module

Schema Components

- **Student_Projects:** title, description, tools used, repository link.
- **Student_Internships:** organisation, role, duration, skills acquired.
- **Student_Languages:** skills/languages with level and category.
- **Student_Certifications, Student_Achievements, Student_SGPAs**.
- **Student_Files:** resume and documents stored in R2 with labels and processing status.

Characteristics

- Captures practical experience and evidence for eligibility (required skills) and resumes.
- Enforces per-file, per-student and per-tenant storage quotas.

Applications and Offers Module

Schema Components

- **Applications:** *id, job_id, student_id, status, status_history, applied_at, notes*
- **Offers:** *application_id (nullable for off-portal placements), student_id, job_id, company_id, ctc, joining_date, status, responded_at*

Characteristics

- State-machine controlled transitions and bulk application with per-student reasons.
- Offer accept/decline/revoke in transactions; batch "mark placed" for off-portal offers.
- Feeds the two-offer policy and placement analytics.

Attendance Module

Schema Components

- **Sessions / Session_Classes:** a session for one or more classes with a schedule and status (scheduled, open, closed).
- **Attendance_Records:** monthly partitions; method, device and marker; de-duplication ledger.

Characteristics

- Barcode (serial/camera) and manual marking, unmark, roster and per-student history.
- Class summaries and the student's own attendance view.

Communication Module

- **Notifications:** per-user in-app notifications, strictly self-scoped; web-push subscriptions.
- **Announcements:** targeted announcements and a versioned placement policy that students must acknowledge.
- **Bulk e-mail:** audience picker (roles, classes, departments, custom lists), cost preview in paise, attachments, delivery through the worker with provider rate limits, and full audit.

Reports and Background Processing Module

- Queues: CSV import, notifications, reports, file processing, e-mail, trial maintenance and partition maintenance.
- Reports are generated asynchronously and downloaded through a tenant-scoped endpoint (no internal paths exposed).
- Per-tenant concurrency caps reserve a slot only after validation and release it on every path (success, failure, crash) with a TTL safety net.

Resume and AI Module

- Deterministic resume generation from the student profile with role-based ordering of skills and projects.
- Optional rewriting of project and internship descriptions into concise, ATS-friendly bullets by a self-hosted Qwen3 1.7B model.
- Only descriptions, titles and tools are sent; names, contact details, USN and other personal data never leave the core service.
- Per-item fallback to the original text on any model error, a bounded cache and CPU-adaptive worker count.

Off-Campus Opportunities Module

- A shared, cross-college opportunity feed imported from registered providers by a scheduler.
- Provider credentials stored encrypted and managed only by the super admin; the provider's brand is not exposed to users.
- Per-tenant filters, click tracking and manual postings by placement officers.

Placement Summary and Statistics Module

- Totals of students, applied and placed; placement percentage; CTC statistics and distribution.
- Drive-level funnels (eligible $\rightarrow$ applied $\rightarrow$ shortlisted $\rightarrow$ selected $\rightarrow$ offer).
- Year-wise reporting for management and accreditation.

Module Interaction Summary

- Subscriptions decide which pages exist for a tenant; roles grant actions on those pages; users hold roles.
- Departments contain classes; classes contain students; sessions record attendance for classes.
- Companies post drives with eligibility rules; the eligibility engine selects students; students apply through the application module subject to resume, policy and offer-tier gates.
- Offers update placement statistics and the offer-policy state of the student.
- Achievements and skills feed eligibility (required skills) and the resume generator.

- The worker executes imports, reports, e-mail and maintenance for all modules; the auth service provisions identities to the examination platform.

This modular architecture keeps each concern isolated while sharing one consistent, tenant-scoped data model.

4.4 Security and Access Control

Security and Authentication

Security is treated as an architectural property rather than a feature. The platform applies *defence in depth*: each layer assumes that the layer above it may fail.

4.4.1 Authentication

- **Password storage:** argon2id with a per-password salt. Parameters follow the OWASP minimum (19 MB memory, 2 iterations) and can be tuned by environment variables; older, heavier hashes are re-hashed lazily on the next successful login. A concurrency semaphore limits simultaneous hashing so that a login stampede queues instead of exhausting the CPU.
- **Access tokens:** RS256-signed JWTs valid for 15 minutes containing only `sub`, `tenant_id`, `jti`, `permission_hash` and `exp`. The private key exists only in the auth service; the core service holds the public key.
- **Refresh tokens:** opaque 7-day tokens stored only as SHA-256 hashes, rotated on every use and grouped into a `family_id`. Presenting an already-rotated token revokes the entire family (theft detection).
- **Multi-factor authentication:** TOTP (RFC 6238) with recovery codes, WebAuthn passkeys and e-mail one-time passwords. The platform super admin is forced to enrol MFA and to change the bootstrap password on first login.
- **Step-up verification:** sensitive actions (granting or revoking roles, editing permissions, suspending tenants, generating password resets) require a fresh, single-use, action-bound step-up token obtained through an e-mail OTP.
- **Account protection:** lockout after five failed attempts, per-endpoint rate limits (login, MFA verify, forgot/reset/change password, refresh, step-up) and uniform responses that do not reveal whether an e-mail exists.
- **Logout and revocation:** logout writes the token's `jti` into a Redis revocation list that every service checks.

- **Single sign-on:** a 256-bit, single-use, short-lived code bound to user, tenant and platform; redirect targets come only from server configuration (no open redirect).

4.4.2 Authorization

Authorization uses dynamic RBAC. A *page* is a feature in the global catalogue (for example `placement.jobs`); a role grants a subset of actions (view, create, edit, delete, export) on each page. A tenant's subscription decides which pages can be granted at all. Every API route in the core service is protected by `requirePermission(page, action)`; the effective permission map is cached in Redis for five minutes and refreshed when the token's `permission_hash` no longer matches. The frontend hides menu items and buttons using the same map, but this is only a usability feature – the backend enforces every rule independently.

4.4.3 Tenant Isolation and Data Protection

- **Application layer:** every query goes through a `scopedWhere` helper that adds `tenant_id` from the verified token. Client-supplied foreign keys (class, user, advisor) are validated against the caller's tenant.
- **Database layer:** PostgreSQL RLS is enabled and *forced* on every tenant table. Each request runs inside a transaction that sets `app.tenant_id` with `SET LOCAL`; without the setting, queries fail closed.
- **Least privilege:** `auth_app` and `core_app` can access only their own database; neither can read the other's tables.
- **Encryption at rest:** sensitive columns (for example phone numbers, identity numbers, MFA secrets, HR contacts) are encrypted in the application with AES-256-GCM using a random 96-bit IV and a 128-bit authentication tag, stored with a `v1:` version prefix followed by the base64 of IV, tag and ciphertext. Tampered or wrongly keyed values fail loudly.
- **Auditing:** every privileged action writes an append-only audit row with before/after snapshots in which secrets and personal fields are redacted.
- **Input handling:** Joi validation with `stripUnknown` on every request body (no mass assignment), UUID validation on path parameters, parameterised queries only.
- **Network:** Helmet security headers (HSTS, CSP, frame options), strict CORS allow-list, `/internal/*` endpoints protected by a shared secret and blocked at the proxy, and an SSRF guard for server-side fetches of administrator-supplied URLs.
- **Files:** uploads are size-limited per file, per student and per tenant, stored in private R2 buckets and served through short-lived signed URLs.

4.5 Database Design

4.5.1 Core Entities and Relationships

The platform revolves around tenants, users, roles, students, classes, attendance, companies, jobs, applications and offers. At a conceptual level the data model is structured around the following entities:

- **Tenant:** a college. Every row in `core_db` carries a `tenant_id`; the platform super admin has `tenant_id = NULL`.
- **User, Role, Page, Permission:** login accounts, roles (system or custom), the global feature/page catalogue and the actions (view, create, edit, delete, export) granted per page.
- **Subscription Plan and Add-on:** Basic, Gold, Premium and Enterprise tiers that decide which pages a tenant can unlock and its student capacity.
- **Department, Class, Student:** academic structure and the student master record with academic, personal and profile sub-records.
- **Session and Attendance Record:** attendance sessions for one or more classes, with one record per (session, student).
- **Company, Job, Drive Category, Drive Round:** recruiters, drives with JSON eligibility rules, CTC tiers and multi-round shortlists.
- **Application and Offer:** a student's application to a job with a status history, and the resulting offer (accepted, declined, revoked).

These entities are normalised so that eligibility rules, analytics, resumes and attendance reuse one consistent student record instead of duplicating it. The design also makes auditing easier: the placement office can trace why a student was eligible, shortlisted or blocked by policy for a given drive.

From an implementation perspective:

- **Identity data** (tenants, users, roles, permissions, tokens, audit logs, subscriptions) lives in `auth_db`, owned only by the auth service.
- **Domain data** (students, classes, attendance, placement, notifications, reports) lives in `core_db`, owned only by the core service. The two services never join across databases; cross-service data travels over HTTP.
- **Flexible attributes** such as job eligibility rules, role permission actions and application status histories are stored as `JSONB`.

- **High-volume tables** (`attendance_records`, `audit_logs`) are range-partitioned by month, and a maintenance job creates future partitions.

The relationship structure supports the many-to-many mappings common in placements. A student may apply to many jobs, a job receives hundreds of applications, a session can cover several classes and a user can hold several roles. Capturing these relationships explicitly lets the platform answer questions such as:

- Which students of a given batch satisfy all eligibility rules of a drive and have not yet been placed in a higher tier?
- What is the department-wise placement percentage and CTC distribution for this academic year compared with previous years?
- What percentage of a class has a complete profile, an uploaded resume, at least one application and an accepted offer?

4.5.2 Data Storage and Consistency

PostgreSQL is used for all transactional data because placement and attendance require ACID guarantees: an application must reference an existing job and student, an offer acceptance must update the student's placement state atomically and attendance must never be counted twice. Uniqueness is enforced in the database (for example `UNIQUE(job_id, student_id)` on applications and a de-duplication ledger maintained by a trigger for attendance), so that application code races cannot create duplicates. Soft deletion (`deleted_at`) allows drives, sessions and students to be restored.

Redis holds short-lived, reconstructible state: cached permission maps, the token revocation list, step-up challenges, SSO codes, rate-limit counters, per-tenant concurrency counters and BullMQ queues. Losing Redis never loses business data. Large binary objects (resumes, documents, logos, generated reports and database backups) are stored in Cloudflare R2.

4.5.3 Simplified ER Diagram

Figure 4.3 gives a simplified view of the core placement and attendance entities and their cardinalities. A tenant (college) owns departments, classes and students; each student can submit many applications, and each job receives many applications, with at most one offer resulting from an application. Attendance sessions cover one or more classes and hold one attendance record per student, while users map to roles through a many-to-many relationship.

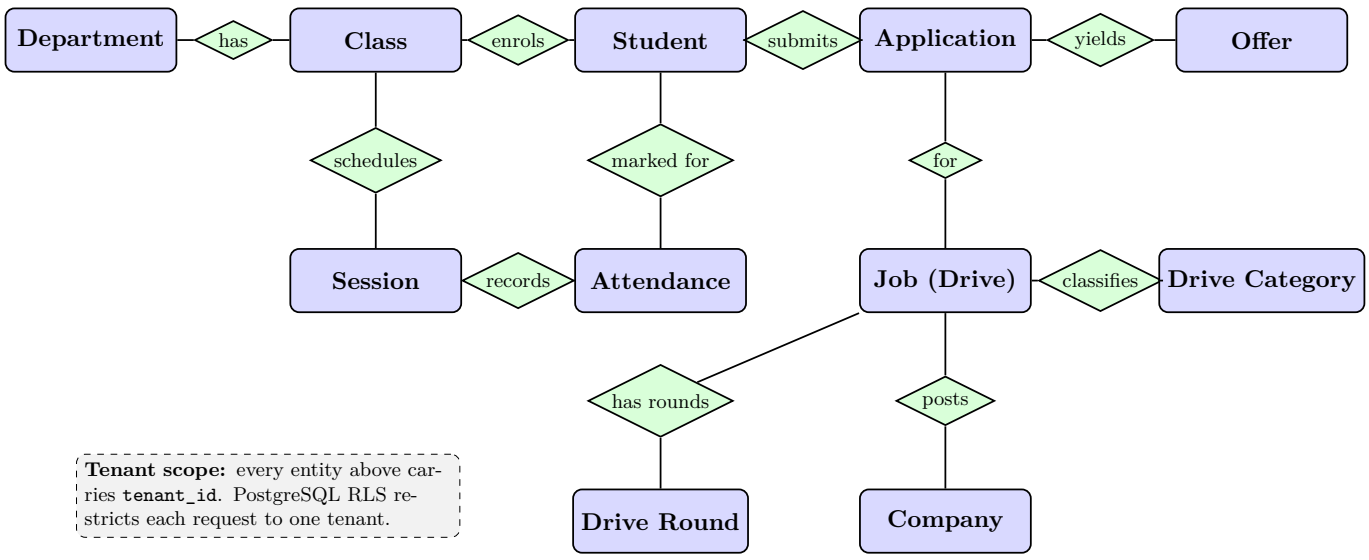


Figure 4.3: Simplified ER diagram of the core placement and attendance entities

4.5.4 Detailed ER Diagram – Identity Database (`auth_db`)

Figure 4.4 shows the identity database owned by the auth service. Tenants hold subscriptions to plans that unlock pages from the global catalogue; roles are granted per-page actions through `ROLE_PERMISSIONS` and assigned to users through `USER_ROLES`. Refresh tokens are stored only as SHA-256 hashes grouped into families for theft detection, passkeys are kept in `WEBAUTHN_CREDENTIALS`, and every privileged action is written to the partitioned `AUDIT_LOGS` table with sensitive values redacted.

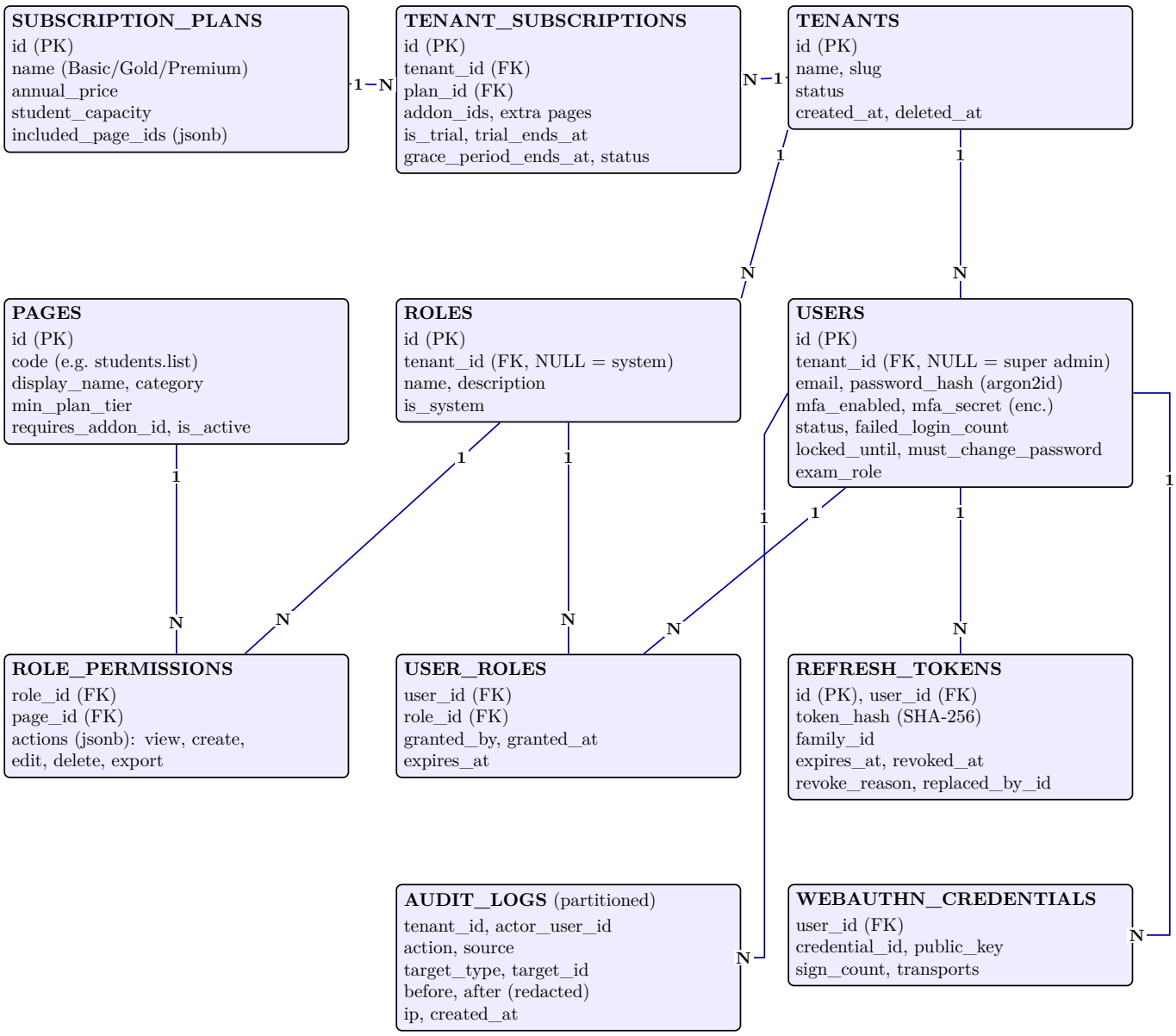


Figure 4.4: Detailed entity-relationship diagram of the identity database (`auth_db`)

4.5.5 Detailed ER Diagram – Domain Database (`core_db`)

Figure 4.5 shows the domain database owned by the core service. Departments contain classes, classes contain students, and each student has sub-records for projects, internships, skills, certifications and files. Attendance records are partitioned by month and de-duplicated per session and student, applications are unique per job and student, and offers reference the application, company and job. Every table carries `tenant_id` and is protected by a forced Row-Level Security policy.

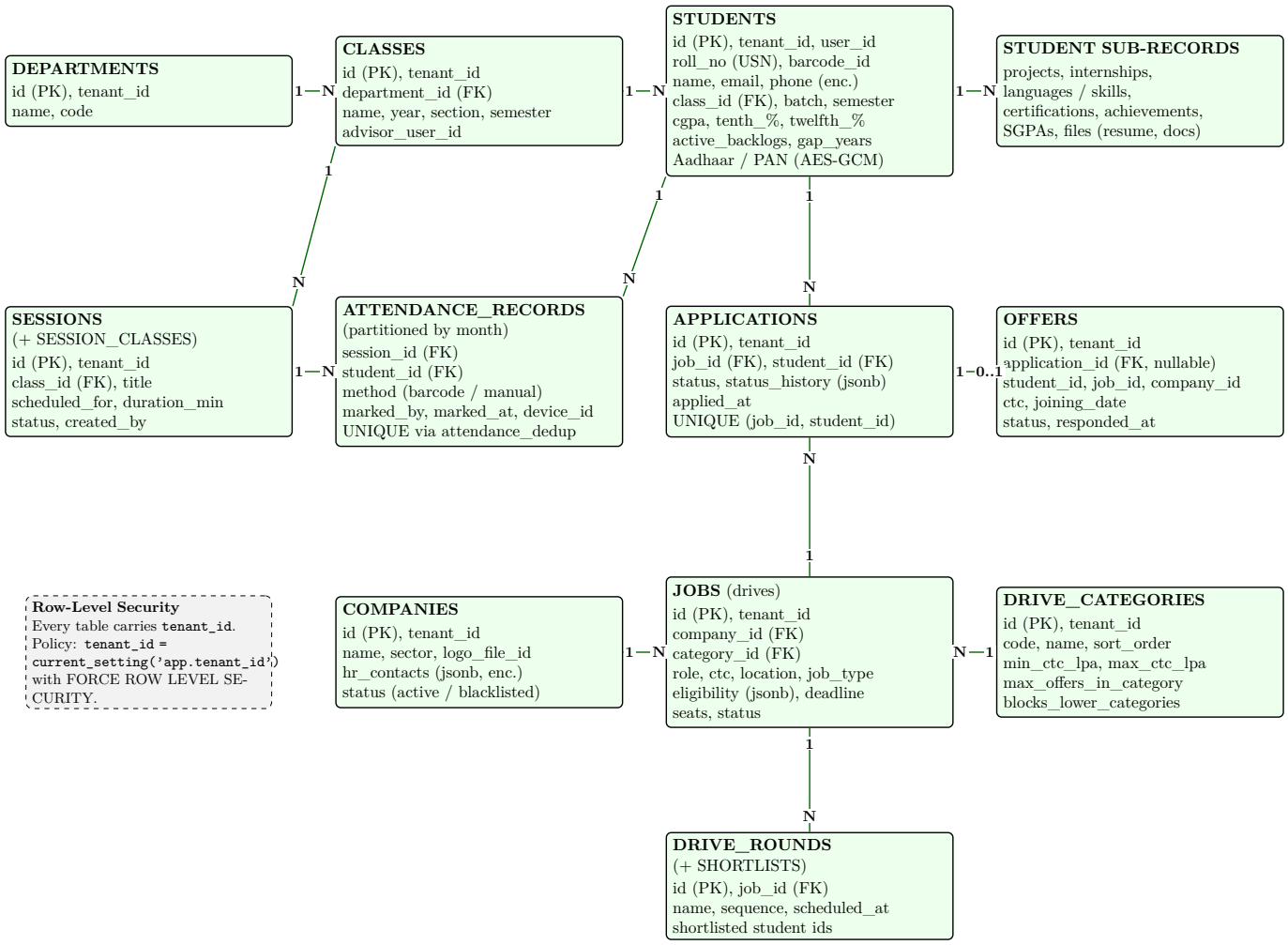


Figure 4.5: Detailed entity-relationship diagram of the domain database (`core_db`)

4.6 Data Flow Diagram (Level 0 and Level 1)

4.6.1 Level 0 – Context Diagram

The context diagram (Figure 4.6) treats the whole platform as a single process and shows its data exchanges with the external entities: students, teachers, placement officers, tenant administrators and management, the platform super admin, the examination platform, e-mail providers and job-feed providers.

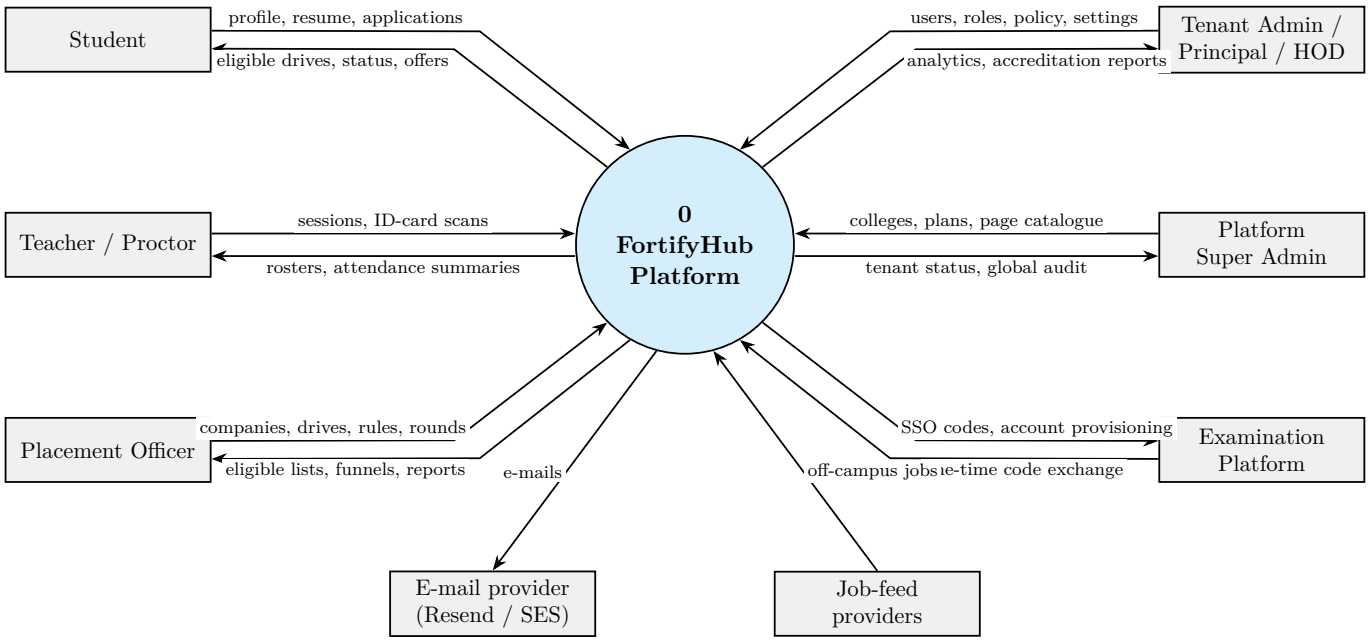


Figure 4.6: Data flow diagram – Level 0 (context diagram)

4.6.2 Level 1

The Level 1 diagram (Figure 4.7) decomposes the platform into six processes – authentication and authorisation, single sign-on and provisioning, student and class management, drive and application management, attendance marking, and reports, analytics and notifications – and the four data stores they use: `auth_db`, `core_db`, Redis and Cloudflare R2.

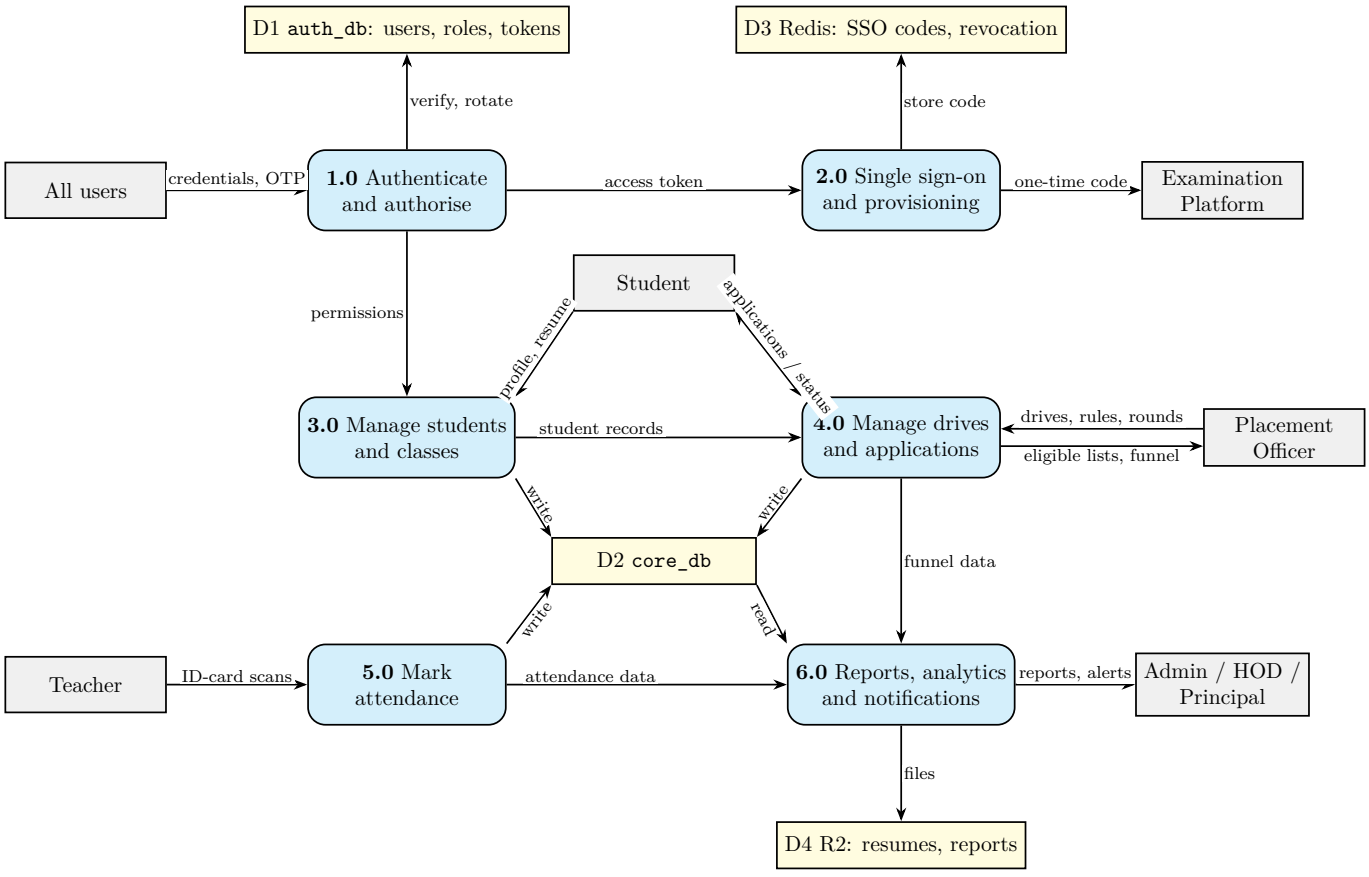


Figure 4.7: Data flow diagram – Level 1

4.7 Use Case Diagram

Figure 4.8 shows the main actors of the platform and the use cases each actor performs. Every actor first authenticates through the login use case (password with MFA, passkey or SSO); the permissions of each actor are decided by the role-based access control described in Section 4.4.

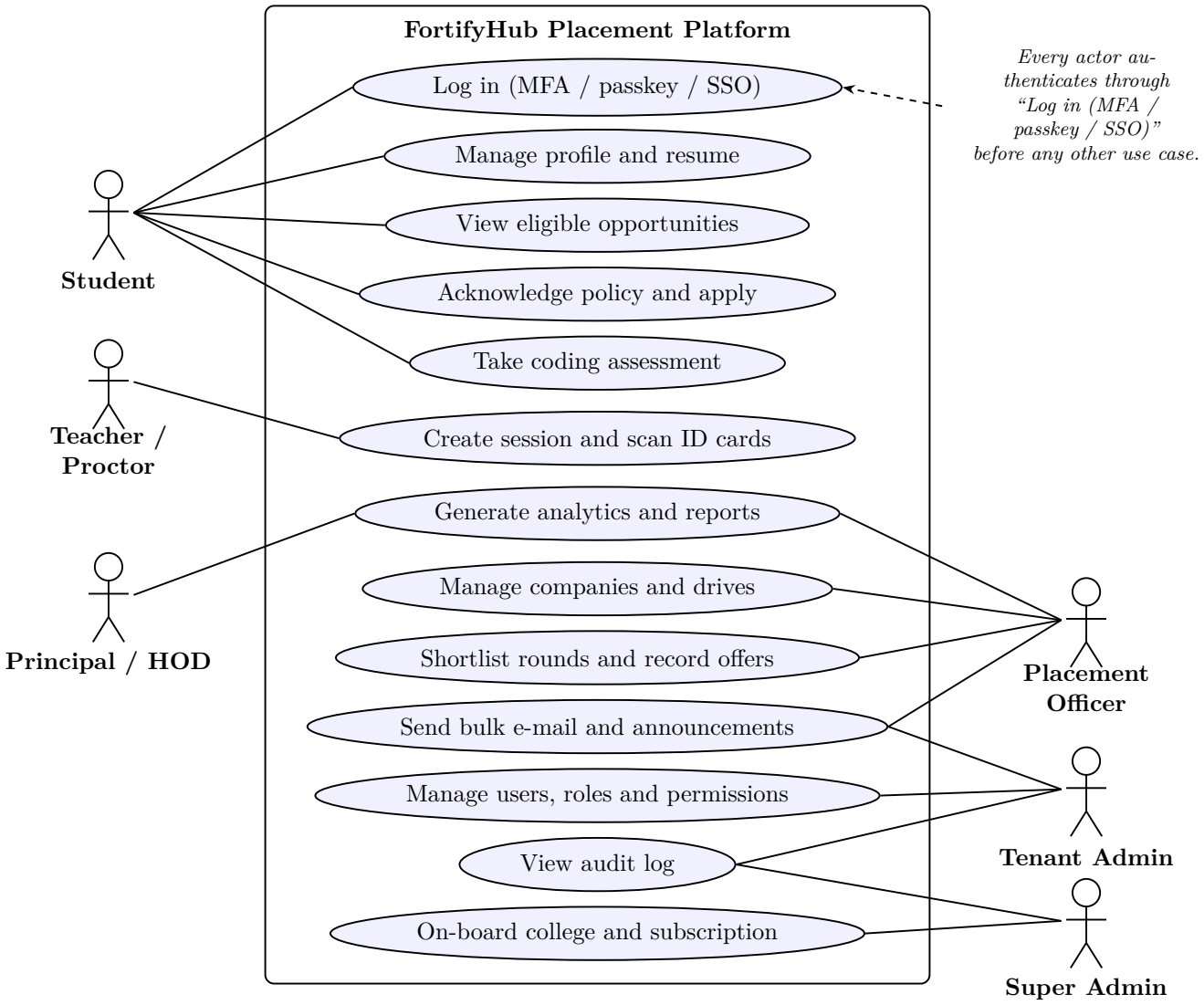


Figure 4.8: Use case diagram of the FortifyHub placement platform

4.8 Activity Diagram

Figure 4.9 shows the activity flow when a student applies to a placement drive. The application passes through the eligibility check and the three apply-time gates – policy acknowledgement, resume on file and the tiered offer policy – before it is created, recorded in the status history and notified.

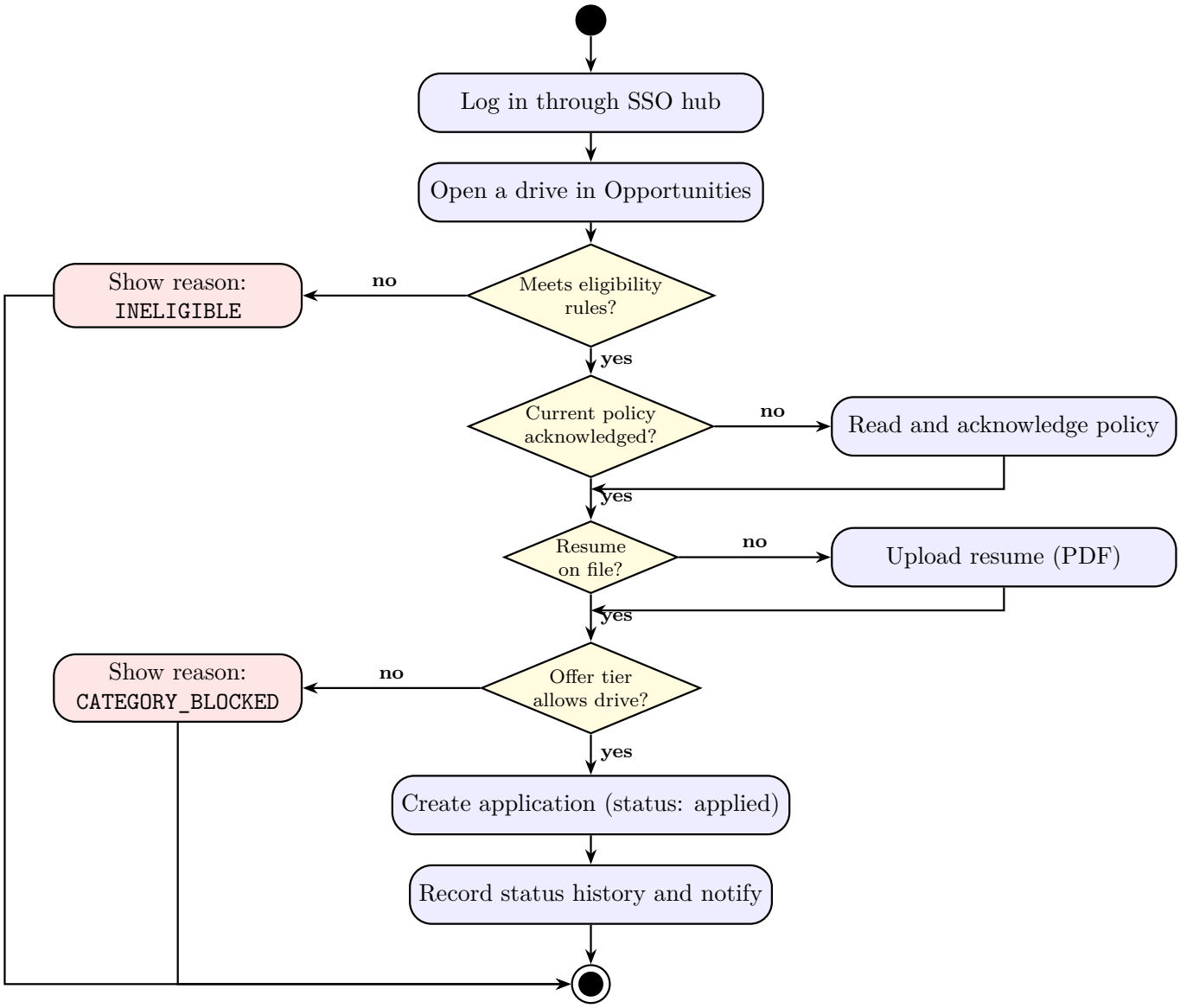


Figure 4.9: Activity diagram – student applying to a placement drive

4.9 Sequence Diagram / Transition Diagram

4.9.1 Request Processing Sequence

Figure 4.10 shows the sequence of checks applied to every authenticated request to the core service.

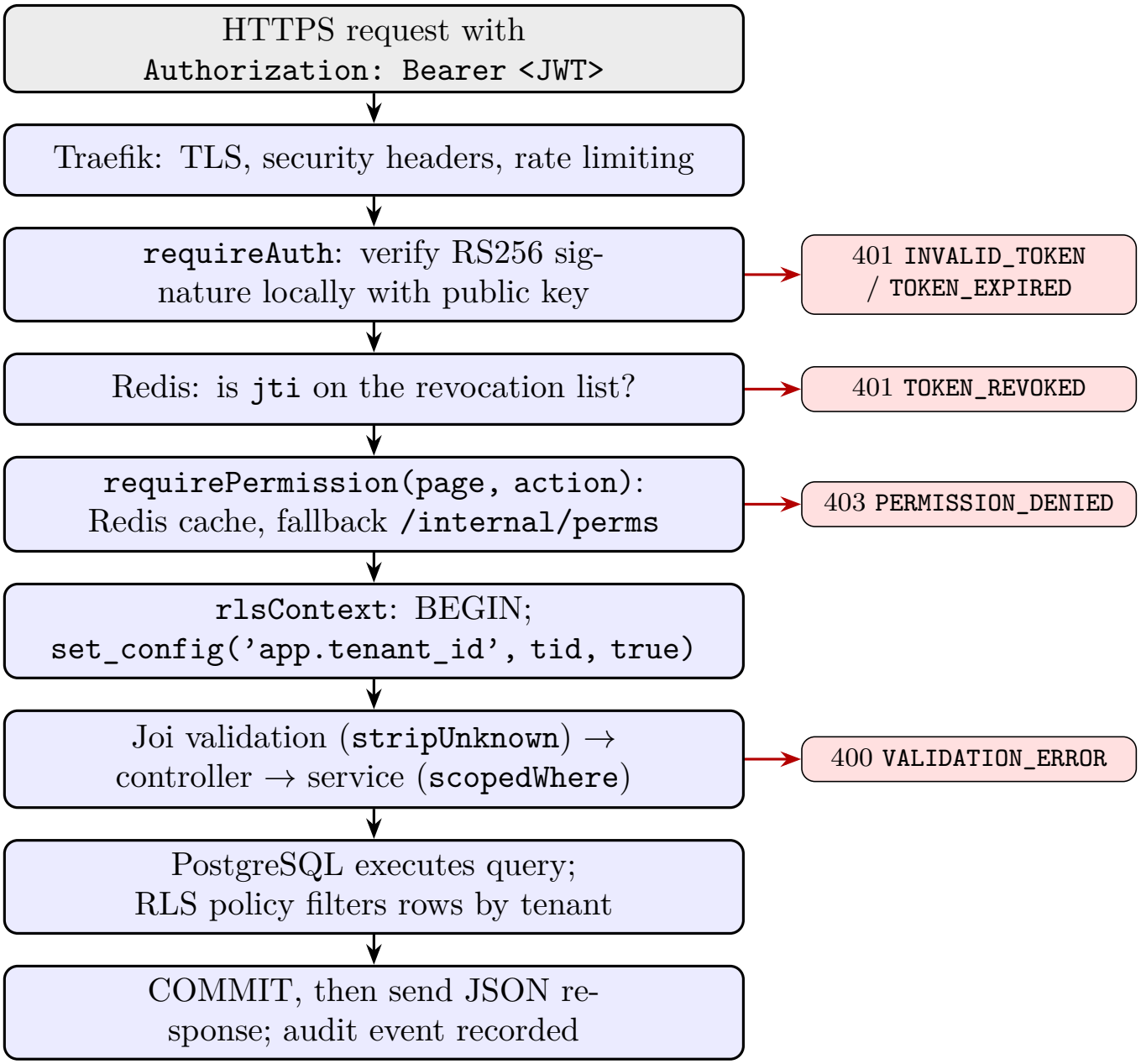


Figure 4.10: Request processing and authorization pipeline in the core service

4.9.2 Single Sign-On Sequence

Figure 4.11 shows the message sequence between the browser, the SSO hub, the auth service and the target platform.

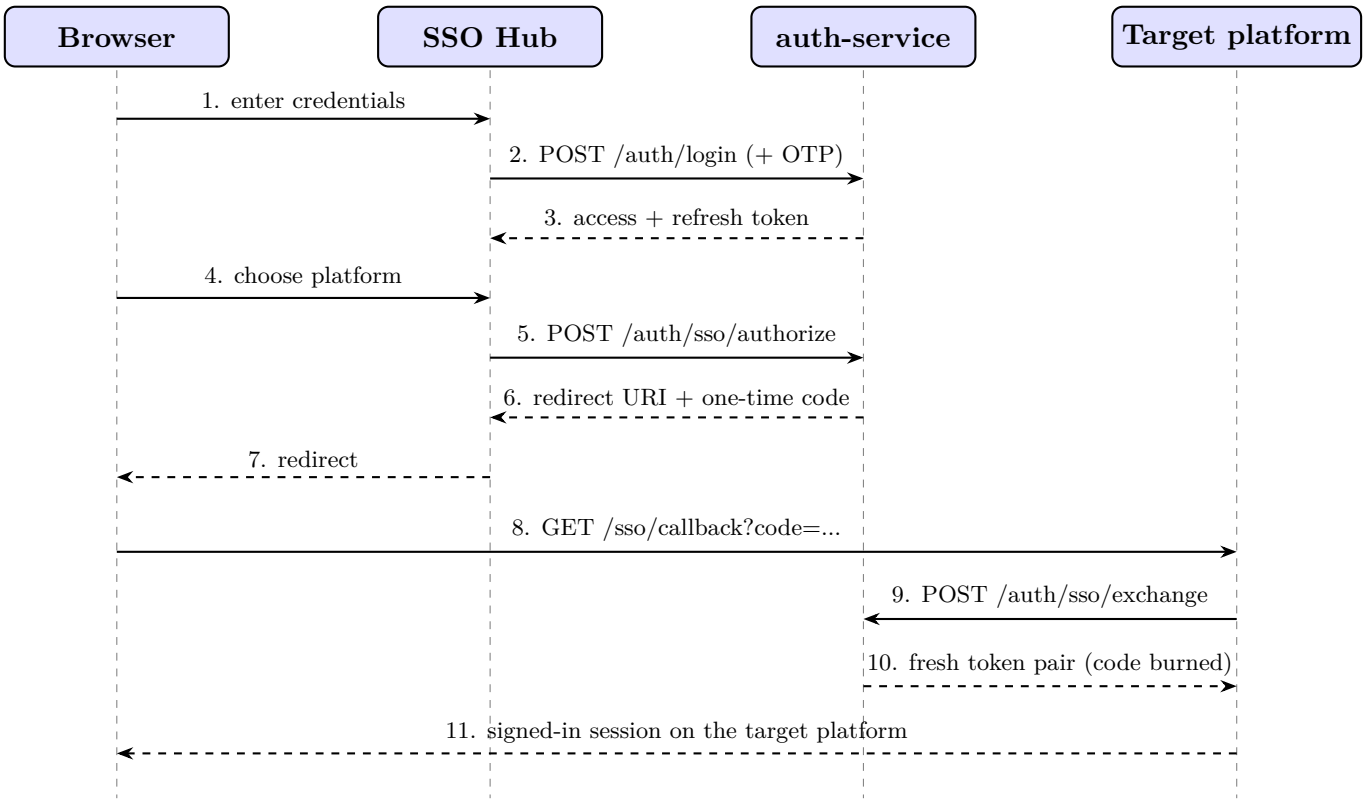


Figure 4.11: Central single sign-on flow with a single-use, platform-bound code

4.9.3 Application State Transition Diagram

Figure 4.12 shows the states of a placement application and the allowed transitions.

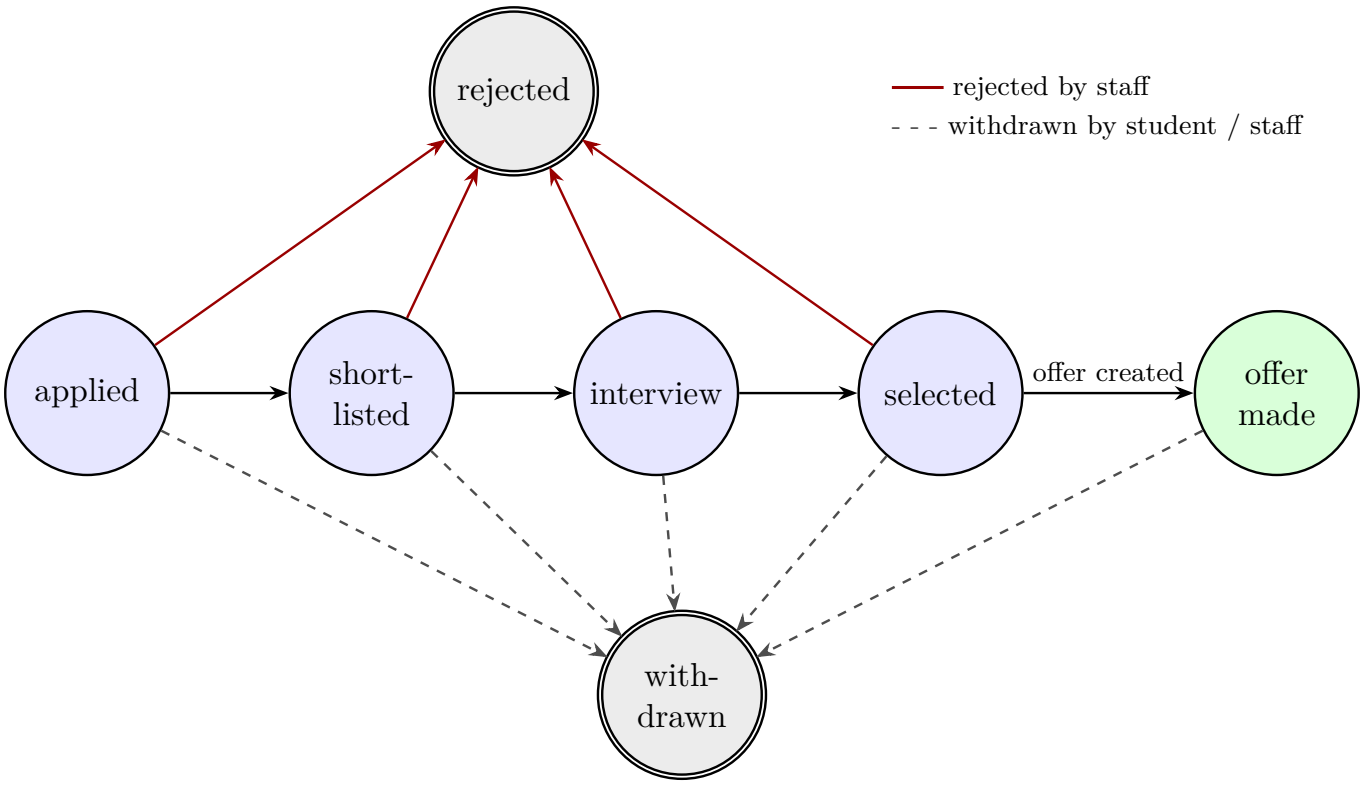


Figure 4.12: Application life-cycle state machine

Chapter 5

System Methodology

This chapter presents the proposed platform, its stakeholders, functional modules, embedded algorithms and the methodology followed for building and validating the system. Following the department template, each algorithm is first written as steps, then explained, and finally given as pseudocode.

5.1 Proposed Methodology

The development followed an Agile, incremental methodology in phases, each with a clear, demonstrable goal, followed by security audits and regression tests before moving on.

5.1.1 Requirement Analysis

Requirements were gathered from the placement office, faculty and students, from the workflows observed during the mini project and from the needs of the first two partner colleges. User stories were written per role (for example "as a placement officer I want only eligible students to be able to apply") and a feature backlog was maintained with identifiers (E1–E14, N1–N32) that are referenced in code comments and tests.

5.1.2 System Design

An architecture specification was written first and treated as the single source of truth: stack, service boundaries, database design, security model, subscription tiers and build order. ER models, the permission model and sequence diagrams for login, refresh and SSO were designed before implementation.

5.1.3 Incremental Implementation

Table 5.1: Implementation phases and goals

Phase	Work carried out	Goal / exit criterion
Phase 1	Local infrastructure (PostgreSQL, Redis), auth-service skeleton, all `auth_db` migrations, seeders (super admin, roles, pages, plans), console e-mail stub.	Super admin logs in locally and receives a signed JWT.
Phase 2	Registration, refresh rotation, logout, MFA, password reset, RBAC endpoints, tenant CRUD, subscriptions, tenant provisioning, audit, rate limits, lockout.	Two colleges created through the API; full auth works.
Phase 3	Core service: local JWT verification, permission middleware, students (+ CSV import via worker), classes, departments, attendance, placement, reports, notifications.	Import students, mark attendance and run a placement drive locally.
Phase 4	React frontend: login/MFA pages, permission-gated shell, super admin panel, role and user management, students, attendance scanner, placement UI.	Full UI working against the local backend.
Security passes	RLS, field encryption, step-up, passkeys, SSRF guard, concurrency caps, audit redaction; security groups 1–11, A–E, R3.	Automated security regression suite passes.
Phase 5	Real e-mail providers (Resend / Amazon SES), bulk e-mail with cost preview, web-push.	Real verification, reset and notification e-mails.
Phase 6	Production deployment: Cloudflare, Traefik/Dokploy, PgBouncer, R2 backups, GitHub Actions CI/CD.	Colleges live in production.
Integration	Central SSO hub, examination-platform provisioning and delete-sync, resume builder with self-hosted AI, off-campus opportunity feed.	One login across platforms; AI-assisted resumes.

5.1.4 Testing and Deployment

Every phase ended with automated smoke tests against the running services, and every security change added a regression test that reproduces the original weakness. Before release, a role-by-role manual walkthrough was performed on the production deployment. Deployment uses Docker images built by GitHub Actions, database migrations run before new images start, and a health check gates each rollout.

5.1.5 System Overview

FortifyHub is a full-stack, multi-tenant web platform that streamlines the academic and placement life cycle of many colleges from one deployment. It manages structured academic data, enforces institutional rules automatically, integrates assessments and analytics, and protects every tenant's data with layered security.

The system comprises:

- **Frontend:** a React single-page application with role-specific dashboards, permission-gated navigation, responsive layouts and barcode scanning.
- **Authentication service:** identity, MFA, RBAC, tenants, subscriptions, SSO and audit.
- **Core service:** academic, attendance, placement, communication and reporting modules.
- **Worker:** asynchronous and scheduled processing.
- **Databases and infrastructure:** PostgreSQL (two databases), PgBouncer, Redis, R2, Traefik and Cloudflare.

5.1.6 Contributions

The major project delivers:

- A production multi-tenant SaaS architecture where colleges are added without code changes and isolated in both application and database layers.
- A hardened authentication service implementing argon2id, RS256 JWTs with local verification, rotating refresh tokens with theft detection, TOTP, passkeys, step-up OTP and SSO.
- A rule-based eligibility engine, application state machine and tiered two-offer policy engine that encode real placement-office rules.
- Barcode-based attendance with an exactly-once guarantee enforced by the database.
- Privacy-preserving AI assistance for resumes using a self-hosted small language model.
- Integration with an examination platform that grades code server-side in a sandbox.
- An automated smoke and security regression suite and a production deployment with CI/CD, pooling and backups.

5.2 Algorithms

Algorithms and Rule Engines Used

This section summarises the key algorithms and rule engines embedded in the platform. Each one is written below as numbered steps.

Algorithm 1: Eligibility Checking

Step 1: Start with a filter restricted to the caller's tenant.

Step 2: If the drive restricts years or departments, find the matching classes in the tenant; if none match, return an empty list; otherwise restrict the filter to those classes.

Step 3: Add the numeric thresholds that are set: minimum CGPA, minimum 10th and 12th percentage, maximum active backlogs and maximum gap years.

Step 4: Add the batch condition (exact batch, batch range ending in the year, or graduation year) and the allowed semesters, if set.

Step 5: Exclude students who must not be considered (for example, already placed) and run one tenant-scoped query ordered by roll number.

Step 6: If required skills are set, keep only candidates whose skills or projects contain every required skill.

Step 7: Return the eligible students.

Algorithm 2: Application Workflow State Machine

Step 1: Read the current status of the application and the requested new status.

Step 2: Look up the requested transition in the table of allowed transitions (applied $\rightarrow$ shortlisted $\rightarrow$ interview $\rightarrow$ selected $\rightarrow$ offer made, with rejection by staff and withdrawal by the student).

Step 3: If the transition is not allowed, reject the request with `INVALID_STATE_TRANSITION`.

Step 4: Otherwise update the status and append the transition, actor and time to the status history.

Algorithm 3: Tiered Two-Offer Placement Policy

Step 1: If the drive has no category or the college has no categories, allow the application.

Step 2: Count the student's accepted offers in each category.

Step 3: Mark as blocked every category whose offer cap has been reached.

Step 4: Find the highest category in which the student holds an offer and which blocks lower categories.

Step 5: If the drive's category is blocked, return `CATEGORY_CAP_REACHED`; if it is below the blocking category, return `CATEGORY_BLOCKED_BY_HIGHER`; otherwise allow the application.

Algorithm 4: Refresh Token Rotation with Theft Detection

Step 1: Hash the presented refresh token with SHA-256 and look it up; if it does not exist, return 401.

Step 2: If the token was already rotated (reuse), revoke every token of its family in a committed transaction and return `REFRESH_TOKEN_REUSED`.

Step 3: If the token has expired, return 401.

Step 4: In a transaction, lock the token row, re-check that it is still unrevoked and that the user is active.

Step 5: Issue a new access token and a new refresh token in the same family, mark the old token as revoked and replaced, and commit.

Step 6: Return the new token pair.

Algorithm 5: Single Sign-On with One-Time Codes

Step 1: The user signs in at the SSO hub and receives an access and refresh token pair.

Step 2: The user chooses a platform; the hub asks the auth service to authorise SSO for that platform.

Step 3: The auth service creates a 256-bit, single-use, short-lived code bound to the user, tenant and platform, and returns a redirect URI taken from server configuration.

Step 4: The browser is redirected to the target platform's callback with the code.

Step 5: The target platform exchanges the code with the auth service (with its client secret); the code is burned and a fresh token pair is issued.

Step 6: The user is signed in on the target platform.

Algorithm 6: Attendance Marking with an Exactly-Once Guarantee

Step 1: Build an idempotency key from the session and the scanned barcode; if a cached result exists, return it.

Step 2: Find the student with that barcode in the tenant (404 if unknown) and check that the student belongs to one of the session's classes (409 if not).

Step 3: In a transaction, take an advisory lock on (session, student) and insert the attendance record if it is not present; a trigger records it in the unique de-duplication ledger.

Step 4: Commit, cache the result for a short time and write an audit entry.

Algorithm 7: Readiness and Engagement Indicators

Step 1: Select the group of students (a class or a department).

Step 2: Count the students with a complete profile, an uploaded resume, at least one application and at least one accepted offer.

Step 3: Divide each count by the group size to obtain P_{profile}, P_{resume}, P_{applied} and P_{placed}.

Step 4: Report the indicators per class and department so that faculty can act on low values before a drive.

Algorithm 8: Resume Project Relevance Ranking

Step 1: Take the target role and its keyword set.

Step 2: For each project, compute the score from keyword matches, number of tools, description length (capped) and system-level scope.

Step 3: Sort projects by score, keep the top N (default 4) and reorder skills so that role-relevant skills come first.

Step 4: Optionally send only the project descriptions to the self-hosted model for rewriting.

Algorithm 9: Server-side Code Judging

Step 1: Ignore any result reported by the browser.

Step 2: Run the first test case; if it fails to compile, return *Compilation Error* without running more cases.

Step 3: Run every test case in the sandbox with per-language time and memory limits and compare the output using the comparison rules or a custom checker.

Step 4: Compute the score as the weighted fraction of passed cases and the overall verdict as the worst non-accepted verdict; report infrastructure failures as *Internal Error* without penalising the student.

Algorithm 10: SSRF Validation for External Job Feeds

Step 1: Reject the URL unless it uses HTTPS.

Step 2: Resolve the host name and reject it unless every address is a public unicast address.

Step 3: Repeat the check at connect time and pin the socket to the validated address.

Step 4: Refuse redirects and cap the response size and time.

5.3 Explanation of the Algorithms

5.3.1 Eligibility Checking

Each job stores its eligibility rules as a JSON document. The engine translates the rules into one tenant-scoped database query and applies the skills check in memory on the reduced candidate set. Missing data is treated conservatively: a student without a recorded 10th percentage cannot enter a drive that requires one.

When a student (or an officer on behalf of students) applies, three further gates run per student: a resume must be on file, the student must have acknowledged the current placement policy, and the tiered offer policy (Section 5.3.3) must allow the drive. Each rejection returns a machine-readable reason (`INELIGIBLE`, `NO_RESUME`, `POLICY_ACK_REQUIRED`, `CATEGORY_CAP_REACHED`, `ALREADY_APPLIED`).

5.3.2 Application Workflow State Machine

Applications move through a finite state machine; any other transition is rejected with `INVALID_STATE_TRANSITION`, and each transition is appended to the application's status history with the actor and time. The allowed transitions are shown in Figure 4.12.

5.3.3 Tiered Two-Offer Placement Policy

Colleges classify drives into categories by CTC (for example *Normal*, *Dream*, *Super Dream*), each with a sort order, a maximum number of accepted offers and a flag that blocks lower categories. The policy engine decides at apply time whether a student may sit for a drive:

The same computation drives the student-facing "offer status" panel, which shows for every category how many offers are held and whether the student can still apply.

5.3.4 Refresh Token Rotation with Theft Detection

If an attacker steals a refresh token and uses it after the legitimate client has already rotated it (or vice versa), the second use is detected and the whole chain is invalidated, forcing a fresh login.

5.3.5 Single Sign-On with One-Time Codes

Figure 4.11 shows the SSO protocol used by the central hub. It follows the authorization-code idea of OAuth 2.0 [14]: tokens are never passed between origins in URLs, only a short-lived, single-use code that is bound to the target platform.

5.3.6 Attendance Marking with an Exactly-Once Guarantee

A barcode scanner (USB serial or phone camera) submits the student's ID-card code. The server resolves the student in the current tenant, verifies that the student belongs to one of the session's classes and marks attendance. Two concurrent scans of the same card could both see "no record yet" and insert twice, so the marking step takes a transaction-scoped advisory lock keyed on *(session, student)* and a database trigger maintains a de-duplication ledger with a unique constraint. A short-lived idempotency key in Redis returns the same response for repeated scans of the same card.

5.3.7 Readiness and Engagement Indicators

The analytics module computes cohort indicators that act as a placement-readiness view for each department and class. For a group G of students:

$$P_{\text{profile}} = \frac{|\{s \in G : \text{cgpa, phone, email and batch present}\}|}{|G|},$$

$$P_{\text{resume}} = \frac{|\{s \in G : \text{resume uploaded}\}|}{|G|},$$

$$P_{\text{applied}} = \frac{|\{s \in G : \text{at least one application}\}|}{|G|},$$

$$P_{\text{placed}} = \frac{|\{s \in G : \text{at least one accepted offer}\}|}{|G|}.$$

Institution-level analytics additionally report the placement percentage, average/highest/lowest CTC, CTC distribution by band, top recruiters, department-wise placement percentage and a year-on-year trend by graduating batch. Low P_{profile} or P_{resume} values point faculty to classes that need intervention before a drive.

5.3.8 Resume Project Relevance Heuristic

For a chosen target role r (full-stack, backend, data, ML, cybersecurity, DevOps, etc.) with keyword set K_r, each project p is scored as

$$\mathit{Score}(p,r) = 100 \cdot |K_r \cap \text{words}(p)| \; + \; 6 \cdot |\text{tools}(p)| \; + \; \min(\text{descWords}(p), 40) \; + \; 8 \cdot \text{scope}(p)$$

where $\text{scope}(p) = 1$ if the title or description mentions a system-level scope (platform, full-stack, pipeline, distributed, real-time, etc.). Role relevance dominates; tools, description length and scope break ties. The top N projects (default 4) appear on the one-page resume, and skills are re-ordered so that role-relevant skills come first. The optional AI step then rewrites only the description text of the selected items.

5.3.9 Server-side Code Judging

The examination platform never trusts results reported by the browser. For each coding answer the server runs every test case through the sandbox with per-language time and memory multipliers:

- a compilation error on the first case marks the whole submission as *Compilation Error* without running further cases;
- for each case the sandbox verdict (accepted, time/memory/output limit exceeded, runtime error) is combined with an output comparison (whitespace, newline, case and floating-point tolerance rules) or a custom checker to decide *Accepted* or *Wrong Answer*;
- the score is the weighted fraction of passed cases, and the overall verdict is the worst non-accepted verdict; infrastructure failures are reported as *Internal Error* and never penalise the student.

5.3.10 SSRF Validation for External Job Feeds

Off-campus opportunities are imported from administrator-configured job-feed providers. Because the server fetches these URLs, every request is validated: HTTPS only; every DNS answer must be a public unicast address (loopback, private, link-local including the cloud metadata address `169.254.169.254`, CGNAT, multicast and reserved ranges are rejected); the check is repeated at connect time and the socket is pinned to the validated IP to defeat DNS rebinding; redirects are disabled; and response size (5 MB) and time (12 s) are capped.

5.4 Pseudocode

5.4.1 Eligibility Checking

```
Input : tenant, eligibility E, optional studentIds, excludeIds
Output: list of eligible students

where <- { tenant_id = tenant }
if E.allowed_years or E.allowed_departments then
    classIds <- classes in tenant matching years / departments
    if classIds is empty then return []
    where.class_id IN classIds
if E.min_cgpa        != null then where.cgpa               >= E.min_cgpa
if E.min_tenth_pct   != null then where.tenth_percentage   >= E.min_tenth_pct
if E.min_twelfth_pct != null then where.twelfth_percentage >= E.min_twelfth_pct
if E.batch != null then
    where (batch = E.batch) OR (batch LIKE '%-' + E.batch) OR (graduation_year = E.
    batch)
```

```
if E.max_backlogs  != null then (active_backlogs IS NULL OR active_backlogs <= E.
    max_backlogs)
if E.max_gap_years != null then (gap_years IS NULL OR gap_years <= E.max_gap_years)
if E.allowed_semesters not empty then where.semester IN E.allowed_semesters
if studentIds given then where.id IN studentIds
if excludeIds given then where.id NOT IN excludeIds      // e.g. already placed

candidates <- SELECT students WHERE where ORDER BY roll_no
if E.required_skills not empty then
    keep only candidates whose skills/projects contain every required skill
return candidates
```

5.4.2 Application Workflow State Machine

```
Input : application a, requested status s', actor u
ALLOWED <- { applied -> shortlisted | rejected | withdrawn,
             shortlisted -> interview | rejected | withdrawn,
             interview -> selected | rejected | withdrawn,
             selected -> offer_made | rejected | withdrawn }
if s' not in ALLOWED[a.status] then
    return error INVALID_STATE_TRANSITION
a.status_history.append({ from: a.status, to: s', by: u, at: now })
a.status <- s'
save a
```

5.4.3 Tiered Two-Offer Placement Policy

```
Input : student s, job category c
Output: null (allowed) or a block reason

if c is null or tenant has no categories then return null
held[k]       <- number of accepted offers of s in category k
blockedCats   <- { k : held[k] >= k.max_offers_in_category }
blockedBelow  <- max(sort_order of k with held[k] > 0 and k.blocks_lower_categories)
if c in blockedCats then return CATEGORY_CAP_REACHED
if blockedBelow != null and c.sort_order < blockedBelow then
    return CATEGORY_BLOCKED_BY_HIGHER
return null
```

5.4.4 Refresh Token Rotation with Theft Detection

```
Input : presented refresh token t
h   <- SHA256(t)
row <- find refresh_token by hash h           ; if none -> 401 INVALID
if row.revoked_at != null then                 ; token already rotated = reuse
```

```
    revoke every token with family_id = row.family_id  (own committed tx)
    return 401 REFRESH_TOKEN_REUSED
if row.expires_at < now then return 401 EXPIRED
BEGIN TRANSACTION
    lock row FOR UPDATE; re-check revoked_at   ; concurrent reuse race
    if revoked now -> revoke family after rollback, 401
    check user is active
    issue new access JWT + refresh token in the same family
    row.revoked_at <- now; row.replaced_by_id <- new token id
COMMIT
return new token pair
```

5.4.5 Single Sign-On with One-Time Codes

```
authorize(user, tenant, platform):
    code <- random 256-bit value
    store code -> { user, tenant, platform } with short TTL, single use
    return redirect_uri(platform) from server configuration + code

exchange(code, platform, client_secret):
    verify client_secret of platform
    entry <- take and delete code            // burned on first use
    if entry is null or entry.platform != platform then reject
    return new access token + refresh token for entry.user
```

5.4.6 Attendance Marking with an Exactly-Once Guarantee

```
scan(session, barcode):
    key <- "scan:" + session.id + ":" + barcode
    if Redis has key then return cached result            // repeated scan
    student <- find student by barcode in tenant           // 404 if unknown
    assert student.class_id in classes(session)            // 409 if not enrolled
    BEGIN
        pg_advisory_xact_lock(hash(session.id, student.id))
        insert attendance_record(method = 'barcode') if not present
        trigger -> insert attendance_dedup (UNIQUE session_id, student_id)
    COMMIT
    cache result under key (short TTL); audit 'attendance.mark'
```

5.4.7 Server-side Code Judging

```
judge(submission, testcases):
    ignore any result sent by the browser
    results <- []
    for each case t in testcases:
```

```
        r <- run in sandbox(submission, t, time and memory limits per language)
        if r = CompilationError then return CompilationError, score 0
        if r = Accepted and not outputMatches(r.output, t.expected) then r <-
    WrongAnswer
        results.append(r)
    score   <- sum(weight of accepted cases) / sum(all weights)
    verdict <- worst non-accepted verdict in results, else Accepted
    return verdict, score            // infrastructure failure -> InternalError
```

5.4.8 SSRF Validation for External Job Feeds

```
safeFetch(url):
    if scheme(url) != "https" then reject
    addrs <- DNS resolve(host(url))
    if any addr in addrs is loopback, private, link-local, CGNAT,
       multicast or reserved then reject
    connect to a validated addr only (re-checked at connect time)
    disallow redirects; cap response size and time
    return response
```

Chapter 6
Implementation and Experimental Setup

This chapter presents the implementation of the major functional and security components of the platform:

- **Authentication service** – password hashing, token issuing and refresh-token rotation.
- **Core service** – local token verification, permission checks, Row-Level Security, field encryption, eligibility, applications, offer policy, attendance and the SSRF guard.
- **Background worker, AI service and frontend** – queues, resume rewriting and permission-aware UI.
- **Examination integration and deployment** – server-side judging, containers and CI/CD.

Code excerpts are taken from the project repository and shortened for readability (omitted parts are marked with . . .).

In addition, following the department template, this chapter describes the test cases and data set used to evaluate the platform, the experimental setup in which it was built and deployed, and how the system is used.

6.1 Test Cases

Test cases were derived from the functional requirements (FR1–FR10), the security requirements and the defects found in internal security audits. Each test case records an identifier, the scenario, the expected result, the actual result and the outcome; a case that failed when first executed is marked *Fail (fixed)* and was re-run after the fix. In total, 58 test cases (TC01–TC58) were defined in six groups, summarised in Table 6.1; the full test cases are listed in Chapter 7.

Table 6.1: Test case groups and outcomes

Test case group	TC range	Cases	Passed first time	Failed, then fixed
Authentication and access control	TC01–TC14	14	11	3
Multi-tenancy and data protection	TC15–TC23	9	6	3
Student portal functionality	TC24–TC33	10	9	1
Admin, placement officer, faculty and super admin	TC34–TC44	11	7	4
Attendance and background processing	TC45–TC51	7	5	2
Coding verdict and integrity	TC52–TC58	7	6	1
Total		**58**	**44**	**14**

In addition to the documented cases, an automated runner (`tests/run-all.js`) executes a cumulative smoke test, feature smoke tests and twelve security regression groups against the running services after every phase (Section 7.3).

6.2 Data Set

FortifyHub is a transactional platform rather than a machine-learning model, so no training data set is used. The data used to build, test and demonstrate the system is:

- **Reference (seed) data:** idempotent seeders create the platform super admin, the default role templates, the global page catalogue and the subscription plans.
- **Tenant data:** colleges created through the API, each with departments, classes and students entered individually or through bulk CSV import (TC36, TC48), together with companies, drives, drive categories and placement policies.
- **Assessment data:** coding problems with visible and hidden test cases on the examination platform, used to verify every verdict type (TC52–TC58).
- **Synthetic load data:** a k6 script generating up to 1,000 virtual students who log in, open an exam and save answers.
- **Protection of the data:** sensitive student fields are stored encrypted with AES-256-GCM (TC20), and every tenant's rows are isolated by Row-Level Security (TC18, TC19).

6.3 Experimental Setup

The experiments use the hardware and software listed in Sections 3.4 and 3.5 and the two deployment scenarios of Section 3.1.3: the local-development scenario (compose file in Appendix G), where every phase was verified by the automated suite against the locally running services, and the single-server production scenario, on which the acceptance walkthrough was performed. The examination platform and the optional self-hosted language model complete the setup.

6.3.1 System Implementation Overview

The project is implemented as a monorepo with one folder per service and a shared folder for code used by more than one service (field cryptography, file helpers). All services are written in JavaScript (CommonJS) on Node.js 20, follow the convention *Joi validation* → *controller* → *service* → *model* and return structured errors of the form `{ error: { code, message } }`. Table 4.1 summarises the technology stack and Table 6.2 the size of the code base.

Table 6.2: Approximate size of the placement platform code base (lines, excluding dependencies)

Component	Lines of code
auth-service (JavaScript)	11,367
core-service (JavaScript)	16,290
worker (JavaScript)	2,148
web frontend (JSX + CSS)	27,925
Automated smoke and security tests	4,232
Total	**61,962**

6.3.2 Repository Structure

```
/placement
|-- ARCHITECTURE.md              single source of truth (stack, data model, security)
|-- docker-compose.local.yml     local PostgreSQL + Redis
|-- services/
|   |-- auth/                    auth-service: controllers, middleware, models,
|   |                            services, validators, migrations, seeders
|   |-- core/                    core-service (modular monolith)
|   |   `-- src/modules/         students, classes, departments, attendance,
|   |                            placement, notifications, email, reports, ...
|   |-- worker/                  BullMQ consumers + scheduler
|   `-- resume-ai/               self-hosted description rewriter (Ollama)
|-- shared/crypto/               AES-256-GCM field encryption (used by core + worker)
|-- web/                         React + Vite frontend
|-- login-portal/                central SSO hub (React + Vite)
|-- deploy/                      production compose, Traefik, backups, droplet setup
|-- infra/pgbouncer/             connection-pool configuration
`-- tests/                       smoke + security regression suites
```

6.3.3 Deployment and CI/CD Implementation

Production Compose (excerpt)

```
core:
  image: ghcr.io/${GITHUB_OWNER}/placement-core:${IMAGE_TAG:-latest}
  restart: unless-stopped
  depends_on:
    pgbouncer: { condition: service_healthy }
    redis:     { condition: service_healthy }
  environment:
    NODE_ENV: production
    DB_HOST: pgbouncer
    DB_PORT: "6432"
    AUTH_SERVICE_URL: http://auth:3001
    INTERNAL_SHARED_SECRET: ${INTERNAL_SHARED_SECRET}
```

```
    FIELD_ENCRYPTION_KEY: ${FIELD_ENCRYPTION_KEY}
    ...
  labels:
    - traefik.enable=true
    - "traefik.http.routers.core.rule=Host(`${PUBLIC_DOMAIN}`) && PathPrefix(`/api`)"
    - traefik.http.routers.core.tls.certresolver=le
    - traefik.http.routers.core.middlewares=secHeaders@file,rateLimit@file
  networks: [pp_net]
```

CI/CD Pipeline

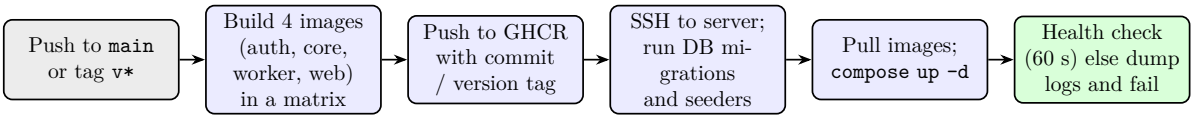


Figure 6.1: Continuous integration and deployment pipeline

Migrations are additive (columns are never dropped mid-deploy), so the old version keeps working while the new images start. Rollback is performed by redeploying a previous image tag.

6.4 Implementation of the Modules

6.4.1 Authentication Service Implementation

Password Hashing with argon2id and a Concurrency Limit

Argon2 is deliberately expensive. On a small server, hundreds of simultaneous logins could saturate every core, so hashing is wrapped in a semaphore: excess logins wait briefly in a queue instead of timing out.

```
const os = require('os');
const argon2 = require('argon2');

// OWASP minimum: 19 MB memory, 2 iterations (tunable via env).
// Older, heavier hashes are re-hashed lazily on the next successful login.
const ARGON2_OPTIONS = {
  type: argon2.argon2id,
  memoryCost: (parseInt(process.env.ARGON2_MEMORY_MB, 10) || 19) * 1024, // KiB
  timeCost: parseInt(process.env.ARGON2_TIME_COST, 10) || 2,
  parallelism: 1,
};

// Semaphore: at most MAX hashes run at once; the rest queue.
const MAX = parseInt(process.env.ARGON2_MAX_CONCURRENCY, 10)
```

```
  || Math.max(2, (os.cpus().length || 2) * 2);
let active = 0;
const queue = [];
function acquire() {
  return new Promise((resolve) => {
    if (active < MAX) { active += 1; resolve(); }
    else queue.push(resolve);
  });
}
function release() {
  active -= 1;
  const next = queue.shift();
  if (next) { active += 1; next(); }
}
async function withLimit(fn) {
  await acquire();
  try { return await fn(); }
  finally { release(); }
}
```

Refresh Token Rotation with Theft Detection

```
async function rotate({ refreshToken, ip, userAgent }) {
  if (!refreshToken) {
    throw errors.unauthorized('MISSING_REFRESH_TOKEN', 'refresh_token is required');
  }
  const tokenHash = hashRefreshToken(refreshToken);        // SHA-256, never stored
    raw
  const row = await RefreshToken.findOne({ where: { token_hash: tokenHash } });
  if (!row) throw errors.unauthorized('INVALID_REFRESH_TOKEN', 'Refresh token is
    invalid');

  // Reuse of an already-rotated token => assume theft, kill the whole family.
  if (row.revoked_at) {
    await revokeFamilyAutocommit(row.family_id, 'theft_suspected');
    throw errors.unauthorized('REFRESH_TOKEN_REUSED', 'Refresh token has been reused -
     family revoked');
  }
  if (row.expires_at < new Date()) {
    throw errors.unauthorized('REFRESH_TOKEN_EXPIRED', 'Refresh token has expired');
  }

  // Revoke + issue + link in ONE transaction, under a row lock.
  let inTxReuseFamilyId = null;
  try {
    return await sequelize.transaction(async (tx) => {
      const locked = await RefreshToken.findByPk(row.id, { transaction: tx, lock: tx.
    LOCK.UPDATE });
```

```
      if (locked.revoked_at) {                              // concurrent reuse race
        inTxReuseFamilyId = locked.family_id;
        throw errors.unauthorized('REFRESH_TOKEN_REUSED', 'Refresh token has been
    reused - family revoked');
      }
      const user = await User.findByPk(locked.user_id, { transaction: tx });
      if (!user || user.status === 'archived' || user.status === 'suspended') {
        throw errors.forbidden('ACCOUNT_INACTIVE', 'Account is not active');
      }
      const fresh = await tokenIssuer.issueTokensForUser({
        user, ip, userAgent, familyId: row.family_id, transaction: tx,
      });
      locked.revoked_at = new Date();
      locked.revoke_reason = 'rotated';
      locked.replaced_by_id = fresh.refresh_token_id;
      await locked.save({ transaction: tx });
      return { status: 'ok', ...fresh };
    });
  } catch (err) {
    if (inTxReuseFamilyId) await revokeFamilyAutocommit(inTxReuseFamilyId, '
    theft_suspected_race');
    throw err;
  }
}
```

Step-up Verification Middleware

Sensitive routes add `requireStepUp` after their permission check. The token is single-use, bound to the user, tenant, action and target, and is consumed before the handler runs.

```
function requireStepUp(opts) {
  const resolveAction = typeof opts.action === 'function' ? opts.action : () => opts.
    action;
  const resolveTarget = typeof opts.target === 'function' ? opts.target : (req) => req
    .params.id;
  return async function stepUpGate(req, res, next) {
    try {
      if (opts.when && !opts.when(req)) return next();     // conditional gating
      const result = await stepUpService.consume({
        token: req.headers['x-step-up-token'] || null,
        userId: req.auth.userId,
        tenantId: req.auth.tenantId,
        action: resolveAction(req),
        targetId: resolveTarget(req),
        ip: req.ip,
        userAgent: req.headers['user-agent'],
      });
      if (result.ok) return next();
      // 403 STEP_UP_REQUIRED (no token) or a generic STEP_UP_INVALID
```

```
      ...
    } catch (err) { return next(err); }
  };
}
```

6.4.2 Core Service Implementation

Local JWT Verification

```
// Verifies the Bearer access JWT LOCALLY with auth-service's public key, then
// checks the Redis revocation list. auth-service being down does not take
// core-service down; only NEW logins fail.
async function requireAuth(req, res, next) {
  try {
    const header = req.headers.authorization || '';
    const match = header.match(/^Bearer\s+(\S+)$/);
    if (!match) throw errors.unauthorized('MISSING_TOKEN', 'Authorization Bearer token
     required');

    let payload;
    try {
      payload = verifyAccessToken(match[1]);      // RS256, public key only
    } catch (err) {
      const code = err && err.name === 'TokenExpiredError' ? 'TOKEN_EXPIRED' : '
    INVALID_TOKEN';
      throw errors.unauthorized(code, 'Access token is invalid or expired');
    }
    if (payload.jti) {
      const revoked = await redis.get(`revoked:${payload.jti}`);
      if (revoked) throw errors.unauthorized('TOKEN_REVOKED', 'Access token has been
    revoked');
    }
    req.auth = {
      userId: payload.sub,
      tenantId: payload.tenant_id || null,
      jti: payload.jti,
      permissionHash: payload.permission_hash || null,
      exp: payload.exp,
    };
    return next();
  } catch (err) {
    return next(err);
  }
}
```

Permission Middleware

```
function requirePermission(pageCode, action) {
  return async function permissionCheck(req, res, next) {
    try {
      if (!req.auth || !req.auth.userId) {
        throw errors.unauthorized('UNAUTHENTICATED', 'requireAuth must run before
    requirePermission');
      }
      // Redis cache; falls back to auth-service /internal/perms on a miss
      // or when the token's permission_hash shows the perms have changed.
      const { perms } = await permissionsClient.getForUser(req.auth.userId, req.auth.
    permissionHash);
      const actions = perms[pageCode];
      if (!actions || !actions.includes(action)) {
        throw errors.forbidden('PERMISSION_DENIED', `Missing permission ${action} on $
    {pageCode}`);
      }
      req.auth.permissions = perms;
      return next();
    } catch (err) {
      return next(err);
    }
  };
}

// Route wiring in the placement module (routes.js)
router.use(requireAuth, requireTenant);   // every placement route is authenticated
router.use(rlsContext);                   // ...and runs in a tenant-scoped
    transaction
router.post('/jobs/:id/apply', placementWriteLimiter,
  requirePermission('placement.applications', 'create'), c.apply);
```

Row-Level Security Policies

The RLS migration enables and *forces* a tenant-isolation policy on every tenant table. `FORCE` is essential because the application user owns the tables, and PostgreSQL otherwise exempts owners from RLS. If `app.tenant_id` is not set, the cast fails and the query is rejected – the policy fails closed.

```
const TABLES = ['departments', 'classes', 'class_enrollments', 'students',
  'student_languages', 'student_projects', 'student_internships', 'sessions',
  'session_classes', 'attendance_records', 'attendance_dedup', 'companies',
  'jobs', 'applications', 'offers', 'import_jobs', 'notifications', 'reports'];

async up(queryInterface) {
  const sql = queryInterface.sequelize;
  for (const t of TABLES) {
    await sql.query(`ALTER TABLE ${t} ENABLE ROW LEVEL SECURITY;`);
```

```
    await sql.query(`ALTER TABLE ${t} FORCE  ROW LEVEL SECURITY;`);
    await sql.query(`DROP POLICY IF EXISTS ${t}_tenant_isolation ON ${t};`);
    await sql.query(`
      CREATE POLICY ${t}_tenant_isolation ON ${t}
        USING      (tenant_id = current_setting('app.tenant_id', true)::uuid)
        WITH CHECK (tenant_id = current_setting('app.tenant_id', true)::uuid);`);
  }
}
```

Each authenticated request runs inside a transaction that sets the tenant with a transaction-local setting, so the value can never leak to another request that later reuses the pooled connection. The response is held until the `COMMIT` has completed, so a client can never observe a response before its data is committed.

```
function rlsContext(req, res, next) {
  if (!req.auth || !req.auth.tenantId) return next();
  sequelize.transaction(async () => {
    // SET LOCAL semantics: gone on COMMIT/ROLLBACK, pool stays clean.
    await sequelize.query("SELECT set_config('app.tenant_id', $1, true)", {
      bind: [req.auth.tenantId],
    });
    return new Promise((commit, rollback) => {
      // res.json / res.send / res.end are patched to await COMMIT first,
      // then flush the response; client disconnects trigger ROLLBACK.
      ...
      next();   // handler runs inside the CLS-scoped transaction
    });
  });
}
```

Field-level Encryption (AES-256-GCM)

```
const crypto = require('crypto');
const ALGO = 'aes-256-gcm';
const IV_LEN = 12;            // 96-bit random nonce per write
const TAG_LEN = 16;           // 128-bit authentication tag
const PREFIX = 'v1:';         // version prefix enables key rotation later

function encrypt(plaintext) {
  if (plaintext == null) return plaintext;                  // NULL stays NULL
  const key = getKey();                                     // 32-byte key from env
  const iv = crypto.randomBytes(IV_LEN);
  const cipher = crypto.createCipheriv(ALGO, key, iv);
  const ct = Buffer.concat([cipher.update(String(plaintext), 'utf8'), cipher.final()])
    ;
  const tag = cipher.getAuthTag();
  return PREFIX + Buffer.concat([iv, tag, ct]).toString('base64');
}
```

```
16
17 // FAIL-CLOSED: a tampered value or a wrong key throws; never returns garbage.
18 function decrypt(stored) {
19   if (stored == null || !stored.startsWith(PREFIX)) return stored; // legacy plaintext
20   const blob = Buffer.from(stored.slice(PREFIX.length), 'base64');
21   const iv = blob.subarray(0, IV_LEN);
22   const tag = blob.subarray(IV_LEN, IV_LEN + TAG_LEN);
23   const ct = blob.subarray(IV_LEN + TAG_LEN);
24   const decipher = crypto.createDecipheriv(ALGO, getKey(), iv);
25   decipher.setAuthTag(tag);
26   return Buffer.concat([decipher.update(ct), decipher.final()]).toString('utf8');
27 }
```

The service refuses to start if `FIELD_ENCRYPTION_KEY` is missing or does not decode to exactly 32 bytes, so encryption can never be silently disabled.

6.4.3 Placement Module Implementation

Eligibility Engine

```
1  async function findByEligibility(req, eligibility = {}, options = {}) {
2    const where = scopedWhere(req);                                    // tenant filter
3    const yearList = nonEmpty(eligibility.allowed_years);
4    const deptList = nonEmpty(eligibility.allowed_departments);
5    if (yearList || deptList) {
6      const classWhere = { tenant_id: req.auth.tenantId };
7      if (yearList) classWhere.year = { [Op.in]: yearList };
8      if (deptList) classWhere.department_id = { [Op.in]: deptList };
9      const ids = (await Class.findAll({ where: classWhere, attributes: ['id'] })).map((
       c) => c.id);
10     if (!ids.length) return [];
11     where.class_id = { [Op.in]: ids };
12   }
13   // Missing data fails the gate (a NULL 10th % cannot enter a 90% drive).
14   if (eligibility.min_cgpa != null)        where.cgpa = { [Op.gte]: eligibility.
       min_cgpa };
15   if (eligibility.min_tenth_pct != null)   where.tenth_percentage = { [Op.gte]:
       eligibility.min_tenth_pct };
16   if (eligibility.min_twelfth_pct != null) where.twelfth_percentage = { [Op.gte]:
       eligibility.min_twelfth_pct };
17
18   const ands = [];
19   if (eligibility.batch) {             // "2027" matches "2027", "2023-2027" or
       graduation_year
20     const b = String(eligibility.batch).trim();
21     const batchOr = [{ batch: b }, { batch: { [Op.like]: `%-${b}` } }];
22     if (/^\d{4}$/.test(b)) batchOr.push({ graduation_year: Number(b) });
23     ands.push({ [Op.or]: batchOr });
```

```
  }
  if (eligibility.max_backlogs != null) {
    ands.push({ [Op.or]: [{ active_backlogs: null },
                          { active_backlogs: { [Op.lte]: eligibility.max_backlogs } }]
     });
  }
  ...                                    // gap years, history of backlogs, semesters
  if (ands.length) where[Op.and] = ands;
  if (options.studentIds) where.id = { [Op.in]: options.studentIds };

  let rows = await Student.findAll({ where, order: [['roll_no', 'ASC']],
                                     limit: options.limit, offset: options.offset });
  ...                                       // required_skills check over the candidate
    set
  return rows;
}
```

Applying to a Drive

```
async function applyToJob(req, id, body) {
  const j = await loadJobInTenant(req, id);
  if (j.status !== 'open') throw errors.conflict('JOB_NOT_OPEN', 'Only open jobs
    accept applications');
  if (j.deadline && new Date(j.deadline) < new Date()) {
    throw errors.conflict('DEADLINE_PASSED', 'Application deadline has passed');
  }
  const requestedIds = body.student_ids && body.student_ids.length ? body.student_ids
    : [body.student_id];
  const eligibleRows = await studentsService.findByEligibility(req, j.eligibility, {
    studentIds: requestedIds });
  const eligibleSet = new Set(eligibleRows.map((s) => s.id));

  const accepted = [];
  const rejected = [];
  for (const sid of requestedIds) {
    if (!eligibleSet.has(sid)) { rejected.push({ student_id: sid, reason: 'INELIGIBLE'
     }); continue; }
    if (!(await studentFileService.hasResume(req, sid))) {
      rejected.push({ student_id: sid, reason: 'NO_RESUME' }); continue;
    }
    const policyReason = await offerPolicyService.checkApplyAllowed(req, sid, j.
    category_id);
    if (policyReason) { rejected.push({ student_id: sid, reason: policyReason });
    continue; }
    if (!(await policyService.hasAcked(req, sid))) {
      rejected.push({ student_id: sid, reason: 'POLICY_ACK_REQUIRED' }); continue;
    }
    try {
```

```
      const created = await Application.create({
        tenant_id: req.auth.tenantId, job_id: j.id, student_id: sid,
        status: 'applied', applied_at: new Date(),
        status_history: [{ status: 'applied', at: new Date().toISOString(), by: req.auth.userId }],
      });
      accepted.push(application(created));
    } catch (err) {
      if (err.name === 'SequelizeUniqueConstraintError') {        // UNIQUE(job_id, student_id)
        rejected.push({ student_id: sid, reason: 'ALREADY_APPLIED' });
      } else { throw err; }
    }
  }
  if (accepted.length) auditClient.record(req, { action: 'application.apply', target_type: 'job', target_id: j.id, ... });
  return { applied: accepted.length, rejected: rejected.length, requested: requestedIds.length,
           accepted, ineligible_or_duplicate: rejected };
}
```

Offer Policy Gate

```
async function checkApplyAllowed(req, studentId, jobCategoryId) {
  if (!jobCategoryId) return null;                    // uncategorised drives bypass policy
  const policy = await loadPolicy(req);               // categories + sort orders (tenant)
  if (policy.catById.size === 0) return null;         // tenant has no tiers defined
  const state = await loadStudentOfferState(req, studentId, policy);
  if (state.blockedCategoryIds.has(jobCategoryId)) return 'CATEGORY_CAP_REACHED';
  const jobSort = policy.sortOf(jobCategoryId);
  if (state.blockedBelowSort != null && jobSort != null && jobSort < state.blockedBelowSort) {
    return 'CATEGORY_BLOCKED_BY_HIGHER';
  }
  return null;
}
```

SSRF Guard for External Job Feeds

```
function isPrivateIPv4(ip) {
  const n = ipv4ToInt(ip);
  if (n === null) return true;                        // unparseable => unsafe
  const inRange = (base, bits) => (n >>> (32 - bits)) === (ipv4ToInt(base) >>> (32 - bits));
  return (
```

```
    inRange('0.0.0.0', 8)     || inRange('10.0.0.0', 8)    ||   // this-net, private
    inRange('100.64.0.0', 10) || inRange('127.0.0.0', 8)   ||   // CGNAT, loopback
    inRange('169.254.0.0', 16)||                                // link-local + metadata
    inRange('172.16.0.0', 12) || inRange('192.168.0.0', 16)||   // private
    inRange('192.0.0.0', 24)  || inRange('198.18.0.0', 15) ||   // IETF, benchmarking
    inRange('224.0.0.0', 4)   || inRange('240.0.0.0', 4)        // multicast, reserved
  );
}
// Fetch: HTTPS only, DNS answers validated, socket pinned to the validated IP
// inside the connect-time lookup (anti DNS-rebinding), maxRedirects = 0,
// 5 MB body cap and 12 s timeout.
```

6.4.4 Attendance Module Implementation

```
async function markAttendanceInTenant(req, { session, student, method, deviceId }) {
  const classIds = await classIdsForSession(req, session.id, session.class_id);
  if (!(await studentsService.isEnrolledInAnyClass(req, student.id, classIds))) {
    throw errors.badRequest('NOT_ENROLLED', 'Student is not enrolled in any class for this session');
  }
  return sequelize.transaction(async (tx) => {
    // Serialise concurrent scans of the same (session, student); released on COMMIT.
    await sequelize.query('SELECT pg_advisory_xact_lock(hashtextextended($1, 0))',
      { bind: [`attn:${session.id}:${student.id}`], transaction: tx });
    const existing = await AttendanceRecord.findOne({
      where: { tenant_id: req.auth.tenantId, session_id: session.id, student_id: student.id },
      transaction: tx });
    if (existing) return { already_marked: true, record: recordPayload(existing), student };
    try {
      const row = await AttendanceRecord.create({
        tenant_id: req.auth.tenantId, session_id: session.id, student_id: student.id,
        marked_at: new Date(), marked_by: req.auth.userId, method, device_id: deviceId || null,
      }, { transaction: tx });
      return { already_marked: false, record: recordPayload(row), student };
    } catch (err) {
      // DB trigger + attendance_dedup UNIQUE is the final guarantee.
      if (err.name === 'SequelizeUniqueConstraintError' || (err.original && err.original.code === '23505')) {
        return { already_marked: true, record: null, student };
      }
      throw err;
    }
  });
}
```

6.4.5 Background Worker Implementation

```
const allWorkers = [
  // [key, queue name, handler, concurrency]
  ['csvImport',            QUEUE_NAMES.csvImport,           csvImportHandler,
       1],
  ['notifications',        QUEUE_NAMES.notifications,       notificationsHandler,
       4],
  ['reports',              QUEUE_NAMES.reports,             reportsHandler,
       2],
  ['fileProcess',          QUEUE_NAMES.fileProcess,         fileProcessHandler,
       2],
  ['email',                QUEUE_NAMES.email,               emailHandler,
       1],
  ['trialMaintenance',     QUEUE_NAMES.trialMaintenance,    trialMaintenanceHandler,
       1],
  ['partitionMaintenance', QUEUE_NAMES.partitionMaintenance,
    partitionMaintenanceHandler, 1],
];
// Optional scoping: an overflow worker on another machine can process only
// self-contained queues, e.g. WORKER_QUEUES=reports,fileProcess,csvImport,email
const queueFilter = (process.env.WORKER_QUEUES || '').split(',').map((s) => s.trim()).
    filter(Boolean);
const wantQueue = (key) => queueFilter.length === 0 || queueFilter.includes(key);
for (const [key, qname, handler, conc] of allWorkers) {
  if (wantQueue(key)) workers.push(makeWorker(qname, handler, conc));
}
if (wantQueue('trialMaintenance') || wantQueue('partitionMaintenance')) {
  await registerRepeatableJobs();        // idempotent repeatable schedules
}
```

6.4.6 Resume Generation and AI Service

Role-based Project Ranking

```
const scopeWords = /(platform|system|full[- ]?stack|end[- ]?to[- ]?end|dashboard|
    engine|pipeline|real[- ]?time|distributed|scalable)/i;
const projectRows = projects
  .map((p) => {
    const tools = splitSkills(p.tools_used);
    const descWords = String(p.description || '').split(/\s+/).filter(Boolean).length;
    const relevance = roleScore(`${p.title} ${p.description} ${p.tools_used}`, role);
    const scope = scopeWords.test(`${p.title} ${p.description}`) ? 8 : 0;
    return { title: p.title, description: p.description || '', tools, link: p.
    repo_link || null,
```

```
            _score: relevance * 100 + tools.length * 6 + Math.min(descWords, 40) +
    scope };
  })
  .sort((a, b) => b._score - a._score)
  .map(({ _score, ...rest }) => rest)
  .slice(0, MAX_PROJECTS);              // top-N (default 4) on the one-page resume
```

Privacy-preserving Rewrite Endpoint

The AI service refuses to start without a shared secret, accepts only descriptions (never names, e-mail, phone or USN), caps batch size and always falls back to the original text.

```
app.use((req, res, next) => {
  if (req.get('X-Resume-Token') !== TOKEN) return res.status(401).json({ error: '
    Unauthorized' });
  next();
});

// POST /rewrite { studentId, items: [{ id, title?, description, tools?: [] }] }
app.post('/rewrite', async (req, res) => {
  const body = req.body || {};
  const studentId = String(body.studentId || 'anon');
  const items = Array.isArray(body.items) ? body.items.slice(0, 30) : [];   // sane
    cap
  if (!items.length) return res.json({ items: [] });
  try {
    // limit() = CPU-adaptive worker pool in front of Ollama
    const out = await Promise.all(items.map((it) => limit(() => rewriteOne(studentId,
    it))));
    res.json({ items: out });
  } catch (e) {
    // whole-batch failure => echo originals so a resume is always produced
    res.json({ items: items.map((it) => ({ id: it.id, description: String(it.
    description || '') })) });
  }
});
```

6.4.7 Frontend Implementation

```
// UX-only guard: the backend enforces the same permission on every API call.
export function usePermission(pageCode, action = 'view') {
  const perms = useAuthStore((s) => s.permissions);
  if (!pageCode) return true;
  if (!perms) return false;
  const actions = perms[pageCode];
  return Array.isArray(actions) && actions.includes(action);
}
```

```
export default function RequirePerm({ page, action = 'view', scope, children }) {
  const has = usePermission(page, action);
  const isSuper = useIsSuperAdmin();
  if (scope === 'super' && !isSuper) {
    return <DeniedPanel reason="This area is reserved for platform administrators." />
    ;
  }
  if (scope === 'tenant' && isSuper) {
    return <DeniedPanel reason="This area is a tenant workspace." />;
  }
  if (!has) return <DeniedPanel reason={`You don't have ${page}:${action} on this
    account.`} />;
  return children;
}

// Usage in App.jsx: every route in the navigation config is wrapped
<Route key={r.path} path={r.path.replace(/^\//, '')} element={
  <RequirePerm page={r.requires} scope={r.scope}>{inner}</RequirePerm>
} />
```

6.4.8 Examination Platform Integration

The examination platform receives identities from the placement platform (SSO and provisioning) and grades code with a server-authoritative judge.

```
export const VERDICT = {
  AC: 'accepted', WA: 'wrong_answer', TLE: 'time_limit_exceeded',
  MLE: 'memory_limit_exceeded', RE: 'runtime_error', CE: 'compilation_error',
  OLE: 'output_limit_exceeded',
  IE: 'internal_error',        // infrastructure failure - never the student's fault
};

export async function judgeSubmission(engineCfg, { language, code, testCases, question
    }) {
  const q = question || {};
  const mult = LANG_MULTIPLIER[language] || { time: 1, mem: 1 };   // fairness for
    interpreted langs
  const baseTimeMs = q.time_limit_ms || 2000;
  const limits = {
    cpu_s:     Math.min((baseTimeMs * mult.time) / 1000, 30),
    memory_kb: Math.min(Math.round((q.memory_limit_kb || 262144) * mult.mem), 512000),
    output_kb: q.output_limit_kb || 4096,
    ...
  };
  const cases = [];
  let sawCompileError = false;
  for (let i = 0; i < testCases.length; i++) {
```

```
    const tc = testCases[i];
    if (sawCompileError) { cases.push({ index: i, verdict: VERDICT.CE, passed: false
    }); continue; }
    const exec = await runOnJudge(engineCfg, language, code, tc.input || '', limits);
    if (exec.verdict === VERDICT.CE) sawCompileError = true;
    let verdict = exec.verdict;
    let passed = false;
    if (verdict === VERDICT.AC) {                    // ran cleanly: now compare output
      passed = compareOutputs(exec.stdout, tc.expected_output || '', q);
      verdict = passed ? VERDICT.AC : VERDICT.WA;
    }
    cases.push({ index: i, verdict, passed, weight: tcWeight(tc), time_ms: exec.
    time_ms });
  }
  ...                                                // weighted score + worst verdict
}
```

6.5 Description of Usage

The test cases, data set and experimental setup described above were used as follows:

- **Test cases:** each phase ended by running the automated smoke and security suite; every documented test case was executed, failures were fixed and re-run, and each fix was pinned by a regression test.
- **Data set:** the seeded reference data bootstrapped every fresh environment; tenant data created through the API and CSV import exercised the placement and attendance workflows; the k6 script generated the load for performance testing; and the assessment problems with hidden cases verified the judge.
- **Experimental setup:** the local environment was used for development and automated testing, and the production deployment for the role-by-role acceptance walkthrough (Section 7.4), using the roles listed in Table 3.1.

Chapter 7

System Testing

This chapter presents the testing approach, functional coverage and observed results for the placement platform, its security controls and the integrated examination platform. Testing included both successful and failed cases; failed cases were either fixed and protected by a regression test, or documented as known limitations.

Testing Strategy

The following kinds of testing were applied:

- **Functional testing:** verifying correct behaviour of each workflow for every role.
- **Negative testing:** invalid inputs, unauthorised access, expired or replayed tokens, cross-tenant identifiers.
- **Security regression testing:** every finding from the internal security audits is reproduced by an automated test that must pass on every run.
- **Isolation testing:** proving tenant isolation and encryption at rest directly at the database level, bypassing the application.
- **Concurrency testing:** simultaneous scans, refresh-token races and queue-slot accounting.
- **Performance and load testing:** login stampedes, bulk imports and a k6 script simulating up to 1,000 students taking an exam.
- **Acceptance testing:** a role-by-role walkthrough on the live production deployment.

7.1 Unit Testing

Unit testing verifies individual functions and middleware in isolation from the rest of the workflow. In this project, each security-critical unit is covered by a focused automated test that exercises exactly that unit through the running service and checks its result, and by the corresponding documented test case. Table 7.1 maps the units to the tests that verify them.

Table 7.1: Units under test and the tests that verify them

Unit	Behaviour verified	Test cases
Password hashing and lockout	Generic error on wrong password; lockout after five failures	TC02, TC03
JWT verification (`requireAuth`)	Missing, tampered or expired tokens rejected	TC04, TC05
Revocation check	Token unusable on every path after logout	TC06
Refresh-token rotation	Reuse and concurrent reuse revoke the family	TC07, TC08
Step-up middleware	Sensitive action needs a fresh, single-use token	TC09, TC10
Permission middleware	Page/action denied without permission	TC11
Rate limiter	MFA brute force throttled	TC12
SSO code exchange	Replayed or cross-platform code rejected	TC13
Tenant scoping (`scopedWhere`) and FK validation	Cross-tenant IDs and foreign keys rejected	TC15–TC17
Row-Level Security policies	Zero rows without tenant context; only own rows with it	TC18, TC19
Field encryption (AES-256-GCM)	Columns stored as `v1:` ciphertext	TC20
UUID validator	Malformed IDs return 400 without stack traces	TC21
SSRF guard	Metadata and private addresses refused	TC22
Eligibility engine	Batch range matched; eligible list correct	TC28, TC38
Offer policy gate	Higher-tier block enforced	TC30
Attendance marking	Exactly one record under concurrency; enrolment checked	TC46, TC47
Judge verdict logic	Accepted, Wrong Answer, Compilation Error, TLE, Runtime Error	TC52–TC56

Justification: the units chosen are the ones whose failure would directly cause a security breach or a wrong placement decision. Testing each of them on its own made it possible to reproduce every weakness found in the internal audits and to prove its fix before the unit was combined with the rest of the workflow.

7.2 Functional Testing

Functional testing verifies that every role can carry out its workflows correctly, including invalid inputs and unauthorised access, as required by FR1–FR10.

7.2.1 Placement System Testing

Authentication and Access Control

Table 7.2: Authentication and access control test cases

TC No.	Test Case	Expected Result	Actual Result	Result
TC01	Valid login with correct role	Tokens issued; redirect to role dashboard	Works correctly	Pass
TC02	Invalid password	Generic "invalid credentials"; failure counted	Works correctly	Pass
TC03	Five consecutive wrong passwords	Account locked temporarily	Locked as expected	Pass
TC04	Access protected API without token	401 `MISSING_TOKEN`	Works correctly	Pass
TC05	Tampered or expired JWT	401 `INVALID_TOKEN` / `TOKEN_EXPIRED`	Works correctly	Pass
TC06	Use access token after logout	401 `TOKEN_REVOKED` on every endpoint	Soft-auth path accepted it initially	Fail (fixed)
TC07	Reuse an already-rotated refresh token	Whole token family revoked; re-login required	Works correctly	Pass
TC08	Two concurrent refreshes with the same token	One succeeds; family revoked on race	Family not revoked initially	Fail (fixed)
TC09	Grant a role without step-up token	403 `STEP_UP_REQUIRED`; OTP modal shown	Works correctly	Pass
TC10	Replay a consumed step-up token	403 `STEP_UP_INVALID`	Works correctly	Pass
TC11	Student opens an admin page / API	403 `PERMISSION_DENIED`; access-denied panel	Works correctly	Pass
TC12	Brute-force MFA verification	429 `RATE_LIMITED` after the limit	Only login limited initially	Fail (fixed)
TC13	SSO code replayed or used for another platform	Exchange rejected	Works correctly	Pass
TC14	Super admin first login	Forced password change and MFA enrolment	Works correctly	Pass

Multi-tenancy and Data Protection

Table 7.3: Multi-tenancy and data protection test cases

TC No.	Test Case	Expected Result	Actual Result	Result
TC15	Tenant A requests a tenant B student by ID	404; no data disclosed	Works correctly	Pass
TC16	Register user with another tenant's slug	Tenant taken from token; request rejected	Client tenant trusted initially	Fail (fixed)
TC17	Create class with advisor from another tenant	Validation error	FK trusted initially	Fail (fixed)
TC18	Direct SQL as `core_app` without tenant context	Zero rows returned	Works correctly	Pass
TC19	Direct SQL with tenant A context	Only tenant A rows	Works correctly	Pass
TC20	Read encrypted columns directly from DB	`v1:` ciphertext only	Works correctly	Pass
TC21	Malformed UUID in path	400 validation error, no stack trace	Works correctly	Pass
TC22	Job-feed URL pointing to `169.254.169.254` or a private IP	Fetch refused by SSRF guard	Works correctly	Pass
TC23	Report download link inspection	No internal file paths or queue IDs exposed	`file://` path exposed initially	Fail (fixed)

Student Portal Functionality

Table 7.4: Student portal functional test cases

TC No.	Test Case	Expected Result	Actual Result	Result
TC24	First login with ID-card number as password	Forced password change	Works correctly	Pass
TC25	Portal is role-scoped	Only own profile and student menu visible	Works correctly	Pass
TC26	Upload resume (PDF)	Stored in R2; status available	Works correctly	Pass
TC27	Apply without acknowledging policy	Policy panel shown; server returns `POLICY_ACK_REQUIRED`	Generic error on combined upload + apply	Fail (fixed)
TC28	Apply to an eligible open drive	Application created; funnel updated	Works correctly	Pass
TC29	Apply twice to the same drive	`ALREADY_APPLIED`	Blocked by unique constraint	Pass

TC No.	Test Case	Expected Result	Actual Result	Result
TC30	Apply above allowed offer tier	`CATEGORY_BLOCKED_BY_HIGHER`	Works correctly	Pass
TC31	Placement policy page as student	Read-only view	Works correctly	Pass
TC32	Generate resume for a target role	Role-relevant skills and top projects first	Works correctly	Pass
TC33	Mobile screen view	Bottom navigation; usable layout	Works correctly	Pass

Admin, Placement Officer, Faculty and Super Admin Modules

Table 7.5: Administration and placement office test cases

TC No.	Test Case	Expected Result	Actual Result	Result
TC34	Super admin creates a college	Tenant, admin, roles and matrix created in one transaction	Works correctly	Pass
TC35	Tenant admin edits role permission matrix	Saved after step-up; audited	Works correctly	Pass
TC36	Create department, class and student	Records created; visible in lists	Works correctly	Pass
TC37	Create company, draft job and open it	Status toggles; eligible students notified	Works correctly	Pass
TC38	Batch filter “2027” vs student batch “2023-2027”	Student eligible	Student excluded initially	Fail (fixed)
TC39	Edit eligibility then view eligible list	List refreshes immediately	Needed page reload initially	Fail (fixed)
TC40	Bulk “Apply students”	Applied count and per-student reasons shown	Showed “Applied 0 of .” (field mismatch)	Fail (fixed)
TC41	Mis-click outside a filled job form	Form not discarded	Form discarded silently	Fail (fixed)
TC42	Create drive category and policy version	Saved; students must re-acknowledge	Works correctly	Pass
TC43	Bulk e-mail with attachment	Cost preview; queued; sent box updated	Works correctly	Pass
TC44	Audit log after mutations	Every write recorded; sensitive fields redacted	Works correctly	Pass

Attendance and Background Processing

Table 7.6: Attendance and background processing test cases

TC No.	Scenario	Expected Behaviour	Actual Result	Result
TC45	Create and start a session; scan ID card	Student marked present	Works correctly	Pass
TC46	Same card scanned twice concurrently	Exactly one record; second reports "already marked"	Duplicate possible initially	Fail (fixed)
TC47	Scan a student from another class	400 `NOT_ENROLLED`	Works correctly	Pass
TC48	CSV import of students	Import job queued; row errors reported	Works correctly	Pass
TC49	Generate report	Job queued → done; CSV downloadable	Works correctly	Pass
TC50	Invalid report request under concurrency cap	No slot consumed	Slot leaked initially	Fail (fixed)
TC51	Undefined route in the frontend	Friendly page; no crash	Works correctly	Pass

Justification: functional test cases were written per role and per workflow so that each functional requirement is exercised at least once from the user's point of view; negative cases (wrong inputs, wrong role, wrong tenant) were included because the platform's value depends on rules being enforced, not only on features being present.

7.3 Integration Testing

Integration testing verifies that the services work correctly together: the frontend, auth service, core service, worker, databases and the examination platform.

7.3.1 Automated Test Suite

The repository contains a runner (`tests/run-all.js`) that executes each test file in its own Node.js process against the running services, in a dependency-friendly order, and exits with the number of failed files. The suite contains a cumulative regression smoke test, feature smoke tests (delete/restore, file upload, step-up, student auto-login, cross-service audit) and twelve security groups.

Table 7.7: Automated security regression groups

Group	Weakness reproduced	Fix verified by the test
Group 1	Registration trusted a client-supplied tenant and had no permission gate.	Tenant derived from the caller's token; `admin.users:create` required.
Group 2	Client-supplied foreign keys (class advisor, student user) were trusted.	All references validated against the caller's tenant; `user_id` rejected in CSV.
Group 3	"Soft" authentication paths skipped the revocation check.	Shared helper checks signature and revocation on every path.
Group 4	Two simultaneous scans could insert duplicate attendance.	Advisory lock on (session, student) for scan and manual paths.
Group 5	Credentials hard-coded in the local compose file.	All secrets from a git-ignored `.env`; git history verified.
Group 6	Rate limiting covered only login.	Per-endpoint limiters on MFA verify, enrol, reset, change password, refresh.
Groups 7–11	Secondary lookups not tenant-scoped; unvalidated UUIDs; internal paths leaked; non-atomic rotation; cross-user notification marking.	Tenant-scoped lookups, UUID validation, no internal paths, one-transaction rotation, self-scoped notifications.
Group A	In-transaction refresh reuse race did not revoke the family.	Family revoked in a committed transaction before the error is returned.
Groups B–E	No throttling on heavy endpoints; worker trusted file paths from the queue; reports exposed `file://` paths; attendance uniqueness only in code.	Per-tenant caps; worker reads paths from DB and checks upload root; tenant-scoped download; DB trigger + unique ledger.
Group R3	Concurrency slot keys differed between producer and worker; slots leaked on invalid requests.	Shared key contract, reserve-after-validation, release on every path, TTL safety net.
RLS smoke	–	With no tenant set, direct SQL as `core_app` returns zero rows; with tenant A set, only tenant A rows.
Encryption smoke	–	API returns plaintext; the same columns read directly from the database are `v1:` ciphertext.

7.3.2 Coding Assessment System Testing (Examination Platform Integration)

Table 7.8: Coding verdict and integrity test cases

TC No.	Test Case	Expected Result	Actual Result	Result
TC52	Correct solution	Accepted; full score	Works correctly	Pass
TC53	Incorrect output	Wrong Answer; partial score by weights	Works correctly	Pass
TC54	Syntax error	Compilation Error; remaining cases skipped	Works correctly	Pass
TC55	Infinite loop	Time Limit Exceeded	Works correctly	Pass

TC No.	Test Case	Expected Result	Actual Result	Result
TC56	Runtime exception	Runtime Error	Works correctly	Pass
TC57	Browser sends forged "all tests passed" results	Ignored; score recomputed on server	Client results trusted in the original code path	Fail (fixed)
TC58	Hidden test cases	Affect verdict; not revealed to student	Works correctly	Pass

7.3.3 Performance and Load Testing

- **Login stampede:** about 100 concurrent logins originally drove the server to 100% CPU for roughly a minute and requests failed. After tuning argon2 to the OWASP minimum (about 3–4 times cheaper per hash) and adding the hashing semaphore, excess logins queue briefly and succeed.
- **Bulk operations:** CSV imports, reports and bulk e-mail run in the worker, so the API remains responsive; per-tenant caps prevent one college from blocking others.
- **Examination load:** a k6 script ramps up to 1,000 virtual students who log in, open an exam and save answers with think time, measuring save success rate and latency. It is run off-hours against the staging environment.
- **AI service:** the number of model workers adapts to CPU load (target 70% of cores) so that resume generation never starves the examination judge.

Justification: most defects found in the project appeared only when components interacted – a soft-authentication path that skipped revocation, a refresh race across two requests, slot keys that differed between the API and the worker, and judge results trusted from the browser. Running the suite against the real, running services is therefore the most reliable way to catch regressions at service boundaries.

7.4 Acceptance Testing

7.4.1 User Acceptance Testing

User acceptance was carried out as a role-by-role walkthrough of the complete data-entry and placement flow on the live production deployment (department → class → student → company → drive → eligibility → application), followed by the student journey.

Table 7.9: Role-based acceptance walkthrough on the production deployment

Role	Workflows verified	Outcome
Tenant admin	Dashboard, users and roles, RBAC matrix, drive categories, student options, engagement, bulk e-mail, placement policy (published), audit and security, settings and 2FA	Accepted
Placement officer	Companies with HR contacts, drive create/open/edit, eligibility, funnel, rounds, preparation resources, off-campus board, reports (worker job queued → done), accreditation report, announcements	Accepted after fixes TC38–TC41
Teacher	Attendance session create → start → barcode scan marks student present	Accepted
Student	SSO hub login, forced password change, role-scoped portal, resume upload, policy acknowledgement, self-apply, read-only policy, notifications	Accepted after fix TC27
Security controls	Step-up e-mail OTP on role and permission changes, confirmation dialogs, audit trail on all writes, redaction of sensitive fields	Accepted

Table 7.10: User feedback summary (ratings out of 5)

Feature	Rating (5)	Feedback
Placement portal usability	–	
Application status and journey tracking	–	
Attendance scanning	–	
Resume builder	–	
Coding assessment verdict clarity	–	
Mobile experience	–	

Overall, the system passed the critical test cases. All defects found during testing were fixed and are now covered by automated regression tests or verified again in the walkthrough.

Justification: acceptance testing was carried out on the live production deployment, by role, along the complete data-entry and placement flow, because the purpose of the project is that real users in a real college can complete their work; it also uncovered usability defects (TC27, TC38–TC41) that automated tests could not detect.

7.5 Defect Summary

Table 7.11: Summary of key failed test cases and their resolution

TC No.	Issue Observed	Status	Action Taken
TC06	Revoked token accepted by a soft-auth path	Fixed	Shared soft-auth helper checks revocation

TC No.	Issue Observed	Status	Action Taken
TC08	Refresh reuse race did not revoke family	Fixed	Family revoked in a separate committed transaction
TC12	Only login was rate-limited	Fixed	Per-endpoint limiters and per-user failure counters
TC16, TC17	Client-supplied tenant / foreign keys trusted	Fixed	Tenant from token; FKs validated in tenant
TC23	Internal file paths in report responses	Fixed	Tenant-scoped download endpoint
TC27	Generic error instead of policy panel	Fixed	Ack panel opens and apply auto-continues
TC38	Batch range not matched	Fixed	Exact, range-suffix or graduation-year match
TC39	Eligible list stale after edit	Fixed	Re-fetch on job update (refresh key)
TC40	Bulk apply showed “Applied 0 of .”	Fixed	Response and UI fields aligned; reasons listed
TC41	Backdrop click discarded filled form	Fixed	Non-dismissible job forms
TC46	Duplicate attendance under concurrency	Fixed	Advisory lock + DB trigger + unique ledger
TC50	Concurrency slot leaked	Fixed	Reserve late, release on every path, TTL
TC57	Client-reported judge results trusted	Fixed	Server-authoritative judge (Judge0/isolate)

Chapter 8

Results and Discussions

The platform was implemented, deployed to production and validated through automated security tests and a production walkthrough. The results show that the system integrates tenant management, identity, attendance, the complete placement workflow and assessments into one secure platform.

8.1 Functional Achievements

Table 8.1: Summary of achieved functional objectives

Objective	**Achieved?**
Multi-tenant provisioning without code changes	Yes
Subscription tiers, add-ons and trial life cycle	Yes
Dynamic RBAC with per-page, per-action permissions	Yes
Hardened authentication (argon2id, RS256, rotation, MFA, passkeys, step-up)	Yes
Two-layer tenant isolation (application + PostgreSQL RLS)	Yes
Encryption at rest of sensitive fields and audit logging	Yes
Automated eligibility verification and offer-tier policy	Yes
Drive rounds, shortlists, selections and offers	Yes
Barcode-based attendance with exactly-once guarantee	Yes
Placement analytics, trends and accreditation reports	Yes
Bulk e-mail, announcements, notifications and web-push	Yes
Role-targeted resume builder with self-hosted AI rewriting	Yes
Secure coding assessment via integrated examination platform	Yes
Central single sign-on across platforms	Yes
Production deployment with CI/CD and backups	Yes
AI-based project ranking with a larger model	Partially (disabled; rule-based ranking used)
Native mobile applications	No (responsive web + web-push)

8.2 Performance Improvements

- **Eligibility shortlisting:** manual spreadsheet filtering is replaced by one tenant-scoped database query, and all eligible students are notified automatically when a drive opens.
- **Resilience:** because tokens are verified locally, an auth-service outage no longer takes down day-to-day work; only new logins are affected.
- **Login capacity:** argon2 tuning made each hash about 3–4 times cheaper, and the semaphore converted a CPU-exhaustion failure under about 100 concurrent logins into short queueing.
- **Responsiveness:** imports, reports and bulk e-mail moved off the request path to the worker.
- **Operational effort:** adding a college is a single form; deployments are one push with automatic migrations and health checks.

8.3 Comparison with the Mini-project Prototype

Table 8.2: Measured scope of the prototype versus the major project

Metric	Mini project	Major project
Colleges supported per deployment	1	Many (multi-tenant)
Pages in the permission catalogue	–	about 65
Default roles per college	5 fixed	9 editable + custom
Tables protected by RLS	0	18 (plus later tables)
Background job queues	0	7
Authentication factors	Password	Password, TOTP, passkey, e-mail OTP
Automated security test groups	0	12 + isolation and encryption smoke tests
Deployment	Docker Compose (local)	Cloudflare + Traefik + CI/CD

8.4 Graphical Analysis of Test Results

Figure 8.1 shows the outcome of the 58 documented test cases by group (Table 6.1). Overall, 44 cases (75.9%) passed when first executed and 14 (24.1%) failed and were fixed and re-verified; after the fixes all 58 cases pass. Most failures were in the administration and placement-office group (4 of 11) and in the authentication and multi-tenancy groups (3 each), which is where the internal security audits concentrated and where the most complex rules live.

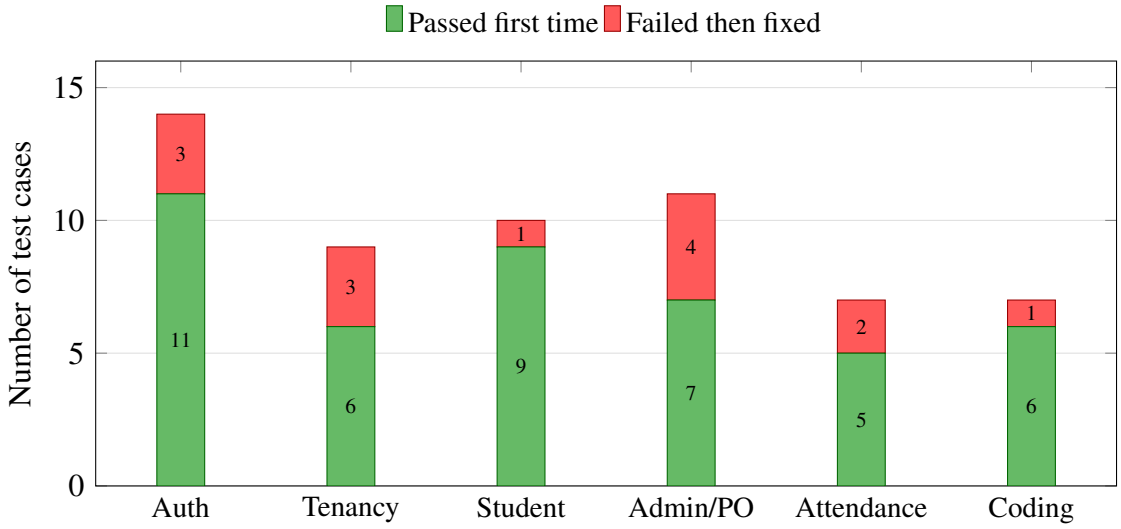


Figure 8.1: Test case outcomes by group

8.5 Quantitative Comparison with the Prototype

Figure 8.2 plots the numeric metrics of Table 8.2. The major project adds editable roles, database-enforced isolation on 18 tables, seven background queues, four authentication factors and twelve automated security test groups, none of which existed in the prototype.

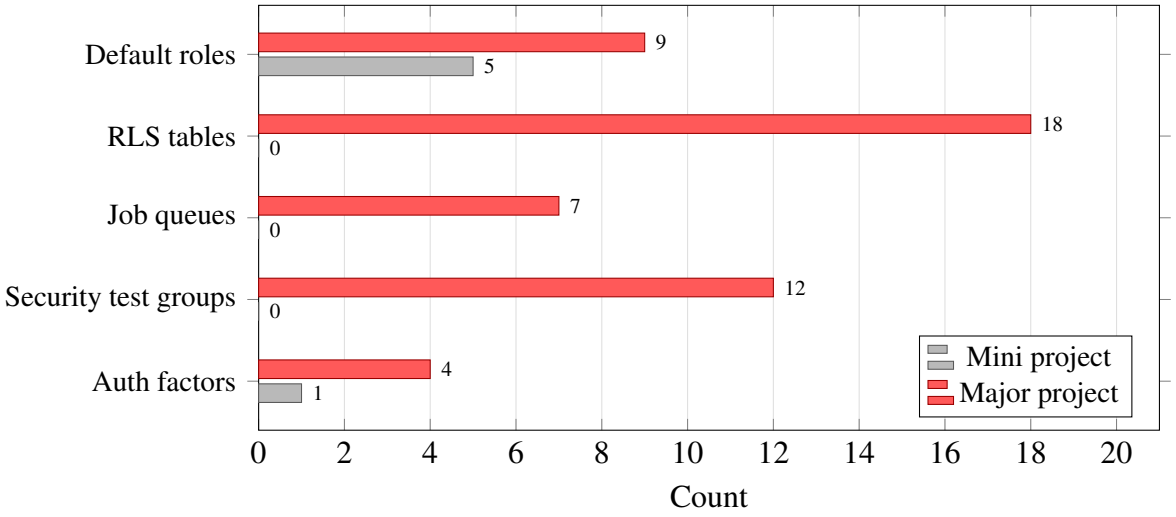


Figure 8.2: Prototype versus major project on measurable metrics

8.6 Code Base Composition

Figure 8.3 shows the size of each component of the code base (Table 6.2). The frontend is the largest component (27,925 lines) because of its role-specific dashboards, while the automated smoke and security tests account for 4,232 lines, about 6.8% of the 61,962 lines.

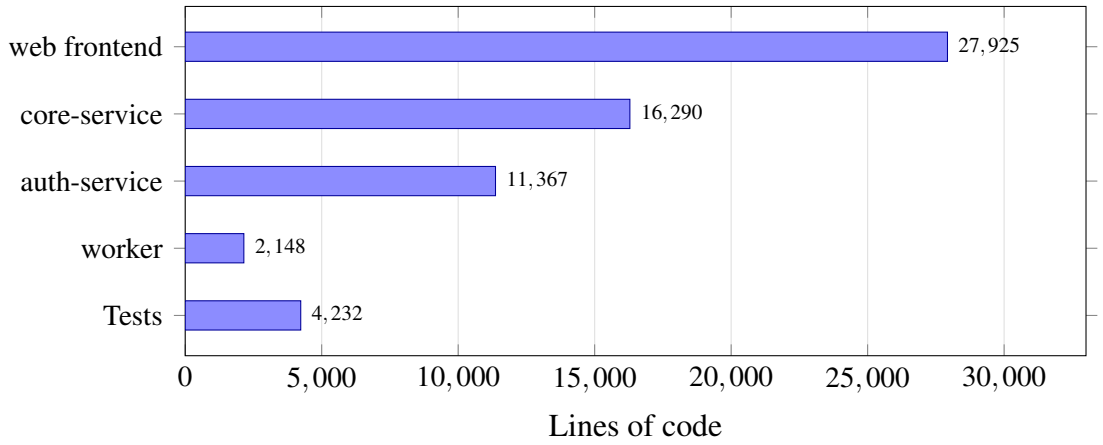


Figure 8.3: Lines of code per component

8.7 Justification of Requirements

Tables 8.3 and 8.4 justify each functional and non-functional requirement of Chapter 3 with the evidence obtained in testing.

Table 8.3: Justification of functional requirements

FR	Requirement	Evidence of fulfilment
FR1	Super admin manages tenants, plans, add-ons and trials	A college with its admin, roles and permission matrix is created in one transaction (TC34).
FR2	Tenant admin manages users, custom roles and page permissions	Permission matrix saved after step-up and audited (TC35); students blocked from admin pages (TC11).
FR3	Login with password, TOTP or passkey; step-up for sensitive actions	Login, lockout and token checks (TC01–TC05), step-up enforced (TC09, TC10), forced MFA for super admin (TC14).
FR4	Departments, classes and students, individually or by CSV	Records created (TC36); CSV import queued with row errors reported (TC48).
FR5	Attendance by barcode or manually, exactly once	Scan marks present (TC45); exactly one record under concurrency (TC46); enrolment checked (TC47).
FR6	Companies, drives, eligibility, rounds, offers, categories, policy	Drive opened and eligible students notified (TC37); eligibility and batch rules correct (TC38, TC39); categories and policy versions (TC42).
FR7	Students view eligible drives, acknowledge policy, upload resume, apply, track	Resume upload (TC26), policy gate (TC27), application created (TC28), duplicate and tier blocks (TC29, TC30).
FR8	Notifications, announcements, web-push and bulk e-mail	Eligible students notified on drive open (TC37); bulk e-mail with cost preview queued and sent (TC43).

FR	Requirement	Evidence of fulfilment
FR9	Analytics, exports and accreditation reports generated asynchronously	Report job queued, completed and downloadable (TC49); accreditation report verified in acceptance testing.
FR10	Role-targeted resumes and SSO to the examination platform	Role-relevant resume generated (TC32); SSO code replay rejected (TC13); one login across platforms in acceptance testing.

Table 8.4: Justification of non-functional requirements

Requirement	Evidence of fulfilment
Security	Twelve automated security groups pass; cross-tenant access refused (TC15–TC17); RLS returns zero rows without a tenant (TC18); fields stored encrypted (TC20); every write audited with redaction (TC44).
Reliability	Local token verification keeps existing sessions working when the auth service is down; background jobs retry; hourly backups to R2.
Performance	Login stampede of about 100 concurrent logins now queues briefly instead of exhausting the CPU; eligibility is one database query; heavy work runs in the worker.
Scalability	Stateless services, PgBouncer pooling, monthly partitions and per-tenant concurrency caps; exam load tested with up to 1,000 virtual students.
Maintainability	Modular monolith with module boundaries, versioned migrations, idempotent seeders, structured logs and CI/CD.
Usability	Mobile bottom navigation (TC33), friendly error pages (TC51) and role-specific dashboards accepted in the walkthrough.
Portability	All configuration from environment variables; the same images run locally and in production.

8.8 Comparison with Existing Systems

Table 8.5 compares FortifyHub with the categories of existing systems reviewed in Chapter 2. A tick means the capability is provided, “partial” means it is provided only in a limited form, and a dash means it is not provided.

Table 8.5: Feature comparison with existing systems

Capability	Coding judges	TPO portals	LMS	Recruit. platforms	Surveyed papers	Fortify Hub
Job postings and drive workflow	–	✓	–	✓	partial	✓
Rule-based eligibility and offer policy	–	partial	–	–	–	✓

Table 8.5 (continued)

Capability	Coding judges	TPO portals	LMS	Recruit. platforms	Surveyed papers	Fortify Hub
Secure, server-side code evaluation	✓	–	–	–	–	✓
Cohort analytics and reports	–	partial	partial	partial	partial	✓
Attendance	–	–	partial	–	–	✓
Fine-grained RBAC and audit trail	–	partial	partial	partial	–	✓
Multi-college tenancy	–	partial	–	–	–	✓
MFA, passkeys and encryption at rest	–	–	–	–	–	✓

Existing systems each cover a part of the problem; FortifyHub combines all of these capabilities in one platform and adds the security controls that none of the surveyed systems describe.

Key Differentiators

The comparison highlights five areas in which FortifyHub goes beyond the reviewed systems:

- **True multi-tenancy:** a single deployment serves many colleges, and isolation is enforced twice – by application-level scoping and by forced PostgreSQL Row-Level Security – so a defect in one query cannot expose another college's data.
- **One platform for placement, attendance and assessment:** placement drives, barcode-based attendance and the coding examination platform share one identity through central single sign-on, instead of being separate tools with separate accounts.
- **Security by design:** argon2id password hashing, RS256 tokens with refresh rotation and theft detection, TOTP, passkeys, step-up verification, AES-256-GCM field encryption and an append-only audit log are built in rather than added later.
- **Configuration instead of code changes:** subscription plans, plan-gated pages, custom roles and the per-page permission matrix are managed from the admin console, so a new college or a new role needs no developer involvement.
- **Placement-specific automation:** rule-based eligibility, tiered offer policies, multi-round shortlists and the self-hosted resume assistant reduce the manual effort that generic LMS and recruitment portals leave to the placement office.

8.9 Visual Results of UI Implementation

8.9.1 Authentication and Single Sign-On

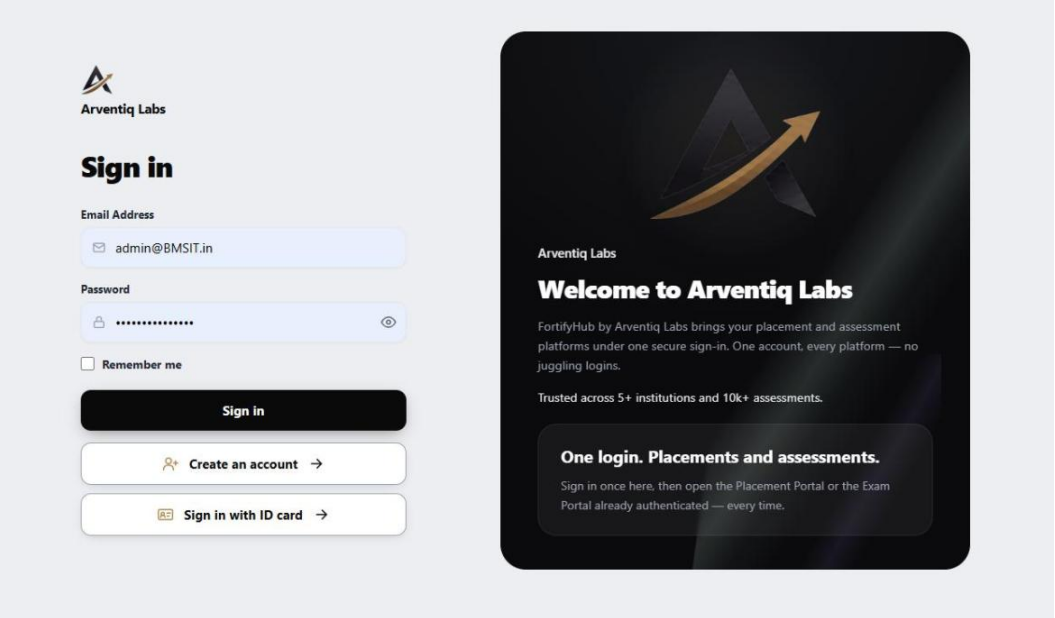


Figure 8.4: Login page with email/password and ID-card sign-in options

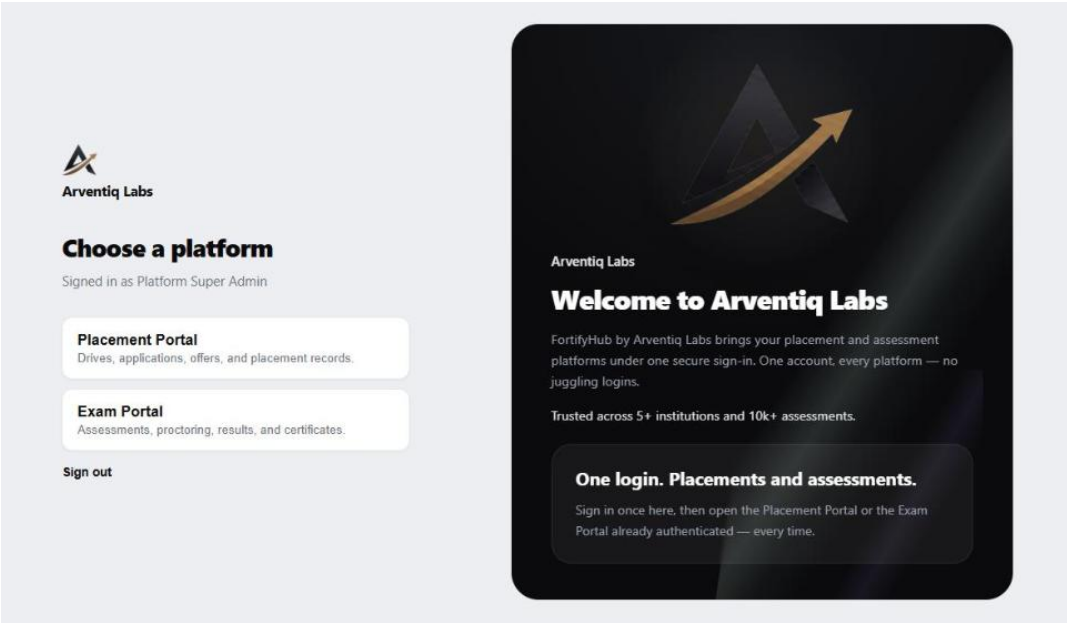


Figure 8.5: Central SSO hub with platform picker

8.9.2 Platform and Tenant Administration

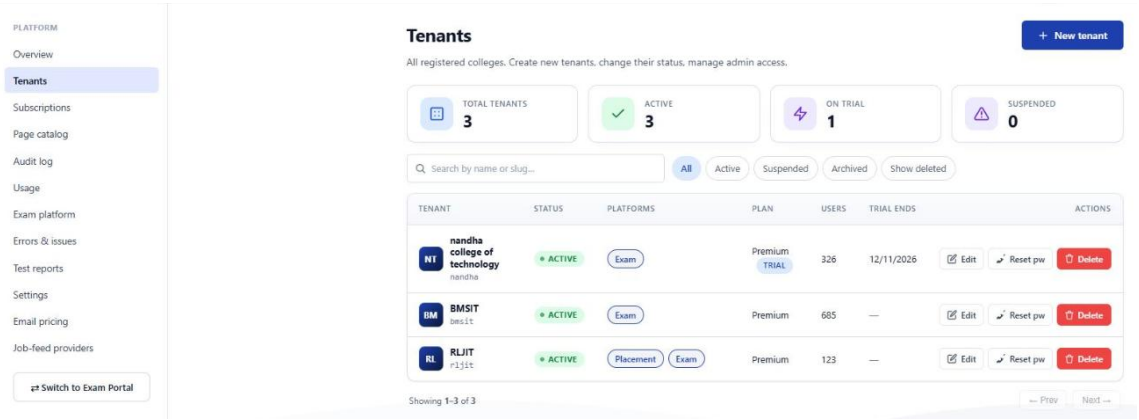


Figure 8.6: Super admin – colleges (tenants) and subscriptions

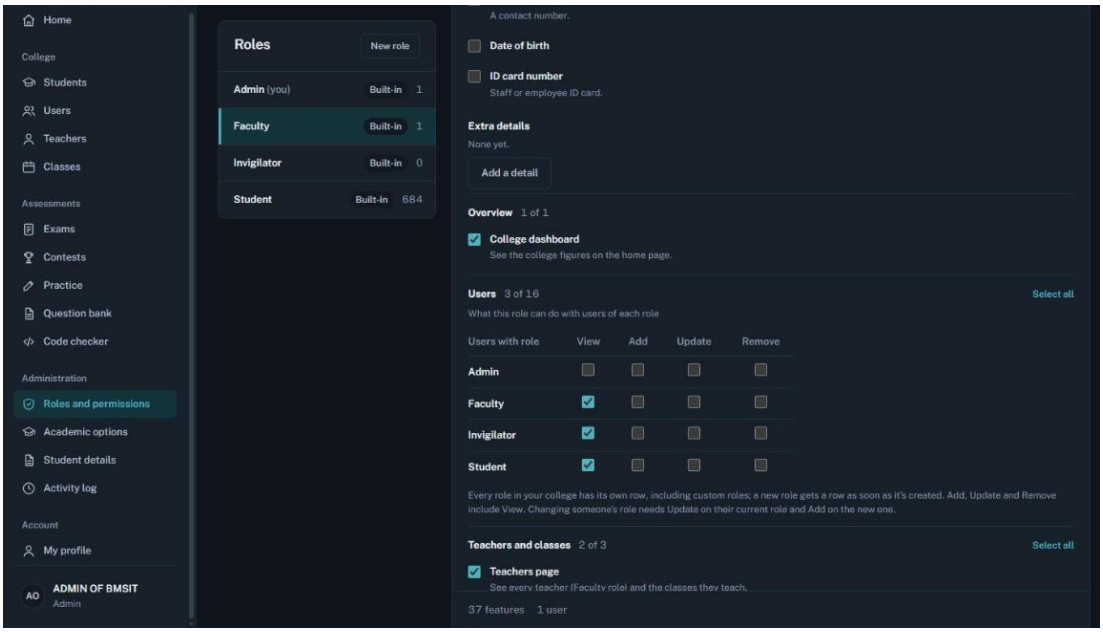


Figure 8.7: Role permission matrix showing per-role View, Add, Update and Remove permissions

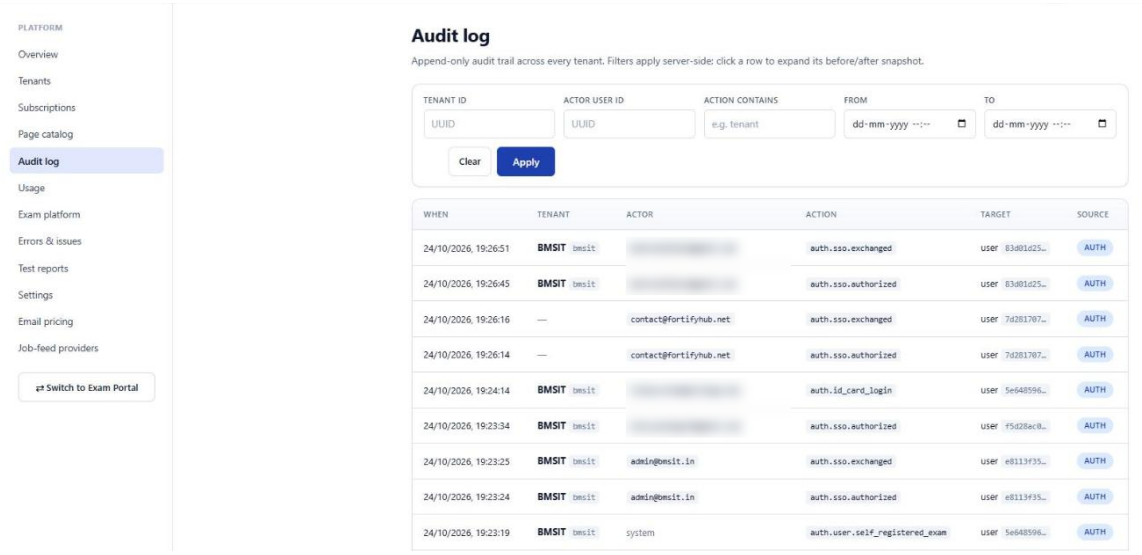


Figure 8.8: Append-only audit log

8.9.3 Platform Console and Feature Management

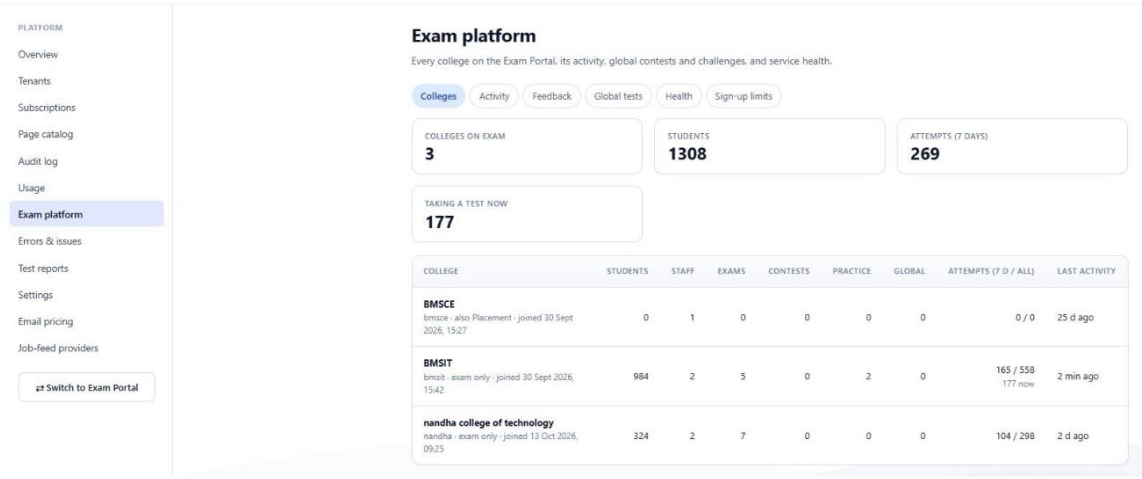


Figure 8.9: Exam platform console: colleges, students and live attempts

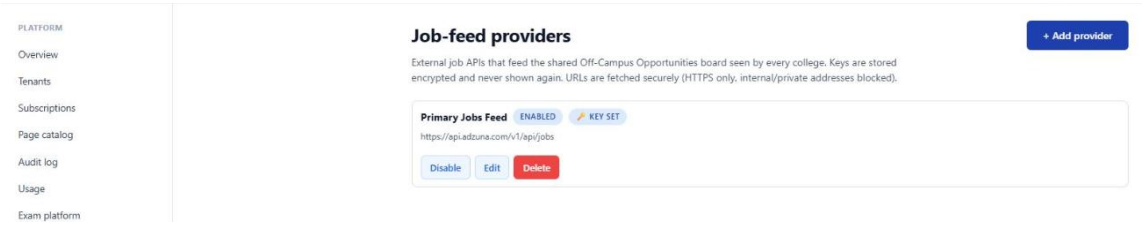


Figure 8.10: Job-feed providers powering the shared off-campus opportunities board

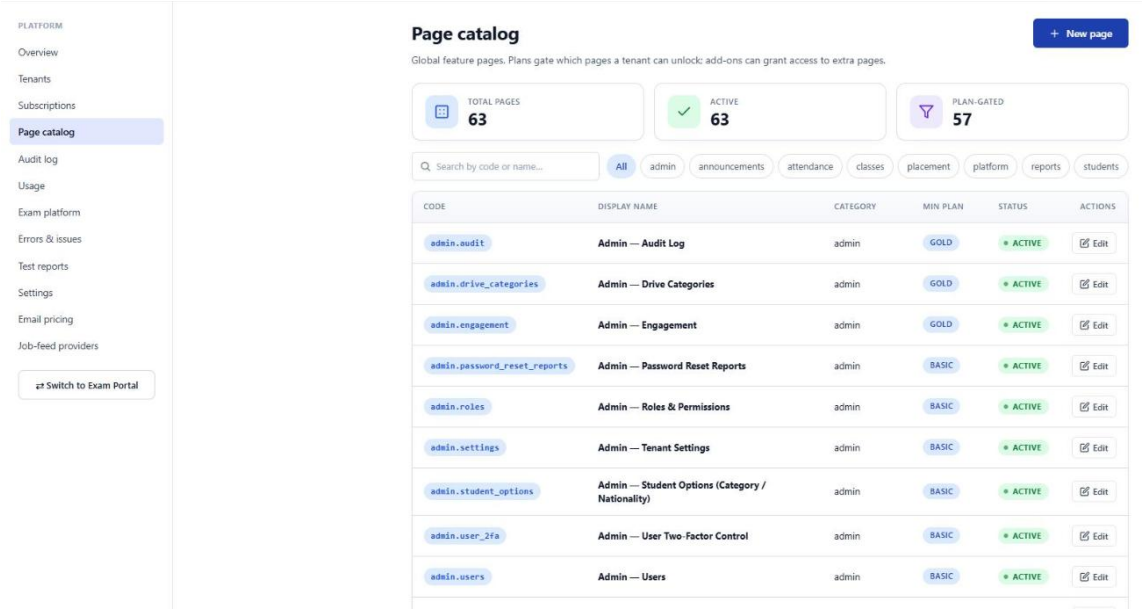


Figure 8.11: Page catalog of plan-gated feature pages

8.9.4 Subscriptions and Test Reports

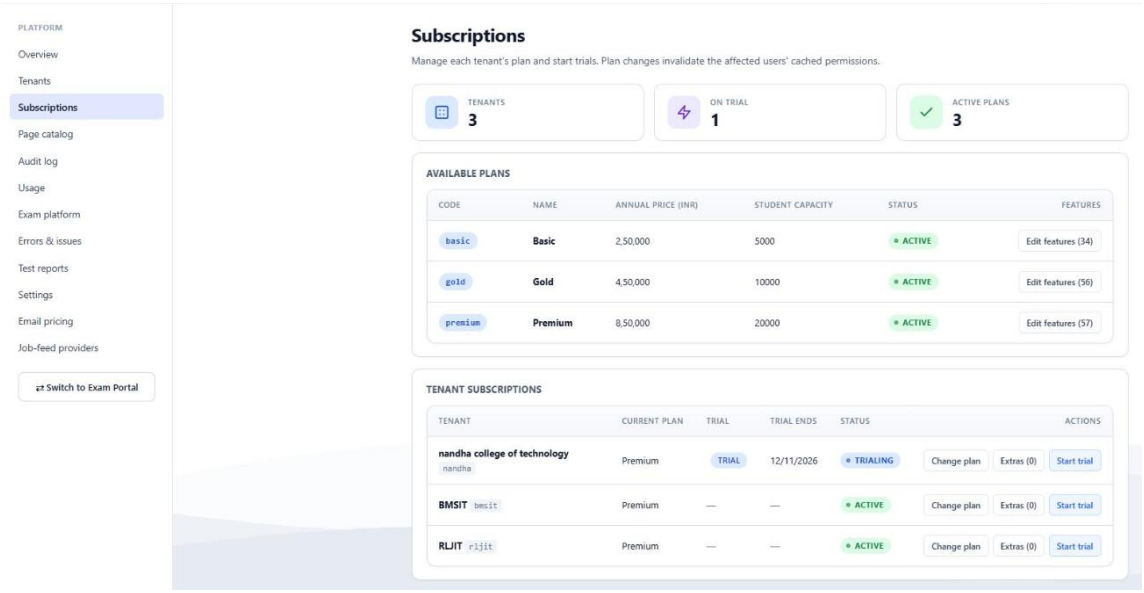


Figure 8.12: Subscription plans and tenant subscriptions

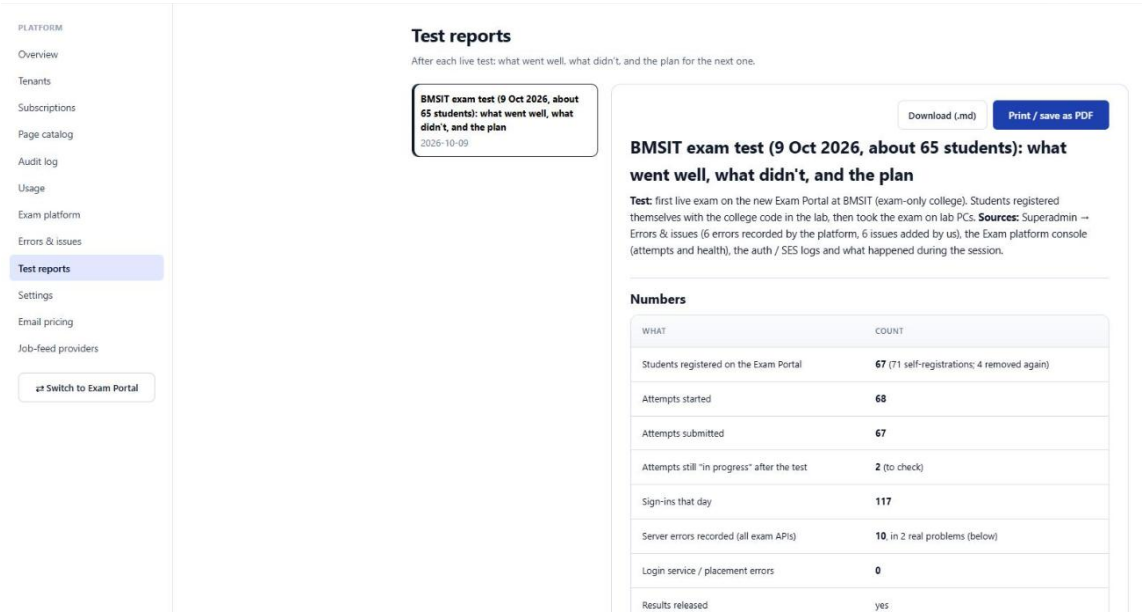


Figure 8.13: Post-exam test report for the first live BMSIT exam

8.9.5 Student Exam Experience

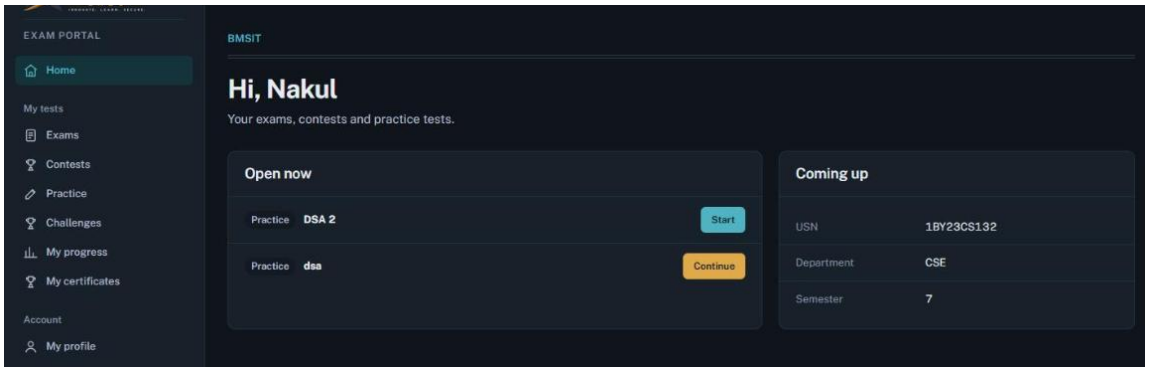


Figure 8.14: Student home showing open practice tests and profile details

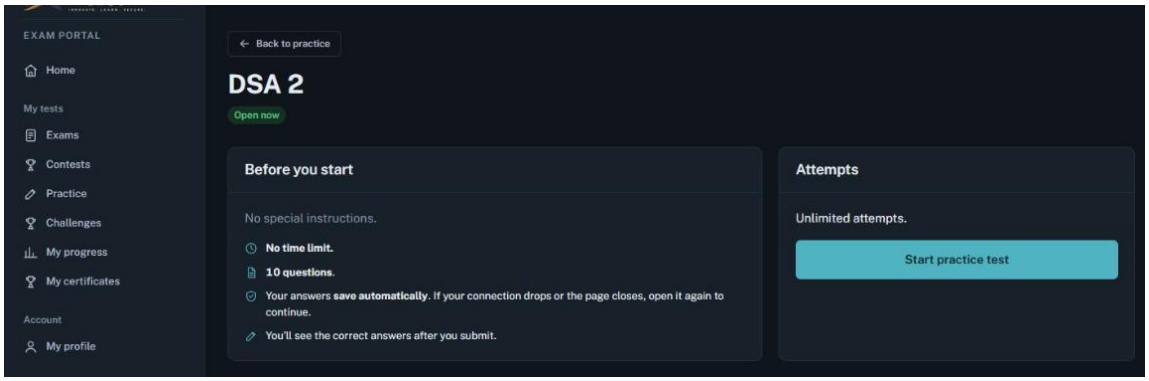


Figure 8.15: Practice test introduction with question count, autosave and unlimited attempts

8.9.6 Attendance

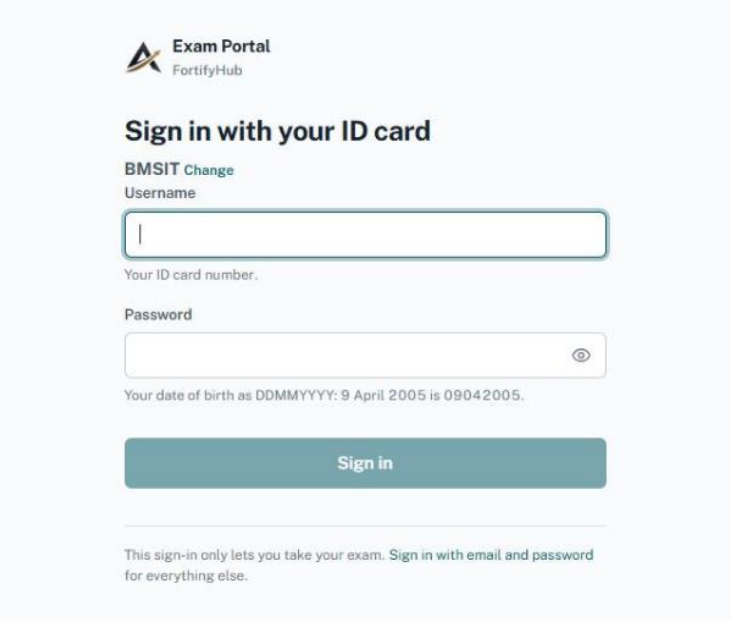


Figure 8.16: ID-card based sign-in used for exam-day attendance

8.10 Visual Results for Coding Assessment Module

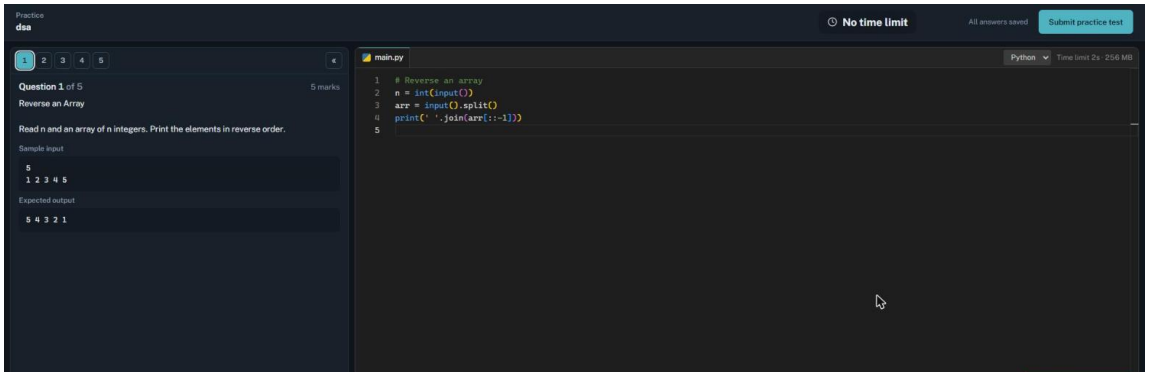


Figure 8.17: Examination platform coding interface

8.10.1 Exam Creation and Question Bank

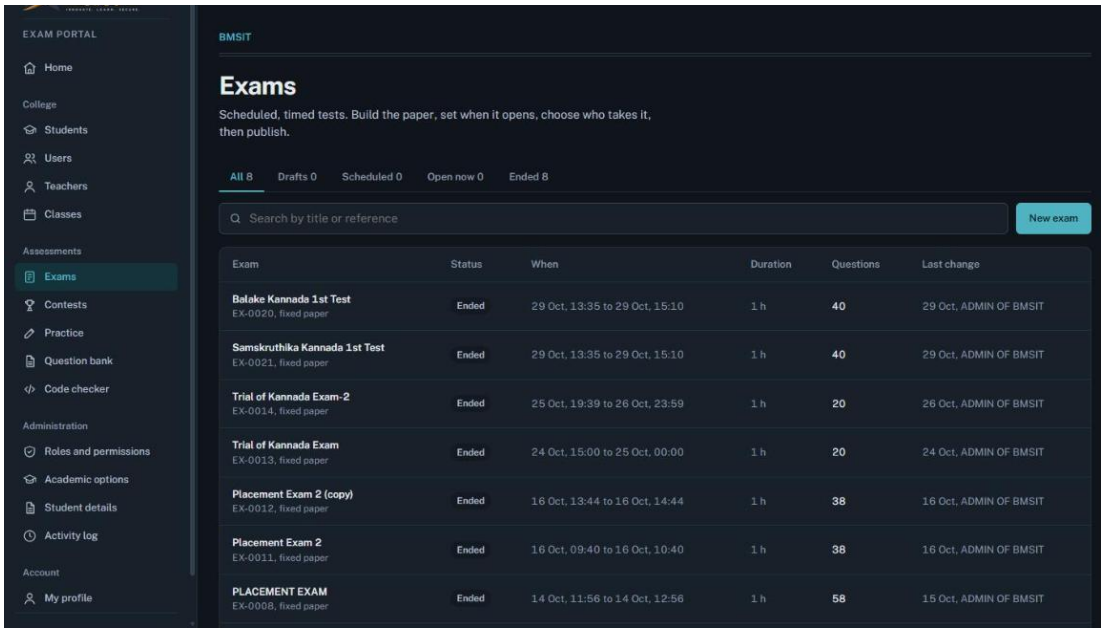


Figure 8.18: Exam management with status, schedule, duration and question count

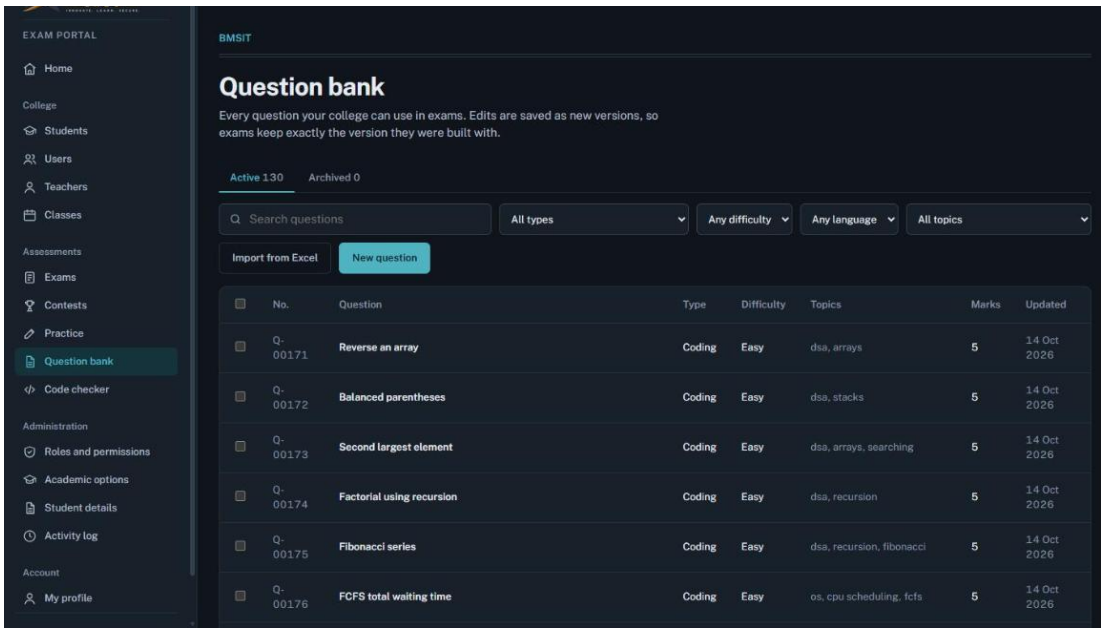


Figure 8.19: Question bank with type, difficulty, topic and marks filters

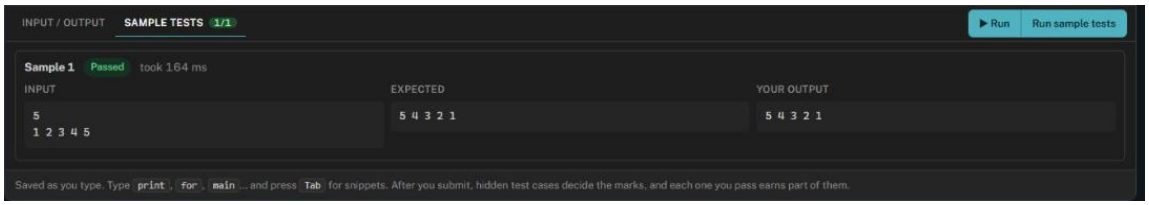


Figure 8.20: Server-side execution result for a sample test case

8.10.2 Deployment

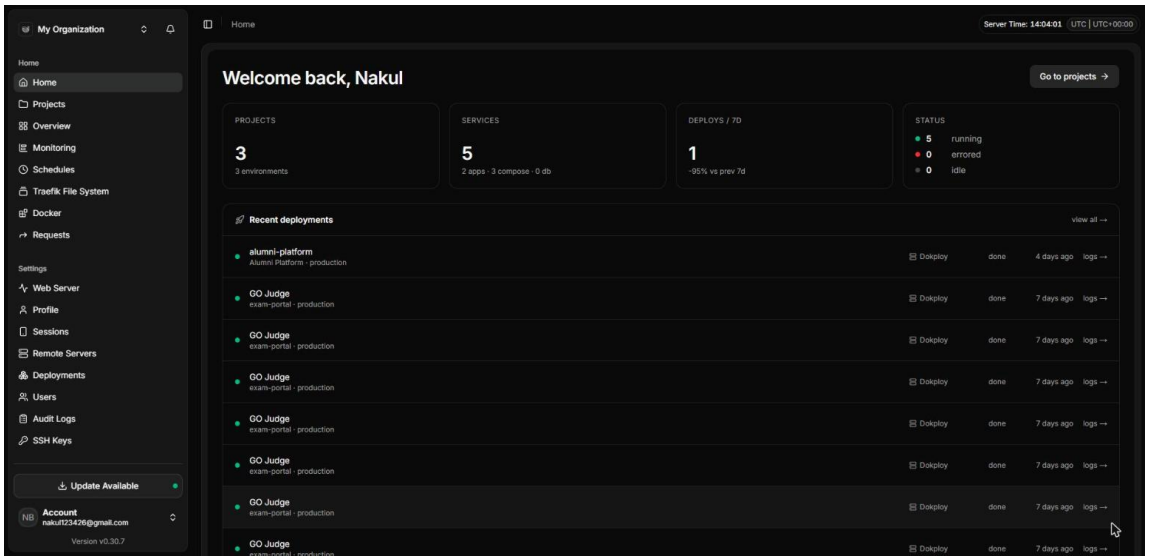


Figure 8.21: Production services running under Dokploy

8.10.3 Server Monitoring and Deployment History

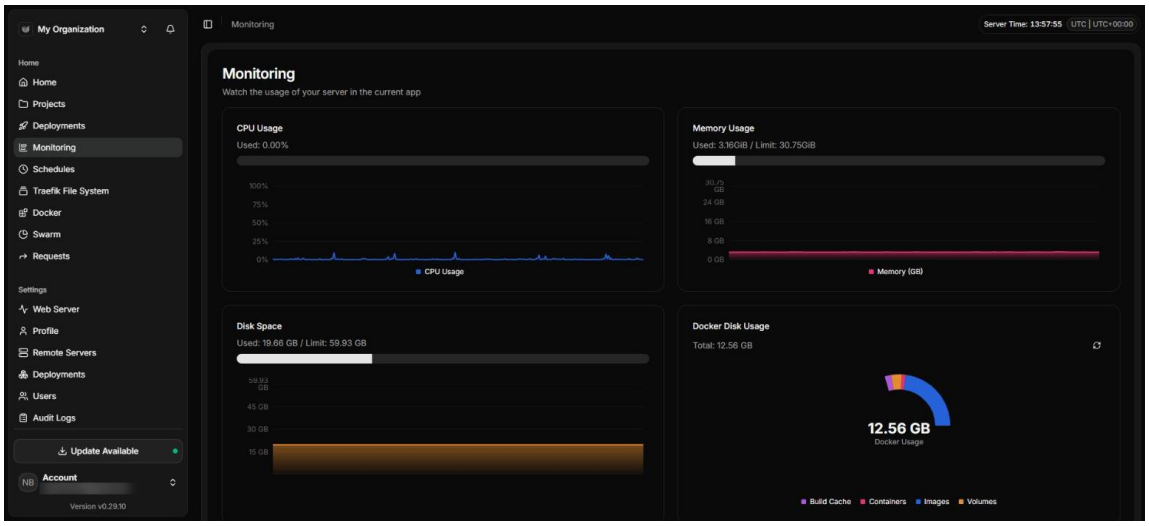


Figure 8.22: Dokploy server monitoring of CPU, memory, disk and Docker usage

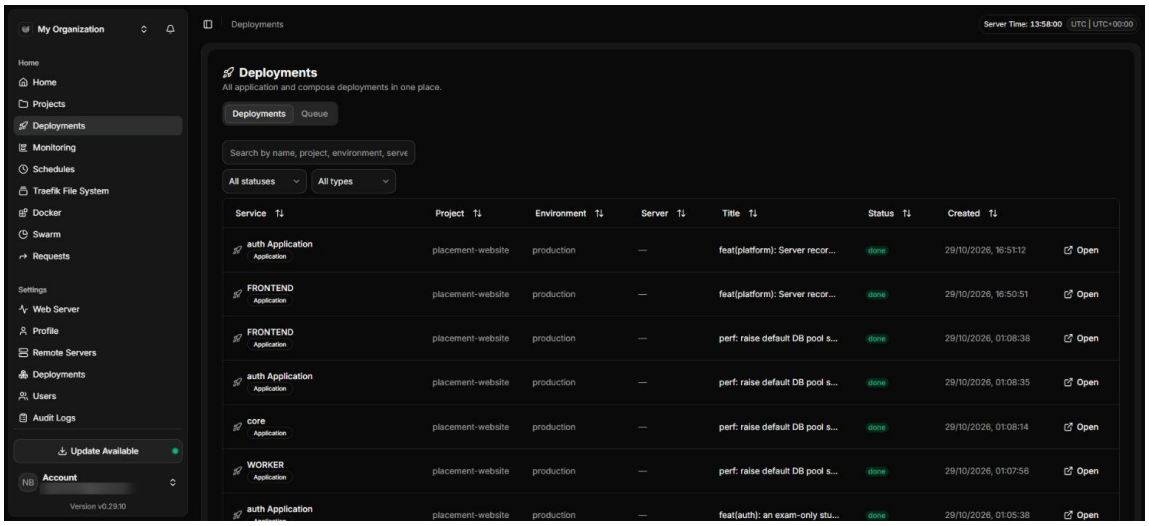


Figure 8.23: Dokploy deployment history of the production services

8.11 Discussion

The results indicate that the platform meets its central objective: a single deployment can safely serve several colleges while automating the placement workflow inside each. Automated eligibility filtering, notification of eligible students and policy-aware applications remove most of the manual checking done by placement officers, and students gain one place to see opportunities, rules, status, selections and their own journey.

The security architecture proved its value during development. Several weaknesses were discovered by internal audits – trusted client identifiers, a soft-authentication path that skipped revocation, a refresh-token race and duplicate attendance under concurrency. Each was fixed at the right layer and pinned by an automated test. Database-level RLS provides a second line of defence: even a query that forgets the tenant filter cannot return another college's data, which was proven by querying the database directly.

The integration with the examination platform shifts assessment toward skills while keeping one identity. Because scores are recomputed on the server in an isolated sandbox, a student cannot forge results by editing requests. The self-hosted language model improves resume wording without sending personal data to any third-party service, showing that useful AI features are possible on a small budget with strict privacy boundaries.

Limitations Identified

- The production deployment runs on a single server; high availability (replicated database, multiple application nodes) is not yet configured.
- Some deployment-platform environment variables proved unreliable, so certain non-secret settings are baked into the build configuration.
- The larger AI model for project ranking is disabled for resource reasons; ranking uses the deterministic heuristic.
- Native mobile apps, calendar sync, Google sign-in and automatic skill extraction from uploaded resumes are pending.
- Formal user-satisfaction measurement with a larger group of students and staff is still to be carried out.

Conclusion

The outcomes of this major project demonstrate that an integrated, multi-tenant placement and academic platform can reduce manual effort, accelerate communication, protect student data and improve transparency. The successful deployment of hardened authentication, two-layer tenant isolation, automated eligibility and policies, exactly-once attendance and server-side assessment confirms that the design is technically sound and ready to scale to more institutions.

Conclusion and Future Enhancements

Conclusion

FortifyHub turns the mini-project prototype into a secure, multi-tenant SaaS platform that unifies placement, attendance, student profiles, communication, analytics and assessments for many colleges from one deployment. By automating eligibility checks, offer policies, notifications and reports, the platform significantly reduces administrative workload and improves coordination between students, faculty, placement officers and management.

Security is built into every layer: argon2id password storage, RS256 tokens verified locally, rotating refresh tokens with theft detection, multi-factor and step-up authentication, per-page RBAC, application- and database-level tenant isolation, AES-256-GCM encryption at rest, SSRF protection and a complete audit trail. The modular architecture, background worker, connection pooling and CI/CD pipeline make the system practical to operate on modest hardware and ready to migrate to larger infrastructure through configuration alone.

Future Enhancements

- **AI-driven matching and learning paths:** match students with suitable roles and suggest up-skilling paths using profile, attendance and assessment data.
- **Resume parsing:** extract skills, projects and internships automatically from uploaded PDF/DOCX resumes.
- **Calendar integration:** subscribable calendar feeds for drives, rounds and exams.
- **Mobile applications and PWA:** installable app with offline support and native push notifications.
- **High availability:** replicated PostgreSQL, multiple application nodes and Kubernetes-based autoscaling.
- **Institutional SSO:** Google/Microsoft sign-in restricted to college domains.
- **Recruiter portal:** direct recruiter accounts for posting drives and shortlisting under tenant control.
- **Compliance features:** configurable data-retention policies, data-export requests and consent management aligned with the Digital Personal Data Protection Act, 2023.

References

[1] M. Mahendru, S. Nigam, R. Bajpai, and Z. Ahmad, "Campus Ease: Elevating Student Placements and Career Success," in *Proc. IEEE Int. Conf. on Electrical, Electronics and Computing Technologies (ICEECT)*, 2024.

[2] S. K. Thangavel, A. Karthikeyan, and P. Rajesh, "Student Placement Analyzer: A Recommendation System Using Machine Learning," in *Proc. IEEE Int. Conf. on Advanced Computing and Communication Systems (ICACCS)*, 2017.

[3] V. Hegde, A. Kamble, and S. Patil, "Predicting Student Placement using PCA and Machine Learning Technique," in *Proc. IEEE Int. Conf. on Computing, Communication and Networking Technologies (ICCCNT)*, 2023.

[4] T. Maragatham and K. Lakshmi, "Student Placement Prediction using Deep Learning Techniques," in *Proc. IEEE Int. Conf. on Electronics, Communication and Aerospace Technology (ICECA)*, 2024.

[5] R. Kadu, P. Sharma, and S. Patil, "Student Placement Prediction and Skill Recommendation System using Machine Learning Algorithms," in *Proc. IEEE Int. Conf. on Information and Communication Technology (ICICT)*, 2024.

[6] N. Kathirisetty, R. Kumar, and S. Gupta, "Student Placement Probabilistic Assessment Using Emotional Quotient with Machine Learning: A Conceptual Case Study," *IEEE Access*, 2023.

[7] T. Ajay, P. Kumar, and S. Rao, "Placement Management System using Machine Learning," *Int. J. of Research and Analytical Reviews (IJRAR)*, 2023.

[8] A. Kumar and R. Singh, "Placement Management System for Campus Recruitment," *Int. J. of Innovative Science and Research Technology (IJISRT)*, 2020.

[9] M. Sinthuja, "Integrated Campus Recruitment Platform (PlaceMe)," *Int. J. for Research in Applied Science and Engineering Technology (IJRASET)*, 2024.

[10] A. Pathak, S. Verma, and K. Joshi, "Smart Campus Placement System using Graph Neural Networks," *TIJER – Int. Research Journal*, 2024.

[11] A. D. Lacasandile, G. V. Tiu, E. M. L. Valencia, R. F. Marcoso, and A. B. R. Canivel, "Revolutionizing Higher Education Internship: A Web-Based Platform to Improve the Relationship Between Internship Seekers, Employers and Educational Institutions," in *Proc. IEEE HNICEM*, 2023.

[12] B. Bhuvaneswaran, R. Reshma, and V. Soniya, "JobQuench: An Intelligent and Automated Placement Management System for Enhanced Campus Recruitment," in *Proc. IEEE Int. Conf. on Augmented Intelligence and Sustainable Systems (ICAISS)*, 2025.

[13] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.

[14] D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," RFC 6749, IETF, Oct. 2012.

[15] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, "Best Current Practice for OAuth 2.0 Security," RFC 9700, IETF, Jan. 2025.

[16] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications," RFC 9106, IETF, Sep. 2021.

[17] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-Based One-Time Password Algorithm," RFC 6238, IETF, May 2011.

[18] W3C, "Web Authentication: An API for Accessing Public Key Credentials – Level 2," W3C Recommendation, Apr. 2021. [Online]. Available: https://www.w3.org/TR/webauthn-2/

[19] P. A. Grassi *et al.*, "Digital Identity Guidelines: Authentication and Lifecycle Management," NIST Special Publication 800-63B, Jun. 2017.

[20] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.

[21] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[22] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST Standard for Role-Based Access Control," *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 224–274, 2001.

[23] C. J. Guo, W. Sun, Y. Huang, Z. H. Wang, and B. Gao, "A Framework for Native Multi-Tenancy Application Development and Management," in *Proc. 9th IEEE Int. Conf. on E-Commerce Technology and 4th IEEE Int. Conf. on Enterprise Computing, E-Commerce and E-Services (CEC/EEE)*, 2007, pp. 551–558.

[24] F. Chong, G. Carraro, and R. Wolter, "Multi-Tenant Data Architecture," Microsoft Corporation, Jun. 2006.

[25] PostgreSQL Global Development Group, "Row Security Policies," PostgreSQL 16 Documentation. [Online]. Available: https://www.postgresql.org/docs/16/ddl-rowsecurity.html

[26] OWASP Foundation, "OWASP API Security Top 10 – 2023." [Online]. Available: https://owasp.org/API-Security/

[27] OWASP Foundation, "Server-Side Request Forgery Prevention Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/

[28] H. Z. Došilović and I. Mekterović, "Robust and Scalable Online Code Execution System," in *Proc. 43rd Int. Convention on Information, Communication and Electronic Technology (MIPRO)*, 2020, pp. 1627–1632.

[29] S. Schleimer, D. S. Wilkerson, and A. Aiken, "Winnowing: Local Algorithms for Document Fingerprinting," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 2003, pp. 76–85.

[30] Qwen Team, "Qwen3 Technical Report," arXiv:2505.09388, 2025.

[31] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[32] OpenJS Foundation, "Node.js Documentation." [Online]. Available: https://nodejs.org/en/docs/

[33] Meta Platforms Inc., "React – The library for web and native user interfaces." [Online]. Available: https://react.dev/

[34] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation." [Online]. Available: https://www.postgresql.org/docs/16/

[35] Redis Ltd., "Redis Documentation." [Online]. Available: https://redis.io/docs/

[36] Taskforce.sh, "BullMQ Documentation." [Online]. Available: https://docs.bullmq.io/

[37] Docker Inc., "Docker Documentation." [Online]. Available: https://docs.docker.com/

[38] Traefik Labs, "Traefik Proxy Documentation." [Online]. Available: https://doc.traefik.io/traefik/

[39] Cloudflare Inc., "Cloudflare R2, Workers and Hyperdrive Documentation." [Online]. Available: https://developers.cloudflare.com/

[40] Ollama, "Ollama Documentation." [Online]. Available: https://github.com/ollama/ollama

APPENDIX A
ABBREVIATIONS

Abbreviation	Expansion
ACID	Atomicity, Consistency, Isolation, Durability
AES-GCM	Advanced Encryption Standard – Galois/Counter Mode
AI	Artificial Intelligence
API	Application Programming Interface
CD	Continuous Deployment
CI	Continuous Integration
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CSE	Computer Science and Engineering
CSV	Comma-Separated Values
CTC	Cost to Company
DB	Database
DFD	Data Flow Diagram
DNS	Domain Name System
ER	Entity–Relationship
FR	Functional Requirement
HOD	Head of the Department
HSTS	HTTP Strict Transport Security
IETF	Internet Engineering Task Force
IV	Initialisation Vector
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMS	Learning Management System
LOC	Lines of Code
MFA	Multi-Factor Authentication
ML	Machine Learning
NFR	Non-Functional Requirement
NIST	National Institute of Standards and Technology
ORM	Object–Relational Mapping
OTP	One-Time Password
OWASP	Open Worldwide Application Security Project
R2	Cloudflare R2 object storage
RBAC	Role-Based Access Control

Abbreviation	Expansion
RFC	Request for Comments
RLS	Row-Level Security
RS256	RSA Signature with SHA-256
SaaS	Software as a Service
SDG	Sustainable Development Goal
SPA	Single-Page Application
SQL	Structured Query Language
SRS	System Requirement Specification
SSO	Single Sign-On
SSRF	Server-Side Request Forgery
TC	Test Case
TLS	Transport Layer Security
TOTP	Time-based One-Time Password
TPO	Training and Placement Office
TTL	Time To Live
UAT	User Acceptance Testing
UI	User Interface
USN	University Seat Number
UUID	Universally Unique Identifier
WAF	Web Application Firewall
WP	Washington Accord Problem-solving attribute

APPENDIX B

COMPLEX ENGINEERING PROBLEM

Mapping of Complex Engineering Problems with Washington Accord WPs (WP1–WP7)

The following tables map the major project against the Washington Accord knowledge and problem-solving attributes (WP1–WP7). A tick (✓) in the *Applicable* column indicates that the competency is demonstrated in this project; the evidence for each tick is summarised after the tables.

WP Code	Description	Competencies	Applicable
WP1	In-Depth Engineering Knowledge	Uses advanced algorithms	✓
		Works with complex data structures	✓
		Applies specialised domain knowledge	✓
		Uses professional-grade tools and platforms	✓
		Combines knowledge from multiple CSE fields	✓
		Performs algorithm optimization	✓
		Refers to research papers or standards	✓
WP2	Wide-Ranging or Conflicting Technical & Non-Technical Issues	Balances security with performance	✓
		Evaluates resource limitations	✓
		Considers user experience needs	✓
		Follows privacy and security rules	✓
		Chooses scalable, maintainable solutions	✓
		Identifies and manages risks	✓
		Considers ethical impacts	✓
WP3	Abstract Thinking & Originality	Designs new algorithms or models	✓
		Creates original system designs	✓
		Introduces innovations in ML models	–
		Applies strong abstract reasoning	✓
		Develops optimization improvements	✓
		Experiments with new data representations	✓
		Adds ideas beyond standard tutorials	✓
WP4	Design and Development of Solutions	Solves real-world problems without ready-made code	✓
		Modifies existing tools/frameworks in advanced ways	✓
		Uses cutting-edge technologies	✓
		Implements algorithms/protocols from scratch	✓
		Handles incomplete or messy data	✓
		Builds non-default cloud or distributed configs	✓

WP Code	Description	Competencies	Applicable
WP5	Use of Modern Tools	Builds custom security mechanisms	✓
		Creates new database techniques	✓
		Proposes design/coding standards	✓
		Improves existing algorithms	✓
		Defines new performance benchmarks	–
		Explains modifications to standard workflows	✓
		Builds custom testing/validation tools	✓
WP6	Nature of Problem: Uncertainty & Ambiguity	Collects requirements from diverse users	✓
		Designs role-based access/permissions	✓
		Manages conflicting stakeholder needs	✓
		Builds interfaces for each stakeholder	✓
		Uses proper modelling techniques	✓
		Ensures secure data handling	✓
		Validates system with user testing	✓
WP7	Interdependence & Multidisciplinary Factors	Breaks system into layers/modules	✓
		Integrates multiple interacting modules	✓
		Connects hardware/software components	✓
		Handles advanced constraints	✓
		Uses DevOps/automation workflows	✓
		Ensures performance & reliability	✓
		Enables cross-technology integration	✓

Table 8.6: Summary of WP competencies matched

WP Code	No. of Competencies	Level
WP1	7 of 7	High
WP2	7 of 7	High
WP3	6 of 7	High
WP4	6 of 6	High
WP5	6 of 7	High
WP6	7 of 7	High
WP7	7 of 7	High

Note: Scale for mapping	Complex Engineering Project
• Low: 1–2 competencies matched • Moderate: 3–4 competencies matched • High: 5 or more competencies matched	• A project is considered complex if: • **Condition A:** At least 3 out of WP1–WP7 are **High** • **AND** • **Condition B:** At least 5 out of WP1–WP7 are **Moderate or High** • If both conditions are met → **Complex Project**

Evidence for the mapping (summary).

- **WP1:** rule-based eligibility engine, application state machine, family-based refresh-token rotation, AES-256-GCM authenticated encryption, partitioned tables and JSONB rules; references to RFC 7519, RFC 6238, RFC 9106, NIST SP 800-63B and OWASP.
- **WP2:** argon2 cost tuned against a 2-vCPU server with a concurrency semaphore (security versus availability); personal data deliberately never sent to the language model (privacy and ethics); fail-closed RLS.
- **WP3:** original tiered two-offer placement policy, versioned ciphertext format (a `v1:` prefix followed by IV, tag and ciphertext), transaction-local tenant context for RLS; no new ML model is proposed (an existing small model is used), hence one competency is not claimed.
- **WP4:** SSO one-time-code protocol, SSRF guard with DNS-rebinding protection, CSV import of messy institutional data, Cloudflare Tunnel, PgBouncer and Traefik configurations.
- **WP5:** custom step-up authentication, DB-level attendance de-duplication with advisory locks and triggers, project coding conventions and a custom security regression suite; no new public benchmark was defined.
- **WP6:** requirements from principals, HODs, teachers, placement officers and students; nine default roles plus custom roles; acceptance walkthrough on the production system.
- **WP7:** layered services, barcode/ID-card scanners (USB serial and camera) integrated with the attendance module, Docker/Dokploy/GitHub Actions, integration with the examination platform and a self-hosted LLM.

Conclusion: All seven attributes are rated **High**; therefore Condition A (at least 3 High) and Condition B (at least 5 Moderate or High) are both satisfied, and the project is considered a **Complex Engineering Project**.

APPENDIX C

Sustainable Development Goal Addressed

Table 8.7: Mapping of the project with UN Sustainable Development Goals

SDG Goal	Target Description	Justification / Mapping Explanation	Expected Impact / Outcome	Level
SDG 4 – Quality Education	Target 4.4: Increase skills for employment	Integrates coding assessments, preparation resources, resume generation and attendance/engagement tracking that highlight skill gaps early.	Improved job readiness and better-aligned student skills.	High
SDG 8 – Decent Work and Economic Growth	Target 8.6: Reduce youth not in employment, education or training	Provides structured access to on-campus and off-campus opportunities with transparent, rule-based eligibility and status tracking.	Higher placement rates and fairer hiring processes.	High
SDG 9 – Industry, Innovation and Infrastructure	Target 9.b / 9.c: Domestic technology development; access to ICT	A locally built, low-cost multi-tenant SaaS on modern container infrastructure that many colleges can share.	Affordable digital infrastructure for institutions.	High
SDG 10 – Reduced Inequalities	Target 10.2: Promote social and economic inclusion	Every eligible student is notified automatically; the same published rules and policies apply to everyone.	Reduced bias and equal access to opportunities.	Moderate
SDG 16 – Peace, Justice and Strong Institutions	Target 16.6: Effective, accountable and transparent institutions	Append-only audit logs, role-based permissions, policy acknowledgement and encryption of personal data.	Accountable, privacy-respecting administration.	Moderate
SDG 17 – Partnerships for the Goals	Target 17.6 / 17.17: Knowledge sharing and partnerships	Connects colleges, recruiters, trainers and the examination platform through shared workflows.	Stronger academia–industry collaboration.	Moderate

Project Guide,
Prof. Beerappa Belasakarge
Assistant Professor
Dept. of CSE

APPENDIX D

Research Publications/Patent

"FortifyHub: Capacity, Scoring and Integrity Analysis of a Serverless Platform for Proctored Online Examinations"

Jayanth Gopala V, Leela Venkata Sai Sumanth E, Nagarjun Gowda K N, Nakul B

FortifyHub: Capacity, Scoring and Integrity Analysis of a Serverless Platform for Proctored Online Examinations

Jayanth Gopala V
Dept. of CSE
BMS Institute of Technology and Management
Bengaluru, India

Leela Venkata Sai Sumanth E
Dept. of CSE
BMS Institute of Technology and Management
Bengaluru, India

Nagarjun Gowda K N
Dept. of CSE
BMS Institute of Technology and Management
Bengaluru, India

Nakul B
Dept. of CSE
BMS Institute of Technology and Management
Bengaluru, India

***Abstract*—A semester-end online examination loads a web platform in an unusual way: thousands of candidates begin within the same minute, each saved answer must survive an unreliable network connection, and once the exam closes the invigilator expects some evidence about possible misconduct. This paper describes FortifyHub, a multi-tenant examination platform built on Cloudflare Workers, Durable Objects and a managed PostgreSQL service, and examines four of its design decisions with elementary mathematics. We show when negative marking stops being fair, and that the zero floor in our grader gives a blind guesser about 2.3 marks out of 100. We size a single-writer exam coordinator and find that keystroke telemetry, not answers, limits one instance to a few hundred candidates. We measure a 150-fold cut in database statements from batched persistence, and a 180-fold write amplification hidden inside it. Finally we show that a match-fraction rule for flagging answer copying flags about one honest pair in six under a plausible ability model, while a wrong-answer statistic lifts the detection rate at a 0.1% false-positive rate from 0.8% to 49% in simulation. Timings of the shipped similarity code, and a rewrite that is 198 times faster at 400 candidates, are reported. Results are analytical, simulated or measured on synthetic data; no field data are claimed.**

***Index Terms*—online examination, serverless computing, Durable Objects, formula scoring, answer-copying detection, PBKDF2.**

I. Introduction

Universities in India run semester-end examinations for tens of thousands of candidates at once, and a growing share of internal and certification tests have moved to browsers. A platform serving them faces three problems that rarely appear together in ordinary web applications. The first is a synchronized burst: nearly every candidate presses "Start" within the same short window, and nearly every candidate submits in the final minutes. The second concerns durability, and the requirement is uneven: an answer lost because a connection dropped becomes a formal grievance, while a lost heartbeat costs nothing. The third is accountability. Because results can be challenged, the platform must be able to justify each mark and give the examiner a reasoned view on possible copying.

We developed FortifyHub at Arventiq Labs for institutes that conduct objective, fill-in-the-blank and coding tests. The platform is multi-tenant, with organizations, exams, questions, attempts and certificates as its main entities, and it is deployed on Cloudflare Pages and Workers with PostgreSQL as the backing store. Published descriptions of exam platforms usually end with the architecture. Our interest is in the limits of individual design choices, since a short calculation can often show where a choice fails before any load test is run. In what follows we take the code base in its current form and quantify four such decisions.

Contributions.

1) A formula-scoring analysis that gives the fair penalty for n options, shows that a customary penalty of $1/4$ mark rewards guessing on four-option items, and quantifies the bias that a zero floor adds even at the fair penalty (Section IV).
2) A memory model of the per-exam Durable Object, showing that telemetry dominates and fixing the number of shards needed for 5000 concurrent candidates (Section V).
3) A persistence-cost model: a 150-fold reduction in statement rate from batching, and a write amplification of $T/\left(2T_f\right)$ from rewriting a growing telemetry array on every flush (Section V-B).
4) An analysis of the answer-similarity detector, with a false-flag calculation, a wrong-answer alternative evaluated by Monte Carlo ROC, and measured runtimes of the shipped implementation (Section VI).

II. Background and Related Work

Formula scoring. Penalizing wrong answers to discourage blind guessing is old; Lord [1] and Budescu and Bar-Hillel [2] discuss when it is neutral for a risk-neutral candidate and how risk attitude changes behaviour. We use only the risk-neutral result.

Copying detection. Frary, Tideman and Watts [3] proposed indices based on the chance of the observed number of identical answers given each candidate's own response pattern, and Wesolowsky [4] refined the idea for multiple-choice tests. For source code, winnowing [5] selects a provable subset of k-gram hashes, and the resemblance measure of Broder [6] underlies the Jaccard comparisons we also use.

Proctoring. Camera-based proctoring [7, 8] is outside our scope. What FortifyHub does record is window-focus and fullscreen violations together with keystroke timing; keystroke dynamics have long been studied as a biometric signal [9].

Platform. A Durable Object is a single-threaded actor, addressed by name, with its own memory and storage [12]. Each isolate has a memory limit and each request a CPU-time budget [13]; the first governs the capacity analysis in Section V-A and the second the hashing analysis in Section

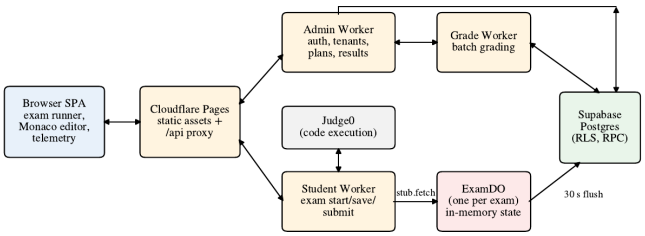


Fig. 1. Request path of FortifyHub.

V-C. Submitted code is executed in Judge0 [14], and password storage follows PBKDF2 [10] and the OWASP guidance [11].

III. System Overview

The request path is shown in Fig. 1. The exam runs in a single-page application served from Cloudflare Pages, which handles question navigation, provides a Monaco editor for coding items, monitors fullscreen state and window focus, and records keystrokes. Requests go either to an admin Worker, responsible for authentication, tenants, plans and results, or to a student Worker, responsible for start, save, heartbeat and submit. On the hot path the student Worker never queries the database; it forwards each call to `ExamDO`, a Durable Object created once per active exam. ExamDO keeps answers, heartbeats, a capped violation log and keystroke events in memory and is designed to respond in under two milliseconds. Dirty state is written to PostgreSQL by a timer every $T_f = 30\,\mathrm{s}$, in batched statements of at most $B = 200$ rows, and again when an attempt is submitted. Grading is done either inside the object or by a stored procedure; coding answers go to Judge0.

Several constants are used in the later analysis. The client sends a heartbeat every 30 s, and the server still accepts saves for 30 s after the deadline. For each attempt the object retains the most recent 5000 keystroke events and 50 violations. Memory held for a submitted attempt is released 90 s after submission; the delay prevents late telemetry from replacing durable data with a shorter array. After 8 warnings the client submits automatically. Plans (free trial, basic, standard, premium) cap exams, students, faculty and concurrency per tenant.

IV. Scoring Models

A. Negative marking

Let an item carry m marks, n options, and a penalty p for a wrong selected answer. Unanswered items score zero, as in the grader. A candidate who picks uniformly among all n options gains

$$\mathbb{E}[\Delta] = \frac{m}{n} - \frac{n-1}{n}\,p, \tag{1}$$

which vanishes at $p^* = m/(n-1)$. For $n = 4$, $m = 1$ this is $p^* = 1/3$. The common rule of $1/4$ mark is below it, so blind guessing gains $0.25 - 0.75 \times 0.25 = 0.0625$ marks per item, or about 6.2 marks on a 100-item paper (Monte Carlo mean 6.24). A candidate who can discard j options before guessing gains in expectation exactly when

$$p < \frac{m}{n-j-1}, \tag{2}$$

so at p^* any elimination makes guessing profitable and the penalty never discourages informed guessing, only blind guessing. Fig. 2(a) plots Eq. (1).

B. Effect of the zero floor

The grader reports $\max(0, round(S))$ for the raw score S. With $c \sim Bin(q, 1/n)$ correct guesses, $S = c(m+p) - pq$ has mean $\mathbb{E}[S] = 0$ at $p = p^*$ and standard deviation

$$\sigma_S = (m+p)\sqrt{q\,\frac{1}{n}\left(1-\frac{1}{n}\right)}.$$

Treating S as normal, the floored mean is $\mathbb{E}[S^+] = \sigma_S/\sqrt{2\pi}$. For $q = 100$, $n = 4$, $p = 1/3$ we get $\sigma_S = 5.77$ and $\mathbb{E}[S^+] = 2.30$ marks; a simulation of 4×10^5 blind guessers gives 2.29. About 4.5% of them report 10 or more out of 100 (Fig. 2(b)). For candidates who genuinely attempt the paper the floor does not affect ranking. Its effect is confined to the bottom of the distribution, where a penalty that is fair in expectation becomes a small positive expected score for candidates who only guess. An institute that wants a strictly neutral rule should either report signed scores to examiners or use the floor only at display time.

C. Coding items

Coding answers are scored by the judge. For weighted test cases with weights w_i and pass indicators x_i the question fraction is

$$s = \frac{\sum_i w_i x_i}{\sum_i w_i},$$

and when subtasks with point values P_k are defined, a subtask scores only if every case in its group passes: $s = \sum_k P_k \prod_{i \in G_k} x_i \,/\, \sum_k P_k$. Limits scale by language: the time limit is $T_L = \mu_L^t T_{\text{base}}$ and the memory limit $\min(\mu_L^m M_{\text{base}}, 512000\ \text{KB})$, with multipliers (μ^t, μ^m) of $(1.0, 1.0)$ for C/C++, $(2.5, 2.5)$ for Java, $(3.0, 3.0)$ for JavaScript and $(4.0, 2.0)$ for Python. A base limit of 2000 ms therefore gives a Python submission 8 s. The weights are the examiner's responsibility; the platform's contribution is that the judge, not the browser, computes x_i.

V. Exam Coordinator: Capacity and Persistence

A. Memory bound

Because `ExamDO` is a single isolate, its resident set bounds how many attempts one instance can hold. For one in-progress attempt,

$$m = q\,b_a + K\,b_k + V\,b_v,$$

with q questions at $b_a \approx 150\,\mathrm{B}$ per stored answer, up to $K = 5000$ keystroke events at b_k bytes each, and up to $V = 50$

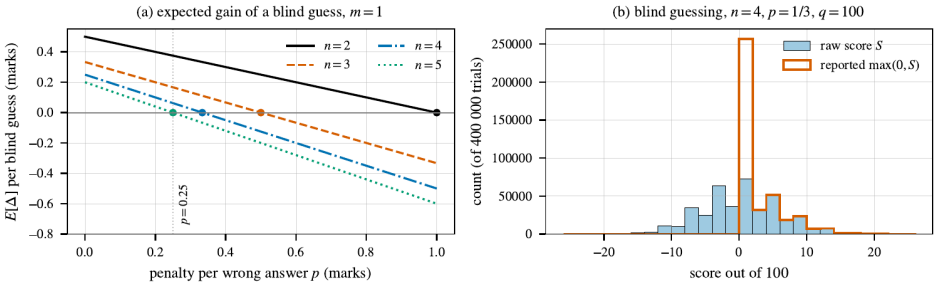


Fig. 2. (a) Expected gain of a blind guess, $m = 1$; (b) blind guessing, $n = 4$, $p = 1/3$, $q = 100$.

TABLE 1
ATTEMPTS PER OBJECT AND SHARDS FOR 5000 CANDIDATES ($\varphi M = 96$ MB, 20 KB FOR ANSWERS AND VIOLATIONS)

Telemetry buffer	$N_{\max}$	Shards S	Basis
150 KB	578	9	assumed low case
320 KB	289	18	5000×64 B
1.5 MB	64	79	worst case in code notes

violations at about $b_v = 100$ B. For $q = 100$ and a nominal $b_k = 64$ B (a small object with V8 overhead) this is $15+320+5 = 340$ KB. The telemetry term is 94% of the total. With an isolate limit $M = 128$ MB of which a fraction $\varphi = 0.75$ is usable after runtime, driver and request buffers,

$$N_{\max} = \left\lfloor \frac{\varphi M}{m} \right\rfloor, \qquad S = \left\lceil \frac{N}{N_{\max}} \right\rceil.$$

Table 1 gives the result. At the nominal size one object holds 289 full buffers, so 5000 candidates need $S = 18$ shards; at the worst case of 1.5 MB per attempt the number is 79. Candidates who have not yet typed much hold much less, so the true requirement lies between the rows and depends on how early buffers fill. The point stands that answers are irrelevant to the sizing and telemetry decides it. Two practical remedies follow: shard by a hash of the candidate identifier, which keeps each shard single-threaded and race-free, or stream telemetry to storage at each flush and keep only a bounded tail in memory.

The 90 s reclaim rule matters for the same reason. If submissions arrive at rate λ_s near the deadline, resident attempts are $A(t) + 90\lambda_s$, and at $\lambda_s = 20\,\mathrm{s}^{-1}$ that adds 1800 attempts that have already finished. Clearing sooner would shrink the peak, but the delay exists to stop late telemetry from truncating durable data, so the cost is deliberate.

B. Statement rate and write amplification

If saves and heartbeats went straight to the database, each would cost one write. Suppose each candidate saves at rate $\lambda_a = 1/60\,\mathrm{s}^{-1}$ and sends a heartbeat at $\lambda_h = 1/30\,\mathrm{s}^{-1}$. The statement rate is then $N(\lambda_a + \lambda_h)$, or $250\,\mathrm{s}^{-1}$ at $N = 5000$. ExamDO keeps heartbeats in memory and writes only dirty answers and keystroke arrays, packing B rows into each statement once every T_f:

$$\lambda_{\text{batch}} = \frac{2}{T_f} \left\lceil \frac{N}{B} \right\rceil,$$

giving $1.67\,\mathrm{s}^{-1}$ at $N = 5000$, a $150\times$ reduction (Fig. 4a). The remaining load is governed mainly by the ceiling term and rises in steps as the cohort passes each multiple of 200 candidates; at this rate a pooled PostgreSQL connection is far from saturated.

Batching has a less visible cost. Keystrokes are stored as one JSON array in a single column, and every flush rewrites that array in full. If events accumulate at rate r and take b bytes serialized, then for an exam of length T the bytes written per candidate are

$$\sum_{f=1}^{T/T_f} \min\left(rfT_f,\, K\right) b \;\approx\; \frac{r\,b\,T^2}{2T_f},$$

against $r\,b\,T$ if each event were written once, an amplification of

$$A \approx \frac{T}{2T_f} \tag{9}$$

while the buffer is below its cap. For $T = 3$ h and $T_f = 30$ s, $A = 180$. With $r = 0.3\,\mathrm{s}^{-1}$ and $b = 22$ B each candidate causes about 12.9 MB of writes instead of 71 KB, which is 64 GB for 5000 candidates, as the exact sum confirms ($A = 180.5$). Shortening T_f increases the amplification (Fig. 4b): a 30 s interval suits answers but not telemetry. If each flush instead appends a new chunk, stored as one row or object, every event is written once and $A = 1$.

C. Password hashing under a CPU budget

Passwords are stored with PBKDF2-HMAC-SHA256 at $I = 10^5$ iterations; records carry their own iteration count, so I can rise without a migration. Older accounts were hashed with a single salted SHA-256 and are upgraded at their next successful login. Take a single GPU that computes $R = 2 \times 10^{10}$ SHA-256 compressions per second, an order of magnitude we assume rather than measure. PBKDF2 costs about $2I$ compressions per guess, so exhausting a keyspace of size $|\mathcal{K}|$ takes

$$t = \frac{2I\,|\mathcal{K}|}{R}.$$

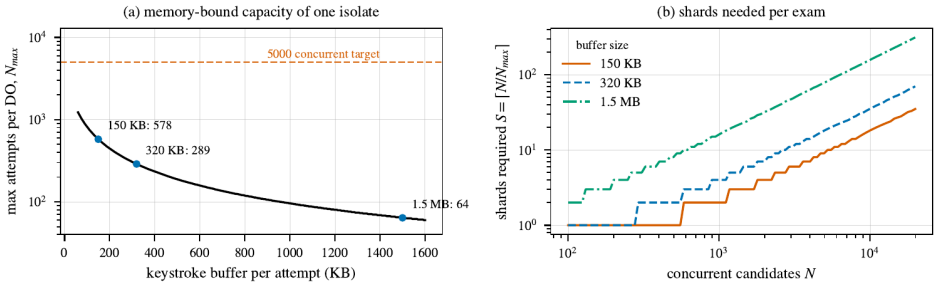


Fig. 3. (a) Memory-bound capacity of one isolate; (b) shards needed per exam.

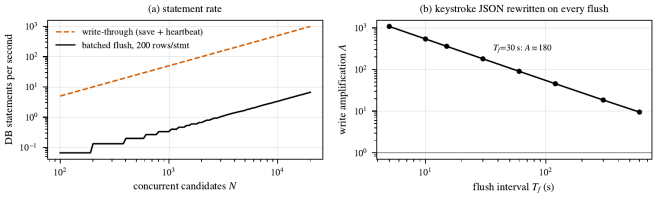


Fig. 4. (a) Statement rate; (b) keystroke JSON rewritten on every flush.

TABLE 2
TIME TO EXHAUST A KEYSPACE ON ONE GPU ($R = 2 \times 10^{10}\,\mathrm{s}^{-1}$)

Keyspace	Single SHA-256	$I = 10^5$	$I = 6 \times 10^5$
10^9 (wordlist)	0.05 s	2.8 h	16.7 h
36^8	2.4 min	327 d	5.4 y
62^8	3.0 h	69 y	415 y

Table 2 shows the effect: moving from one SHA-256 to 10^5 iterations multiplies attacker time by 2×10^5, turning a 2.4-minute search of 36^8 into 327 days.

Why not the OWASP figure of 600 000 [11]? Workers have a per-request CPU budget B_{cpu} (50 ms by default on the paid plan). According to benchmarks recorded in the code base, hashing took 23 ms at 10^5 iterations, so $c \approx 0.23\,\mu$s per iteration, and roughly 100 ms at 6×10^5. Reserving half of the budget for the rest of the request gives an upper bound of

$$I_{\max} = \left\lfloor \frac{h\, B_{\text{cpu}}}{c} \right\rfloor = \left\lfloor \frac{0.5 \times 50\ \text{ms}}{0.23\,\mu\text{s}} \right\rfloor \approx 1.09 \times 10^5.$$

The deployed iteration count is therefore already at this bound, and any increase would first need a higher configured CPU limit. Passwords drawn from a wordlist remain weak regardless, as Table 2 shows: at 10^5 iterations a 10^9-word list is exhausted in under three hours.

VI. Integrity Analytics

Two detectors run in the browser of an examiner: a pairwise similarity scan over submitted attempts, and a keystroke burst check that adds to a pair's score. Neither decides anything. Both rank pairs for a human to read.

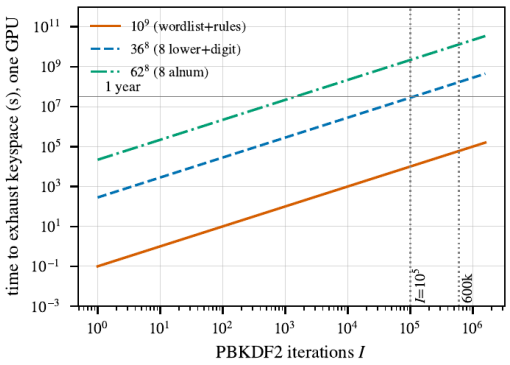


Fig. 5. Time to exhaust three keyspaces against PBKDF2 iteration count, for one GPU.

A. Match fraction and its false flags

For each pair the scan averages a per-question similarity over questions both candidates answered: exact match for objective items, 2-shingle Jaccard similarity for fill-in answers, and a winnowed fingerprint overlap for code. A pair is flagged when this mean reaches a threshold τ, default 0.6, with 0.1 added per candidate whose keystroke burst score exceeds 50.

Consider only objective items. Two independent candidates with abilities a_1, a_2 (probability of a correct answer) who spread wrong answers evenly across $n - 1$ distractors agree on an item with probability

$$\pi = a_1 a_2 + \frac{(1 - a_1)\,(1 - a_2)}{n - 1}.$$

The number of agreements $X \sim Bin(q, \pi)$, so an honest pair is flagged with probability $P\,(X \geq \lceil \tau q \rceil)$. For two candidates of ability 0.8 on a four-option test, $\pi = 0.653 > \tau$, so the *typical* honest pair is flagged. Table 3 and Fig. 6(a) give the rest. For a class of 60 (1770 pairs) at $a = 0.7$ and $q = 50$ we expect about 285 honest pairs above the line. With abilities drawn independently from $Beta(6.5, 3.5)$, mean 0.65, 17.5% of 6×10^4 simulated honest pairs reach $\tau = 0.6$.

The cause is simple. Agreement on *correct* answers is expected between good students and carries no evidence of copying. This is the observation behind the error-similarity indices of Frary et al. [3].

TABLE 3
P(HONEST PAIR FLAGGED AT $\tau = 0.6$), EQUAL ABILITIES a, $n = 4$

	$a = 0.5$	0.6	0.7	0.8	0.9
$q = 30$	0.0025	0.030	0.244	0.792	0.998
$q = 50$	0.0001	0.006	0.161	0.827	1.000
$q = 100$	4×10^{-8}	0.0001	0.066	0.889	1.000

TABLE 4
DETECTION OF SIMULATED COPYING ($q = 50$): AUC AND TRUE-POSITIVE RATE AT FALSE-POSITIVE RATE 10^{-3}

c	Statistic	AUC	TPR@10^{-3}	TPR@10^{-2}
0.15	match fraction	0.68	0.003	0.034
0.15	wrong-answer z	0.85	0.106	0.280
0.30	match fraction	0.83	0.008	0.085
0.30	wrong-answer z	0.96	0.494	0.715

B. A wrong-answer statistic

Let w_i be candidate i's number of wrong answers and W the number of items on which both are wrong *and identical*. Under independence, with plug-in estimates of ability from observed error rates,

$$W \sim Bin\left(q, \widehat{\rho}\right), \qquad \widehat{\rho} = \frac{\left(w_1/q\right)\left(w_2/q\right)}{n-1},$$

and the evidence score is $z = -\log_{10} P\left(W \geq W_{\text{obs}}\right)$. This conditions on how well each candidate did, so strong honest pairs are not penalized.

We evaluated both statistics on simulated pairs with $q = 50$, $n = 4$, abilities as above, 6×10^4 honest pairs, and 1.5×10^4 pairs per copying level in which the second candidate copies each answer from the first with probability c. Table 4 and Fig. 6(b) show that the match fraction is a weak detector: at a 0.1% false-positive rate it finds under 1% of pairs with $c = 0.30$, whereas z finds 49%. A fixed cut of $z \geq 3$ flagged 0.005% of honest pairs. The simulation assumes independent questions and evenly attractive distractors. Real distractors differ in appeal and real items differ in difficulty, both of which raise honest agreement, so the numbers show direction and rough size only. A pilot on past examinations would give the calibration that we lack.

C. Code fingerprints and keystroke bursts

Code is normalized (comments removed, literals replaced by tokens, identifiers replaced by a placeholder), hashed in 5-token grams, and winnowed with window $w = 4$. Winnowing guarantees that any shared run of at least $t = w + k - 1 = 8$ tokens produces a common fingerprint and ignores runs shorter than $k = 5$ [5]. Fingerprint density for random hashes is $2/(w+1) = 0.4$; simulation with 200 random streams gave 0.400. The same normalization makes the method immune to variable renaming, and also makes unrelated solutions to a short problem look alike, so boilerplate-heavy tasks need a higher threshold or a template subtraction.

The keystroke check treats a run of at least four inter-key gaps under 50 ms as a burst and scores the share of events inside bursts. Model gaps as lognormal with median μ and shape σ. The per-gap probability of falling under 50 ms is $g = \Phi\left(\left(\ln 50 - ln\mu\right)/\sigma\right)$, and for a buffer of $K = 2000$ gaps the chance of at least one false burst is approximately $1 - exp\left[-K(1-g)g^4\right]$. Table 5 shows this is negligible for ordinary typing and about 0.6% for very fast typists. Since the score must exceed 50 (more than half of all events inside bursts), human typing essentially never triggers it. Pasted text does trigger it, and some mobile keyboards with autocomplete may as well. This is a further reason why the $+0.1$ bonus should be regarded as uncalibrated.

TABLE 5
FALSE-BURST PROBABILITY FOR LOGNORMAL TYPING, $K = 2000$

Typist	μ (ms)	σ	g	P(false burst)
Casual	250	0.6	0.0037	3.6×10^{-7}
Average	180	0.5	0.0052	1.5×10^{-6}
Fast	100	0.4	0.0416	5.7×10^{-3}

D. Cost of the pairwise scan

With N candidates the scan visits $N(N-1)/2$ pairs. In the shipped implementation each answer is matched to the partner's answer by a linear search inside the loop over answers, so one pair costs $O\left(q^2\right)$. Each candidate's keystroke log is also re-analysed for every pair, which adds $O(K)$ per pair and gives a total cost of $O\left(N^2\left(q^2 + K\right)\right)$.

We timed the actual `Plagiarism.compare` in Node.js 22 on synthetic objective data ($q = 100$, 2000 keystroke events per candidate, one thread, best of three runs). Time grew linearly in the number of pairs (log-log slope 1.04), at about $116\,\mu$s per pair: 9.25 s for $N = 400$. Extrapolated to $N = 5000$ this is roughly 24 minutes of blocked browser thread, before counting the memory for a per-question record for each of 1.25×10^7 pairs. A rewrite that precomputes per-candidate answer vectors and keystroke scores, compares flat arrays, and keeps only pairs above threshold takes 46.8 ms for $N = 400$, a speedup of $198\times$ (Fig. 7). It is $O\left(Nq + N^2 q\right)$ with a small constant. Beyond a few thousand candidates the quadratic term returns, and indexing on (question, wrong option) pairs, so that only candidates sharing at least one wrong answer are ever compared, is the natural next step.

VII. DISCUSSION AND LIMITATIONS

What the analysis changes. Three of the results change how the system should be operated. The grader's floor and the default penalty need an explicit policy. Before 5000-candidate exams are run, the coordinator must be sharded or telemetry streamed to storage, since the memory estimate in Section V-A does not support a single instance. Finally, the similarity report should rank pairs by a wrong-answer statistic instead of a raw match fraction. Each of these changes is small, and far less costly than finding the problem during a live examination.

Threats to validity. No field data were available for this study. The values in Table 3 and Table 4, and the false-flag rate, rest on assumed ability distributions and on independence

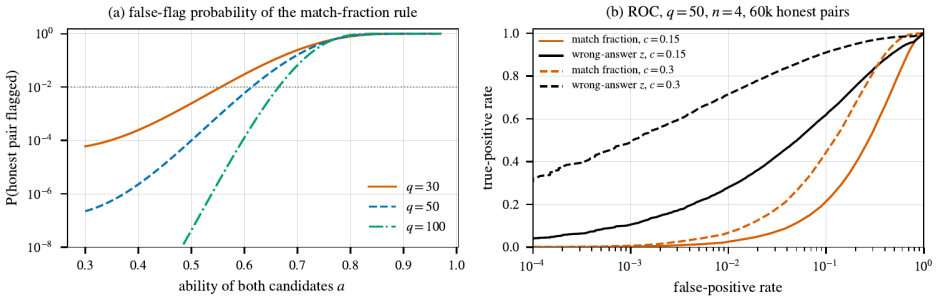


Fig. 6. (a) False-flag probability of the match-fraction rule; (b) ROC, $q = 50$, $n = 4$, 60k honest pairs.

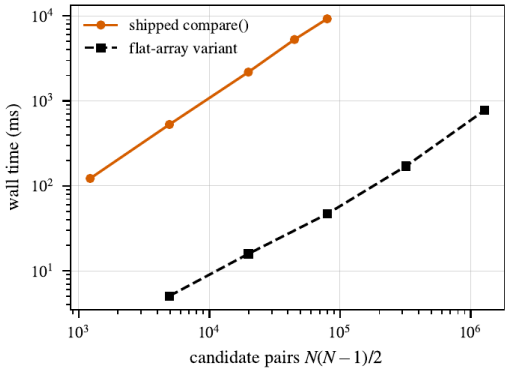


Fig. 7. Measured wall time of the similarity scan against number of pairs, synthetic data, $q = 100$, $K = 2000$.

between questions. The per-event size b_k, the usable memory fraction φ, the save rate λ_a and the GPU rate R are likewise assumed, and each is stated where it is used. All runtime figures were measured on one machine with synthetic data and cannot be taken to predict behaviour in browsers on laboratory hardware. The capacity note in the code was revised after an internal review in September 2026; it accounts for telemetry, yet still concludes that 5000 buffers fit in one isolate. Under our model this holds only when the average buffer is well below the 150–320 KB range, so a load test with realistic typing is needed before relying on it.

Ethics. Keystroke timing and violation logs are personal data. Limits on retention, notice to candidates, and the rule that a flag alone never justifies a sanction belong to the deployment policy rather than to the algorithms described here.

VIII. Conclusion

Keeping the hot path in memory makes a serverless exam platform inexpensive to operate, but it shifts the cost to isolate memory limits and to the way state is flushed. Short calculations located four problems in FortifyHub: a fair penalty of $1/(n-1)$ with a floor that still favours guessers, an $N_{\max}$ set by telemetry, a write amplification of $T/(2T_f)$, and a similarity threshold that flags many honest pairs of strong students. As next steps we plan a pilot on archived paper-based and online examinations to calibrate the ability model and the thresholds, a move to append-only telemetry chunks, and a server-side wrong-answer index for large cohorts.

References

[1] F. M. Lord, “Formula scoring and number-right scoring,” *J. Educational Measurement*, vol. 12, no. 1, pp. 7–11, 1975.

[2] D. Budescu and M. Bar-Hillel, “To guess or not to guess: A decision-theoretic view of formula scoring,” *J. Educational Measurement*, vol. 30, no. 4, pp. 277–291, 1993.

[3] R. B. Frary, T. N. Tideman, and T. M. Watts, “Indices of cheating on multiple-choice tests,” *J. Educational Statistics*, vol. 2, no. 4, pp. 235–256, 1977.

[4] G. O. Wesolowsky, “Detecting excessive similarity in answers on multiple choice exams,” *J. Applied Statistics*, vol. 27, no. 7, pp. 909–921, 2000.

[5] S. Schleimer, D. S. Wilkerson, and A. Aiken, “Winnowing: Local algorithms for document fingerprinting,” in *Proc. ACM SIGMOD*, 2003, pp. 76–85.

[6] A. Z. Broder, “On the resemblance and containment of documents,” in *Proc. Compression and Complexity of Sequences*, IEEE, 1997, pp. 21–29.

[7] Y. Atoum, L. Chen, A. X. Liu, S. D. H. Hsu, and X. Liu, “Automated online exam proctoring,” *IEEE Trans. Multimedia*, vol. 19, no. 7, pp. 1609–1624, 2017.

[8] K. Hylton, Y. Levy, and L. P. Dringus, “Utilizing webcam-based proctoring to deter misconduct in online exams,” *Computers & Education*, vol. 92–93, pp. 53–63, 2016.

[9] F. Monrose and A. D. Rubin, “Keystroke dynamics as a biometric for authentication,” *Future Generation Computer Systems*, vol. 16, no. 4, pp. 351–359, 2000.

[10] K. Moriarty, B. Kaliski, and A. Rusch, “PKCS #5: Password-based cryptography specification version 2.1,” RFC 8018, IETF, 2017.

[11] OWASP Foundation, “Password Storage Cheat Sheet.” [Online]. Available: https://cheatsheetseries.owasp.org

[12] Cloudflare, “Durable Objects documentation.” [Online]. Available: https://developers.cloudflare.com/durable-objects/

[13] Cloudflare, “Workers platform limits.” [Online]. Available: https://developers.cloudflare.com/workers/platform/limits/

[14] Judge0, “Open-source online code execution system.” [Online]. Available: https://judge0.com

APPENDIX E

Plagiarism Report (Similarity Report and AI Detection)

The pages that follow contain the plagiarism check of this major project report:

the Turnitin similarity report and the AI-writing detection report.

APPENDIX F

API Examples

Login and Token Refresh

```
POST /auth/login
{ "email": "officer@college.edu", "password": "********" }

200 OK   (MFA not enrolled)
{ "status": "ok", "access_token": "<RS256 JWT, 15 min>",
  "refresh_token": "<opaque, 7 days>", "user": { ... } }

200 OK   (MFA enrolled)
{ "status": "mfa_required", "challenge_token": "<short-lived>" }

POST /auth/refresh
{ "refresh_token": "<previous token>" }
-> new access_token + refresh_token; previous refresh token revoked
-> reuse of a revoked token: 401 REFRESH_TOKEN_REUSED (family revoked)
```

Applying Students to a Drive

```
POST /api/jobs/{jobId}/apply
Authorization: Bearer <access token>
{ "student_ids": ["<uuid-1>", "<uuid-2>", "<uuid-3>"] }

201 Created
{ "applied": 1, "rejected": 2, "requested": 3,
  "accepted": [ { "student_id": "<uuid-1>", "status": "applied", ... } ],
  "ineligible_or_duplicate": [
    { "student_id": "<uuid-2>", "reason": "NO_RESUME" },
    { "student_id": "<uuid-3>", "reason": "CATEGORY_CAP_REACHED" } ] }
```

Structured Error Format

```
403 Forbidden
{ "error": { "code": "PERMISSION_DENIED",
             "message": "Missing permission create on placement.applications" } }
```

APPENDIX G

Local Development Setup

All credentials are read from a git-ignored `.env` file; the compose file contains no secrets.

```
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER:     ${POSTGRES_USER:?missing POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?missing POSTGRES_PASSWORD}
      AUTH_DB_NAME:      ${AUTH_DB_NAME:-auth_db}
      AUTH_DB_USER:      ${AUTH_DB_USER:-auth_app}
      CORE_DB_NAME:      ${CORE_DB_NAME:-core_db}
      CORE_DB_USER:      ${CORE_DB_USER:-core_app}
    ports:
      - "${POSTGRES_HOST_PORT:-15432}:5432"
    volumes:
      - ./scripts/postgres-init.sh:/docker-entrypoint-initdb.d/00-init.sh:ro
  redis:
    image: redis:7-alpine
    command: ["sh", "-c", "redis-server --requirepass \"$$REDIS_PASSWORD\" --
    appendonly yes"]
  minio:                        # local S3-compatible stand-in for Cloudflare R2
    image: minio/minio:latest
```

APPENDIX H

Security Controls Checklist

Table 8.8: Mapping of OWASP API Security Top 10 (2023) risks to implemented controls

OWASP API risk	**Control in FortifyHub**
API1 Broken Object Level Authorization	`scopedWhere` on every query, tenant validation of foreign keys, PostgreSQL RLS
API2 Broken Authentication	argon2id, RS256 JWT, revocation list, refresh rotation with reuse detection, MFA, passkeys, lockout
API3 Broken Object Property Level Authorization	Joi `stripUnknown`, explicit response mappers, audit redaction
API4 Unrestricted Resource Consumption	Per-endpoint rate limits, per-tenant concurrency caps, file size and storage quotas
API5 Broken Function Level Authorization	`requirePermission(page, action)` on every route, step-up for sensitive actions
API6 Unrestricted Access to Sensitive Business Flows	Policy acknowledgement, offer-tier gates, rate-limited writes
API7 Server Side Request Forgery	SSRF guard: HTTPS only, public IPs only, connect-time pinning, no redirects
API8 Security Misconfiguration	Helmet headers, strict CORS, `/internal/*` blocked at proxy, secrets only in environment
API9 Improper Inventory Management	Single architecture specification, versioned migrations, page catalogue
API10 Unsafe Consumption of APIs	Size/time caps on external feeds and AI responses; graceful fallbacks